Shunyaya Symbolic Mathematical Temperature (SSMT) — Zero-Centric Standard

Status: Public Research Release (v2.3)
Date: October 30, 2025
Caution: Research/observation only. Not for critical decision-making.

Citation: When implementing or adapting, cite the concept name "Shunyaya Symbolic Mathematical Temperature (SSMT)" as the origin of the symbolic mathematical temperature approach.

Purpose

Make temperature a unitless, comparable, bounded, auditable signal that everyone can read the same way, across devices, fleets, cities, and planets. SSMT is a **representation-only standard**: it replaces raw °C/°F/K with symbolic contrasts and optional bounded dials, without forcing any downstream model or vendor stack.

SSMT does not change physics. It standardizes how temperature is expressed and governed.

Key idea

1. **Convert once to Kelvin** (`T_K`).
2. **Apply a Kelvin floor** (`T_K := max(T_K, eps_TK)`).
3. **Map to a unitless contrast** `e_T` using a declared lens and anchors.
4. **Optionally map to a bounded dial** such as `a_T := tanh(c_T * e_T)` so it can be pooled, alerted on, and audited safely.

After this, machine logic consumes `e_T`, `a_T`, `a_phase`, or `a_phase_fused` instead of raw °C/°F/K.

Lineage (zero-centric)

• **Shunyaya Symbolic Mathematics (SSM)** introduced the zero-centric pattern: represent any quantity as a contrast around a published baseline, and (optionally) a bounded alignment dial that always lives in (-1,+1).
• **Shunyaya Symbolic Mathematical Temperature (SSMT)** applies that pattern to temperature: convert to Kelvin, express "how far from nominal" as a portable contrast, and expose a smooth near-pivot dial instead of brittle hard cutoffs.

• **Shunyaya Symbolic Mathematical Chemistry (SSM-Chem)** uses the same approach for thermodynamics and reaction energetics. When combined, SSMT can act as the environmental gate for chemical stability and safety, for example `RSI_env := g_t * RSI`, where `g_t` is derived from temperature state and hysteresis.

Why SSMT now

• **Eliminate unit drama.** After a single to-Kelvin step, `e_T` means the same thing everywhere — across vendors, countries, subsystems, and APIs.
• **Be phase-aware.** A smooth near-pivot dial replaces brittle `32 F vs 0 C` branching at freeze, melt, warp, boil, gel, etc.
• **Make ML and analytics cleaner.** `e_T` is unitless and centered. `a_T` and `a_phase` live in (-1,+1). They become stable features that can be pooled, ranked, alerted on, and audited.
• **Enable audit and safety.** A small manifest makes intent explicit. Symbol-only alert rules become replayable and vendor-neutral.
• **Reduce cost.** One canonical signal layer replaces duplicated °C/°F code paths, ad-hoc alert thresholds, and per-site tuning.

Global adoption

SSMT is designed for immediate, independent adoption. The lightest profile (S1 interchange) only needs two math steps — Kelvin conversion and a lens — and can emit a minimal payload:

```
{ timestamp_utc, e_T, manifest_id, health }
```

This requires:
• No central server.
• No key exchange.
• No proprietary contract.

One lens per deployment (or per study) is mandatory. That lens, and its anchors, are declared in the manifest and must not silently change mid-run. Optional signals (`a_phase`, `Q_phase`, `a_T`, `a_phase_fused`, privacy offsets) can be added later without breaking comparability, as long as they are declared.

Governance remains local but becomes transparent: the manifest captures the chosen lens, pivots, ranges, and hysteresis rule. Validation is reproducible using simple benches such as near-pivot flicker reduction and °C↔°F flip detection.

One-minute math (ASCII, copy-ready)

Always convert to Kelvin first and clamp:

`T_K := max(T_K, eps_TK)`
where `eps_TK > 0` (for example, `1e-6`).

Pick exactly one lens, publish it, and keep it fixed:

• Default (log lens):
`e_T := ln( T_K / T_ref )`

• Linear (tight process windows):
`e_T := ( T_K - T_ref ) / DeltaT`

• Beta (cold-sensitive / cryogenic emphasis):
`e_T := ( T_ref / T_K ) - 1`

• kBT (energy-coupled form):
`e_T := (R*T_K)/E_unit - (R*T_ref)/E_unit`

• Hybrid (piecewise wide-range):
if `abs(T_K - T_ref) > tau`:
`e_T := ln( T_K / T_ref )`
else:
`e_T := ( T_K - T_ref ) / DeltaT`

• Quantum-safe log (extreme lows / near 0 K safety):
`e_T := ln( 1 + (T_K / T_ref) )`

Optional bounded alignment (for pooling and dashboards):

`a_T := tanh( c_T * e_T )`
`a_T := clamp_a( a_T, eps_a )` // enforce `|a_T| <= 1 - eps_a`

Phase proximity around a critical pivot (freeze, gel, warp, boil, human safety band):

`d_m := ( T_K - T_m ) / DeltaT_m`
`a_phase := tanh( c_m * d_m )` // lives in (-1,+1); sign shows which side of `T_m`

Multi-pivot fused dial (multiple survival bands at once):

`a_phase_fused := tanh( sum_i( c_m_i * ( T_K - T_m_i ) / DeltaT_m_i ) )`

Soft hysteresis memory (reduces alert flicker near a pivot):

```
p_side := 0.5 * ( 1 + tanh( k_side * ( T_K - T_m ) ) )
Q_phase := rho*Q_prev + (1 - rho)*clip( p_side, 0, 1 )
```

Key knobs and constants:

`c_T > 0`, `c_m > 0`, `DeltaT > 0`, `DeltaT_m > 0`, `k_side > 0`, `rho in (0,1)`, `eps_a ~ 1e-6`, `eps_TK > 0`, `tau > 0`.

What changes vs what does not

• **What changes.** Machine logic, analytics, safety dashboards, and ML pipelines consume only symbolic signals (`e_T`, `a_T`, `a_phase`, `a_phase_fused`, `Q_phase`). Alerts become unitless. Example:
HeatAdvisory if `e_T >= +0.8` for 3 hours.
FreezeRisk if `a_phase <= -0.6` for 20 minutes.

• **What does not change.**
– Physics and calibration remain exactly as they are.
– Human-facing UIs can still show °C/°F for comfort or regulatory reporting.
– Those human-readable displays do not feed back into machine logic.

This separation is deliberate. It keeps policy, safety, audit, and ML consistent no matter where the data came from.

Outputs at a glance

• **Full SSMT:**
`{ e_T, a_T, a_phase or a_phase_fused, Q_phase, T_m_tag_list, manifest_id, health }`

• **SSMT-Lite (Quick Start):**
`{ e_T }`
plus optional `{ a_phase, Q_phase }`.
No `a_T` required. Default lens is typically the log lens unless declared otherwise.

Both surfaces carry `manifest_id`, which resolves to the published lens, anchors, pivots, and validity ranges.

Scope and non-goals

• **Scope.**
SSMT is a representation layer for temperature. It produces zero-centric contrasts and bounded dials (`e_T`, `a_phase`, `a_phase_fused`, optional `a_T`) that are portable, auditable, and phase-aware. Each stream is bound to a published manifest (declared lens, declared pivots, declared ranges).

• **Non-goals.**
SSMT is not a physics model. SSMT is not a predictor. SSMT does not replace Kelvin for lab math. It does not "fix" a bad sensor. It gives you a safe, comparable symbolic language to express thermal state in operations, infrastructure, audit, and ML.

• **Assumptions (declared).**
– Correct conversion to Kelvin with a positive floor:
`T_K := max(T_K, eps_TK)`
– A single declared lens with its anchors (`T_ref`, `DeltaT` or `E_unit`, `tau`, etc.) that remains fixed for the run / fleet / study.
– Published phase pivots (`T_m`, `DeltaT_m`, `c_m`) and, if applicable, their tags (`T_m_tag_list`).
– UTC timestamps for emitted records.
– Declared validity ranges for `T_K` (for example, `T_valid_range_K`), and a health flag when out of range.

• **Sensor drift and calibration.**
SSMT assumes that the incoming °C/°F/K reading reflects reality after your own calibration. If a sensor drifts, SSMT will emit a wrong-but-self-consistent `e_T`. That is intentional: it preserves auditability. Calibration and correction remain your responsibility.

• **Limits.**
In extreme regimes (near absolute zero, re-entry heating, cryogenic propellants, hot structural stress), you may need an alternate lens such as the Beta form
`e_T := ( T_ref / T_K ) - 1`
or the quantum-safe log form
`e_T := ln( 1 + (T_K / T_ref) )`
to keep the signal numerically stable and interpretable.
Privacy is not automatic: if you publish `T_ref` and lens details, `e_T` can be inverted to approximate `T_K`. Obfuscation must be declared if used.

• **Operational confirmations (3-Test).**
– **Ground (unit invariance).**
Example: let `T_ref := 298 K` and use the log lens `e_T := ln( T_K / T_ref )`.
If `T_K := 273 K`, then `e_T ≈ ln(273/298) ≈ -0.088`.
If `T_K := 373 K`, then `e_T ≈ ln(373/298) ≈ +0.226`.
The same Kelvin state gives the same `e_T` whether it started as °C or °F. This confirms unit invariance after the to-Kelvin step.

– **Edge (phase symmetry / flicker control).**
Let `T_m := 273.15 K`, `DeltaT_m := 2 K`, `c_m := 1`.
For `T_K := 272.5 K`,
`a_phase := tanh( (272.5 - 273.15)/2 ) ≈ tanh(-0.325) ≈ -0.314.`
For `T_K := 273.8 K`,
`a_phase := tanh( (273.8 - 273.15)/2 ) ≈ tanh(+0.325) ≈ +0.314.`
Magnitudes are nearly symmetrical, signs flip. This replaces brittle if/else at 0 C (32 F) with a smooth dial.
Soft hysteresis from `Q_phase` further suppresses rapid alert chatter near `T_m`.

– **Prior (drift / flip detection).**
If a stream that was stable near `e_T ≈ -0.05` suddenly jumps to `e_T ≈ +1.2` in one sample while short-term variance collapses, that is a classic °C↔°F flip or label mismatch, not a physical spike.
Symbolic checks can flag this at ingest before it becomes a false incident.

• **Minimum math (ASCII recap).**

```
T_K := max(T_K, eps_TK)
e_T := ln( T_K / T_ref )
a_phase := tanh( c_m * ( T_K - T_m ) / DeltaT_m )
a_T := tanh( c_T * e_T ) // optional bounded alignment
```

• **Reminder.**
All formulas are given in plain ASCII. SSMT is an observation and representation layer for governance, alerting, and feature extraction. It is not a substitute for calibration, physics models, SOPs, or engineering judgment.

Openness & Attribution (normative)

• **Open standard.** SSMT is permanently open for all to use in any context — public, industrial, municipal, national, academic, commercial, or off-world habitat. No registration, license negotiation, or fees apply.
• **Citation.** When implementing or adapting this specification, cite the concept name "Shunyaya Symbolic Mathematical Temperature (SSMT)" as the origin of the symbolic mathematical temperature approach.
• **Independence.** Implementations do not require any central registry, hosted service, or maintainer approval. Conformance is by self-declaration: compute the formulas exactly as written under a published manifest. No implementer may claim exclusive ownership of the standard.
• **Optional extensions.** Any organization may add optional blocks (for example, gating, privacy offsets, signatures) so long as the core symbolic definitions and baseline formulas remain intact and auditable. Extensions must not alter the meaning of `e_T`, `a_phase`, `a_phase_fused`, or `a_T` without declaring that change.
• **Datasets.** Any datasets used in examples retain their original licenses and usage rules.
• **No warranty or endorsement.** The specification is provided "as-is," with no warranty, and no endorsement or affiliation is implied by implementation. Implementing SSMT does not grant the right to present the implementer as an official steward or exclusive representative.

Intent. The goal is to remove adoption friction so temperature can be governed, compared, alerted on, insured, and audited using the same symbolic language everywhere — across vendors, across borders, and across planets — while preserving a clear foundational link to the Shunyaya framework.

Where Kelvin suffices

If you are doing single-device physics, lab computation, or narrow closed-loop control, plain Kelvin may be enough. Use SSMT when you need:
• Cross-vendor comparability.
• Phase-aware survivability dials (freeze, warp, gel, boil, human safety).
• Portable ML features that remain bounded and auditable.
• Governance that can be written once and enforced across fleets, cities, or habitats.

Authoring guardrails (normative)

• **One lens per study.** Do not silently change lenses mid-run.
• **Publish anchors.** Declare `T_ref` (and `DeltaT` or `E_unit` if relevant), plus `eps_TK`.
• **Validity ranges.** Publish `T_valid_range_K`. Emit `health.range_ok := false` if `T_K` leaves that range.
• **Clamps.** If you emit `a_T`, you must publish `eps_a` and `c_T`, and you must clamp before any pooling.
• **Phase pivots.** Publish every pivot (`T_m`, `DeltaT_m`, `c_m`) and its tag. For multi-pivot, publish the ordered `T_m_tag_list`.
• **Manifest first.** Every emitted record must carry a `manifest_id` that resolves to the full recipe (lens, pivots, ranges, hysteresis policy, and privacy mode if any).

Quick-glance glossary (ASCII)

`T_K` : temperature in Kelvin after unit standardization and floor
`T_ref` : declared reference temperature (K)
`DeltaT` : declared linear scale (K) for the linear lens
`E_unit` : declared energy-per-mole scaling constant for kBT-style lens
`c_T > 0` : sharpness for `a_T`
`e_T` : unitless contrast (unbounded real number unless you clamp or post-process)
`a_T` : bounded alignment dial in (-1,+1) for dashboards, gating, priors
`T_m` : material / safety / survival pivot (e.g., freeze, gel, warp, human danger)
`DeltaT_m` : softness width (K) that defines how quickly you transition around `T_m`
`c_m > 0` : sharpness for `a_phase`
`a_phase` : phase proximity dial in (-1,+1); sign shows which side of `T_m`
`a_phase_fused` : fused multi-pivot dial via summed standardized distances
`Q_phase` : soft hysteresis memory for flicker suppression near `T_m`
`rho` : memory parameter in (0,1) for hysteresis
`eps_a` : small positive clamp for alignments (for example, `1e-6`)
`eps_TK` : small positive floor for Kelvin (for example, `1e-6`)
`tau` : Hybrid lens switch or smoothing threshold (K)
`R` : gas constant (J/mol/K), used in kBT-style lens
`T_valid_range_K` : declared valid range of operation for this deployment

Which lens when — decision tree (ASCII, copy-ready)

1. Compute your intended operating span:
   `span_K := max(T_K) - min(T_K)` over expected conditions.
2. If you operate in cryogenic or tightly life-critical bands (for example, keeping material just above gel point or just below warp point):
   Beta lens
   `e_T := ( T_ref / T_K ) - 1`
3. Else if you expect extremely wide swings (for example, cross-city to cross-planet comparison, deep cold to desert heat):
   Log lens
   `e_T := ln( T_K / T_ref )`
4. Else if you operate in a tight industrial window (for example, process control within ~50 K):
   Linear lens
   `e_T := ( T_K - T_ref ) / DeltaT`
5. Else if the primary concern is thermodynamic work or reaction energy per mole:
   kBT lens
   `e_T := (R*T_K)/E_unit - (R*T_ref)/E_unit`
6. Else if you need both "small offset sensitivity" near nominal and "big deviation capture" far away:
   Hybrid lens
   if `abs(T_K - T_ref) > tau`:
   `e_T := ln( T_K / T_ref )`
   else:
   `e_T := ( T_K - T_ref ) / DeltaT`

6a. Smooth Hybrid (to remove any slope kink at `tau`):

```
s := 1 / ( 1 + exp( -(abs(T_K - T_ref) - tau)/width ) )
e_T := s * ln( T_K / T_ref ) + (1 - s) * ( (T_K - T_ref) / DeltaT )
```

7. If you are operating near extremely low `T_K` and you want to avoid instability around zero:
   Quantum-safe log lens
   `e_T := ln( (T_K/T_ref + alpha) / (1 + alpha) )`
   with `alpha > 0`.
8. Publish your choice. Keep it fixed for that deployment. Do not silently switch lenses mid-stream.

Plain analogies (quick read)

• **`e_T`** is a deviation score from an agreed baseline `T_ref`.
`e_T = 0` means "at baseline".
Positive means "hotter than baseline".
Negative means "colder than baseline".
(Exact sign and slope depend on the chosen lens; you declare that lens.)

• **`a_T`** is a bounded "thermal alignment dial." It says how strongly this reading should count, or how aggressively it should trigger routing or gating, without letting any one reading dominate.

• **`a_phase`** / **`a_phase_fused`** answer the survival question:
"Which side of the pivot are we on, and how far into danger are we?"
This replaces fragile on/off rules at magic numbers like 0 C or 32 F.

• **`Q_phase`** is short-term memory. It prevents alarms from flapping on and off when a reading dances around a pivot.

All of these are designed so that policy, safety, audit, and ML can talk the same thermal language — locally, globally, and beyond Earth — without fighting over raw units, and without hiding responsibility for calibration or judgment.

TABLE OF CONTENTS

1) Encoding SSMT

Practical, copy-ready rules and helpers to convert raw **°C/°F/K** into **unitless symbols** for analytics and machine logic.

1.1 Standardize to Kelvin (first, always)

```
to_kelvin(T_raw, unit):
u = lower(unit) # accept "C","c","celsius", etc.
if u in {"k","kelvin"}: T_K = T_raw
elif u in {"c","celsius"}: T_K = T_raw + 273.15
elif u in {"f","fahrenheit"}: T_K = (T_raw - 32) * 5/9 + 273.15
else: raise "unknown unit"
T_K = max(T_K, eps_TK) # Kelvin floor for numerical safety (publish eps_TK)
return T_K
```

• **Precision hint.** Keep at least **0.01 K** internally; round only for human display.
• **Clock and context.** Attach **UTC timestamp** and **sensor ID** outside this function.

1.2 Pick exactly one lens (publish and keep fixed)

All lenses are **monotone in `T_K`** and produce a **unitless contrast `e_T`**. Choose once per study or mission.

• **Default (log lens; wide ranges, cross-fleet):**
```
e_T := ln(T_K / T_ref)
```

• **Linear (tight process windows):**
```
e_T := (T_K - T_ref) / DeltaT
```

• **Beta (cold-sensitive regimes):**
```
e_T := (T_ref / T_K) - 1
```

• **kBT (energy-linked, per mole via R):**
```
e_T := (R*T_K)/E_unit - (R*T_ref)/E_unit
```

• **Hybrid (mixed regimes; piecewise; lens token "hybrid"):**
```
if abs(T_K - T_ref) > tau: e_T := ln(T_K / T_ref) else: e_T := (T_K -
T_ref) / DeltaT
```

• **Quantum-safe log (near 0 K; zero-centric at `T_ref`; lens token "qlog"):**
```
e_T := ln( (T_K/T_ref + alpha) / (1 + alpha) ) # alpha > 0
```

Lens decision guide (quick):
• **Human comfort / buildings** `span_K < 50 K` -> **linear**
• **Environmental / fleets / cross-planet** -> **log** (default)
• **Cryo / radiative / cold-dominant** -> **beta**
• **Chem/bio gating vs energy scale** -> **kBT**
• **Mixed small+large departures around baseline** -> **hybrid** (set `tau`)
• **Near absolute zero or strict safety near** `T_K ~ 0` -> **qlog** (set `alpha`)

Domain guards (must hold):
• log, beta: require `T_K > 0` and `T_ref > 0`
• linear: require `DeltaT > 0`
• kBT: require `E_unit > 0`
• hybrid: require `DeltaT > 0` and `tau > 0`
• qlog: require `T_ref > 0` and `alpha > 0`

Practical defaults (recommended starting points)

• **Lens.** log (default for cross-fleet, wide ranges).
• **Anchors.** `T_ref = 298.15 K`.
• **Kelvin floor.** `eps_TK = 1e-6` (apply after conversion).
• **Phase (when freezing matters).** `T_m = 273.15 K`, `DeltaT_m = 2.0 K`, `c_m = 1.2`, `rho = 0.90`, `k_side = 2.0`.

• **Emissions by tier.**
– **S1 (Lite):** emit `e_T` + `manifest_id` + `health`.
– **S2:** add `a_phase` (and `Q_phase`) only if freezing or melting matters.
– **S3:** add `a_T` only for bounded fleet dials or ML priors.
• **Clamps and precision.** `eps_a = 1e-6`; `eps_w = 1e-12`; keep at least `0.01 K` internally.
• **Validity.** Publish `T_valid_range_K` appropriate to domain, for example weather `[240,320] K`, buildings `[250,320] K`; flag out-of-range.
• **Alert seeds (tune per ops).** `Phi_freeze = 0.10`; `E_hot = 0.8`; dwell times matched to cadence.
• **Pooling (if used).** Rapidity pooling of alignments; equal weights unless confidence is known.
• **Hybrid lens defaults.** `tau = 5.0 K` (start here; tune).
• **qlog defaults.** `alpha = 1.0` (start here; tune).
• **Do not over-engineer. One lens per deployment**; do not emit `a_T` unless you actually need a bounded dial.

1.3 Compute the contrast (e_T)

```
encode_eT(T_K, lens, T_ref=None, DeltaT=None, E_unit=None, R=8.314462618,
tau=None, alpha=None):
if lens == "linear": return (T_K - T_ref) / DeltaT
if lens == "log": return ln(T_K / T_ref)
if lens == "beta": return (T_ref / T_K) - 1
```

```
if lens == "kBT": return (R*T_K)/E_unit - (R*T_ref)/E_unit
if lens == "hybrid":
if abs(T_K - T_ref) > tau: return ln(T_K / T_ref)
else: return (T_K - T_ref) / DeltaT
if lens == "qlog": return ln(((T_K / T_ref) + alpha) / (1 + alpha))
raise "unknown lens"
```

• **Invariance.** After one to-K step, `e_T` carries identical meaning across degC/degF sources.
• **Zero-centricity.** `e_T = 0` at `T_ref` for all lenses above. qlog achieves this via normalization by `(1 + alpha)`.

Parameters used in examples: `T_ref = 298.15 K`, `DeltaT = 10 K`, `tau = 5 K`, `alpha = 1.0`, `E_unit = R*T_ref`.

1.3.x Smooth Hybrid (continuity-safe) lens

```
encode_eT(T_K, lens, T_ref=None, DeltaT=None, tau=None, width=None):
if lens == "smooth_hybrid":
dT = abs(T_K - T_ref)
e_lin = (T_K - T_ref) / DeltaT
e_log = ln(T_K / T_ref)
s = 1 / (1 + exp(-(dT - tau) / width)) # 0 < s < 1
return s*e_log + (1 - s)*e_lin
```

• **Purpose.** Preserve familiar hybrid behavior while removing the **slope kink** at `dT = tau` via a smooth sigmoid blend `s`.
• **Continuity.** `e_T` is continuous for all `T_K > 0`. For any `tau >= 0` and `width > 0`, both value and slope change smoothly through `dT ≈ tau`.
• **Zero-centricity.** At `T_K = T_ref`, `e_lin = 0` and `e_log = 0` so `e_T = 0` independent of `s`.
• **Monotonicity.** For fixed `T_ref`, `e_T` increases strictly with `T_K`.
• **Determinism.** Single declared lens choice. `tau` and `width` are fixed in the manifest.
• **Parameter guardrails.** Use `tau >= 0`, `width > 0`. **Recommended starts (indoor/HVAC):** `DeltaT = 10 K`, `tau = 5 K`, `width = 2 K`.
• **Invariance.** Same invariance as other lenses after the to-K step.
• **Additional parameters for this lens:** `tau` (K), `width` (K).

1.4 Optional bounded alignment (a_T) for pooling and dashboards

```
a_T := tanh(c_T * e_T)
a_T := clamp_a(a_T, eps_a) # enforce |a_T| <= 1 - eps_a
```

• **Knobs.** `c_T > 0` for sharpness; `eps_a ~ 1e-6` for safety clamp.
• **Why bounded.** Keeps dials in `(-1, +1)`, enables **safe averaging** via rapidity.

1.5 Phase proximity dial (near freezing or melting, etc.)

```
d_m := (T_K - T_m) / DeltaT_m
a_phase := tanh(c_m * d_m)
a_phase := clamp_a(a_phase, eps_a)
```

• **Knobs.** `c_m > 0`, `DeltaT_m > 0`.
• **Interpretation.** `a_phase ~ -1` well below pivot (**solid-favoring**), `0` near pivot, `+1` well above (**liquid-favoring**).
• **Multiple materials.** Keep a list of `{T_m, DeltaT_m, c_m, T_m_tag}` per stream if needed.

Optional fused multi-pivot dial (e.g., freezing + boiling):
`a_phase_fused := tanh( sum_i( c_m_i * (T_K - T_m_i) / DeltaT_m_i ) )`
• Emit either **`a_phase`** (single pivot) or **`a_phase_fused`** (multi-pivot), not both.

1.5.x Adaptive hysteresis (rho by noise)

Goal.
Stabilize alerts around a critical pivot (for example, freezing, gel point, warp point, human safety band) by automatically increasing memory when readings are noisy, and relaxing memory when readings are calm.

Step 1 — Estimate recent noise (dimensionless).
Use a sliding window of the most recent samples to capture short-term volatility around the pivot:

```
sigma_T := std( T_K[t-W+1 : t] )
sigma_norm := sigma_T / DeltaT_m
```

Where:

- `W` is the window length in samples (for example, 30).
- `DeltaT_m` is the softness width around the pivot `T_m` (for example, 2.0 K).
- `sigma_norm` is a normalized noise estimate with no units.

Step 2 — Map noise to hysteresis strength (bounded).
Convert `sigma_norm` into a smoothing weight `rho` in `[rho_min , rho_max]`:

```
rho_raw := 1 - exp( - sigma_norm / sigma0 )
rho := clip( rho_raw , rho_min , rho_max )
```

Where:

- `sigma0 > 0` is a tuning constant that controls how fast `rho` ramps up with noise.
- `0 <= rho_min < rho_max < 1`.
- Higher `rho` means "remember longer before declaring state changed."

Step 3 — Compute side likelihood around the pivot (soft step).
Build a smooth indicator of which side of the pivot you are on:

```
p_side := 0.5 * ( 1 + tanh( k_side * ( T_K - T_m ) ) )
```

Where:

- `k_side > 0` controls how sharply `p_side` flips across `T_m`.
- `p_side` is in `(0 , 1)`, with values near 0 meaning "cold side" and values near 1 meaning "hot side."

Step 4 — Adaptive exponential smoother (flicker suppression).
Use `rho` to update a short-term memory of which side you have effectively been on, without flapping:

```
Q_phase := rho * Q_phase_prev + ( 1 - rho ) * clip( p_side , 0 , 1 )
```

Where:

- `Q_phase_prev` is the previous timestep's value of `Q_phase`.
- `Q_phase` stays in `[0 , 1]`.
- Near `0` means "we have effectively been on the cold side."
- Near `1` means "we have effectively been on the hot side."

Knobs (tunable).

- `W` (samples in the sliding window).
- `sigma0 > 0`.
- `0 <= rho_min < rho_max < 1`.
- `k_side > 0`.
- Recommended starting values:
  - `W = 30`
  - `sigma0 = 0.1`
  - `rho_min = 0.50`
  - `rho_max = 0.95`
  - `k_side = 0.5 1/K`

Initialization.
At the first timestep:
`Q_phase_0 := clip( p_side_0 , 0 , 1 )`

Interpretation.

- When the signal is calm (`sigma_norm -> 0`), `rho` shrinks toward `rho_min`, the memory relaxes, and `Q_phase` responds more quickly to real changes.
- When the signal is noisy (large `sigma_norm`), `rho` approaches `rho_max`, the memory strengthens, and `Q_phase` resists rapid flipping caused by jitter around `T_m`.
- This means you get **fast reaction when the environment is stable** and **stability when the environment is unstable**.

Usage.

- Emit `a_phase` (or `a_phase_fused`) as the continuous bounded dial in `(-1 , +1)` for "how far into risk and on which side."
- Use `Q_phase` as the debounced side-memory for actual decision thresholds and alerts, especially for policies like "stay above safe band for 15 minutes before escalating."

1.6 Soft hysteresis near the pivot (reduces alert chatter)

```
p_side := 0.5 * (1 + tanh(k_side * (T_K - T_m)))
Q_phase := rho * Q_prev + (1 - rho) * clip(p_side, 0, 1)
```

• **Knobs.** `k_side > 0` for edge sharpness; `rho in (0,1)` for memory.
• **Usage.** Use `Q_phase` as a memory dial to gate **clear** vs **risk** states.

1.6.x Multi-pivot fusion — Normalized form

Normalized per-pivot contrast (dimensionless):

```
z_i := c_m_i * (T_K - T_m_i) / DeltaT_m_i
```

Weighted, scale-stable aggregator (default `w_i = 1`):

```
W := max(sum_i w_i, eps_w)
z_bar := (1 / W) * sum_i(w_i * z_i)
```

Bounded fused dial:

```
a_phase_fused := tanh(z_bar)
a_phase_fused := clamp_a(a_phase_fused, eps_a)
```

• **Why normalize.** Summing many pivots can inflate magnitude; dividing by total weight keeps the fused scale comparable across materials with different counts of transitions.
• **Weights (optional).** Choose `w_i > 0` to emphasize critical pivots; default `w_i = 1`.
• **Knobs.** `c_m_i > 0`, `DeltaT_m_i > 0`, `w_i > 0`; guard `eps_w > 0`.
• **Usage.** Emit either **`a_phase`** or **`a_phase_fused`**, not both. `Q_phase` from 1.6 can be applied on top of `a_phase_fused`.

1.7 Validity and out-of-range handling (normative)

• Publish `T_valid_range_K = [T_min, T_max]`.
• If `T_K` is outside the range, set `health.range_ok := false` and compute with a **saturated proxy** for stability:

```
T_K_eff := min(max(T_K, T_min), T_max)
e_T_eff := encode_eT(T_K_eff, ...)
a_T_eff := tanh(c_T * e_T_eff) # if emitting a_T
```

• Emit an extra flag `oor := true | "below_min" | "above_max"` if used.

• For lens domain violations, for example `T_K <= 0` under log or beta, also set `health.sensor_ok := false`.

1.8 Fleet or array pooling (optional, safe, associative)

To combine multiple **alignments** (from sensors or lanes), use **rapidity pooling**:

```
pool_alignments(a_list, w_list=None, eps_a=1e-6, eps_w=1e-12):
if w_list is None: w_list = [1 for _ in a_list]
U = sum_i w_list[i] * atanh(clamp_a(a_list[i], eps_a))
W = max(sum_i w_list[i], eps_w)
return tanh(U / W)
```

• **Weights.** `w_i` can be `1/variance`, health scores, or equal.
• **Lite note.** If running Lite (no `a_T`), you may pool **`a_phase`** instead.

1.9 Emitted record (Full SSMT)

Unitless, portable, self-describing:

```
{
"timestamp_utc": "...",
"e_T": <number>,
"a_T": <number, optional>,
"a_phase": <number, optional>,
"Q_phase": <number, optional>,
"T_m_tag_list": ["<string>", ... , optional],
"manifest_id": "<string>",
"health": { "range_ok": true/false, "drift": true/false, "sensor_ok":
true/false },
"oor": false | "below_min" | "above_max"
}
```

• **Machine logic rule.** Consume only **`e_T`** (and **`a_phase`** or **`a_phase_fused`** where relevant). Human UIs may render **degC/degF** for display only.

1.10 Commissioning checklist (must-do before first byte)

• Fix **lens** and publish **`T_ref`** (and **`DeltaT`** or **`E_unit`** if applicable).
• Declare **pivots**: `T_m`, `DeltaT_m`, `c_m`, `k_side`, `rho`, `T_m_tag` (or `T_m_tag_list`).
• Publish **guards**: `eps_a`, `eps_w`, `eps_TK`, `T_valid_range_K`.
• **Hybrid/qlog knobs** if chosen: publish `tau` and/or `alpha`.
• Decide **health flags**: enable `range_ok`, `sensor_ok`, `drift`.
• Freeze **manifest**: assign `manifest_id` and attach it to **every stream**.

1.11 Optional — lens_mode = "auto" (resolves at commissioning; single lens per run)

If you must cover mixed regimes in one fleet, declare a **deterministic** auto policy. Keep thresholds in the manifest. Do not switch per-record heuristically.

```
lens_mode := "auto"
if min(T_K) <= T_qsafe: lens := "qlog"
elif span_K >= 500: lens := "log"
elif min(T_K) < 100: lens := "beta"
elif span_K <= 50: lens := "linear"
else: lens := "hybrid"
```

• **Resolve once during commissioning** using a fixed window; record the resolved lens in the manifest and keep it constant for the entire run.
• **Publish** `T_qsafe` (for example `1.0 K` start).

1.12 Central reference — clamps, guards, and defaults (normative)

Alignment clamps (safety).

`clamp_a(a, eps_a)` enforces `|a| <= 1 - eps_a`. Default `eps_a = 1e-6`. Applies to `a_T`, `a_phase` (and any bounded dial). If you emit a bounded dial, you **must** publish `eps_a`.

Pooling guard (denominator).
When pooling alignments via rapidity:

```
a_pool := tanh( (sum_i w_i * atanh(clamp_a(a_i, eps_a))) / max(sum_i w_i, eps_w) )
```

Default `eps_w = 1e-12`. Properties: **bounded**, **associative**, **order-invariant**.

Kelvin floor (safety).
After conversion: `T_K := max(T_K, eps_TK)`. Default `eps_TK = 1e-6`. Purpose: numerical stability and domain safety near `0 K`.

Phase dial defaults (if used).
`a_phase := tanh(c_m * ((T_K - T_m) / DeltaT_m))` then clamp. Suggested defaults: `T_m = 273.15 K`, `DeltaT_m = 2.0 K`, `c_m = 1.2`, `rho = 0.90`, `k_side = 2.0`.

Memory (optional).

```
p_side := 0.5 * (1 + tanh(k_side * (T_K - T_m)))
Q_t := rho * Q_{t-1} + (1 - rho) * clip(p_side, 0, 1)
```

Manifest rule.
Pivots **must** be published as `{tag, T_m, DeltaT_m, c_m}`; hysteresis knobs published if used.

Lens domain guards (must).
• log, beta: `T_K > 0`, `T_ref > 0`
• linear: `DeltaT > 0`
• kBT: `E_unit > 0`
• qlog: `T_ref > 0`, `alpha > 0`

Zero-centricity (must).
For all lenses, `e_T = 0` at `T_ref`. For qlog, zero-centricity is enforced by normalization with `(1 + alpha)`.

Validity and saturation (must).
Publish `T_valid_range_K = [T_min, T_max]` with `0 < T_min < T_max`. If `T_K` is outside the range: set `health.range_ok := false`, set `oor` to `"below_min"` or `"above_max"`, and compute with
`T_K_eff := min(max(T_K, T_min), T_max)` # stability only; do not coerce thresholds.

Precision and rounding.
Maintain at least `0.01 K` internally; round only for human display. Keep floats in symbol space to `>= 6` decimal places in reports.

Alignment channel (if emitted).
`a_T := tanh(c_T * e_T)` then clamp with `eps_a`. Publish `c_T > 0` (sharpness). Only emit `a_T` when bounded pooling, dashboards, or priors are required.

Hybrid/qlog knobs (defaults).
`tau = 5.0 K` (hybrid threshold; tune per domain).
`alpha = 1.0` (qlog offset; tune per domain).
`T_qsafe = 1.0 K` (auto policy near-zero selector; tune per domain).

Cross-references.
• Pooling: see **1.8**.
• Validity handling: see **1.7**.
• Phase dial and hysteresis: see **1.5–1.6**.

1.13 Practical defaults (declare once)

Clamps and guards (global)
`eps_a = 1e-6` # alignment clamp; enforce `|a| <= 1 - eps_a`
`eps_w = 1e-12` # denominator guard for pooling and means

Lens dials (contrast `e_T` and smooth_hybrid)
`c_T = 1.0` # slope for `a_T := tanh(c_T * e_T)`
`DeltaT = 10.0` # K (linear lens scale near `T_ref`)
`tau = 5.0` # K (smooth_hybrid transition center)
`width = 2.0` # K (smooth_hybrid blending width)
`rho = 0.80` # memory for `Q_phase` (use adaptive if enabled)

• **Units.** `DeltaT`, `tau`, `width` in kelvin (K).
• **Scope.** Declare once per study; tune only via the manifest.
• **Safe ranges.** `0 < eps_a << 1`, `0 < eps_w << 1`, `c_T > 0`, `DeltaT > 0`, `tau >= 0`, `width > 0`, `0 < rho < 1`.
• **Suggested starts (indoor/HVAC).** `DeltaT = 10 K`, `tau = 5 K`, `width = 2 K`, `rho = 0.8`.

---

2) Manifest

A tiny, explicit contract that makes streams **portable**, **auditable**, and **simple to adopt** (including very simple use cases). This section keeps **defaults minimal**, offers a **Lite** path, and adds **optional validation hooks** so empirical evidence can accumulate cleanly. (References: if a **privacy** block is present, semantics are per **Addendum J**. If a **gate** block is present, semantics are per **Section 3.1 env-gate**.)

2.1 Manifest principles (normative)

• **One lens per study/mission.** Do not change mid-run.
• **Anchors are public.** Anyone can reconstruct `e_T` from `T_K` and verify. (**Privacy note:** inverse mapping is possible if anchors are public; see Addendum J.)
• **Minimal first.** Start with the smallest set (**Lite/S1**), add fields only when needed.
• **Machine-checkable.** Every rule below is designed to be validated automatically.
• **Backwards-safe.** New fields are optional; old consumers must not break.
• **Optional blocks.** If present: **privacy** per Addendum J; **gate** per Section 3.1 (env-gate).

2.2 Minimal manifest (S1, Lite-compatible)

For **dashboards**, **weather**, **audits**, and **simple IoT**. Emits `e_T` (and optionally `a_phase`).

```
SSMT_manifest_v1:
version: "1.1" # spec version
compliance_level: "S1" # S1 | S2 | S3
lens: "log" # log | linear | beta | kBT | hybrid | qlog
anchors:
T_ref: 298.15 # Kelvin
DeltaT: null # only if lens == "linear"
E_unit: null # only if lens == "kBT"
tau: null # only if lens == "hybrid" (K)
alpha: null # only if lens == "qlog" (> 0)
guards:
clamp_a_eps: 1e-6 # applies if a_phase used
eps_TK: 1e-6 # Kelvin floor (safety)
T_valid_range_K: [180.0, 500.0]
phase_block:
T_m_list: [] # optional; empty = no phase dial
```

```
rho: 0.90 # default memory
k_side: 2.0 # default soft edge
outputs:
emit_e_T: true
emit_a_phase: false
emit_a_T: false
notes: "Minimal manifest; unitless anomaly only"
# Optional blocks (declare only if needed):
# privacy: { mode: "P1", step: 0.01 } # see Addendum J (P0/P1/P2)
# gate: { enabled: false } # if enabled, behavior per Section 3.1
```

Why simple: no `a_T`, no pooling, phase dial off by default. Teams can **ship S1 quickly** and upgrade later.

2.3 Full manifest (S3, phase + alignment + pooling)

For **fleets**, **safety gating**, and **ML-ready pipelines**.

```
SSMT_manifest_v1:
version: "1.1"
compliance_level: "S3" # S1/S2/S3 (S3 = full)
lens: "log" # log | linear | beta | kBT | hybrid | qlog
anchors:
T_ref: 298.15
DeltaT: null
E_unit: null
c_T: 0.7 # alignment sharpness (> 0, only if emit_a_T)
tau: 5.0 # only if lens == "hybrid" (K)
alpha: 1.0 # only if lens == "qlog" (> 0)
guards:
clamp_a_eps: 1e-6
eps_w: 1e-12 # pooling denominator guard
eps_TK: 1e-6 # Kelvin floor (safety)
T_valid_range_K: [180.0, 500.0]
phase_block:
T_m_list:
- { tag: "water", T_m: 273.15, DeltaT_m: 2.0, c_m: 1.2 }
rho: 0.90
k_side: 2.0
pooling:
weight_rule: "equal" # equal | inv_var | health
health_flags:
enable_range_ok: true
enable_sensor_ok: true
enable_drift: true
validation: # empirical hooks (optional)
dataset_ref: "url-or-id"
test_vectors_ref: "url-or-id"
metrics: { E_HOT: 0.8, E_COLD: 0.8, Phi_freeze: 0.10 }
outputs:
emit_e_T: true
emit_a_T: true
emit_a_phase: true # single or fused (multi-pivot)
emit_Q_phase: true
notes: "Full feature set for fleets and safety"
# Optional blocks:
```

```
# privacy: { mode: "P2", b: 1.0, eps: 1.0 } # see Addendum J
# gate:
# enabled: true
# lane: "w1*abs(a_T)+w2*Q_phase+w3*abs(a_phase)"
# w: { w1: 0.5, w2: 0.5, w3: 0.0 }
# L: 60
# kappa: 1.0
# mu: 3.0
# rho_g: 0.9
# theta_g: 0.2 # semantics per Section 3.1
```

Why validated: the **validation** block captures dataset/test-vector pointers so evidence **accumulates and reproduces**.

2.4 Field definitions (normative)

• **version:** spec version string, for example `"1.1"`.
• **compliance_level:** `"S1"` (`e_T` only), `"S2"` (add `a_phase`), `"S3"` (add `a_T`, pooling, hysteresis).
• **lens:** `"log" | "linear" | "beta" | "kBT" | "hybrid" | "qlog"`.
• **anchors:**
o **T_ref (K)** mandatory.
o **DeltaT (K)** required iff `lens == "linear"` and must satisfy `DeltaT > 0`.
o **E_unit (J/mol)** required iff `lens == "kBT"` and must satisfy `E_unit > 0`.
o **c_T > 0** required iff `emit_a_T == true`.
o **tau > 0** required iff `lens == "hybrid"`.
o **alpha > 0** required iff `lens == "qlog"`.
o **T_ref_policy** (optional string): `"fixed" | "seasonal" | "diurnal" | "rolling-<window>"`. If non-fixed, document policy in notes; **do not change mid-run**.
• **guards:**
o `clamp_a_eps in (0, 1e-3]`, default `1e-6`.
o `eps_w in (0, 1]`, default `1e-12`, used only for pooling.
o `eps_TK > 0`, default `1e-6` (Kelvin floor applied after conversion).
o `T_valid_range_K = [T_min, T_max]` with `0 < T_min < T_max`.
• **phase_block (optional):**
o `T_m_list`: zero or more pivots with `{tag, T_m, DeltaT_m > 0, c_m > 0}`.
o `rho in (0,1)`, memory; default `0.90`.
o `k_side > 0`, soft edge; default `2.0`.
• **pooling (optional):** define `weight_rule` as `"equal"`, `"inv_var"`, or `"health"`.
• **health_flags:** toggles for `range_ok`, `sensor_ok`, `drift`.
• **validation (optional):** references to datasets and test vectors; not used by runtime logic.
• **privacy (optional):** `{ mode: "P0"|"P1"|"P2", step?, b?, eps? }` — semantics per Addendum J.
• **gate (optional):** `{ enabled, lane, w, L, kappa, mu, rho_g, theta_g }` — semantics per Section 3.1.
• **outputs:** explicit booleans to avoid ambiguity (`emit_e_T` must be true).
• **notes:** free text; avoid PII.

2.5 Emitted payload (normative keys)

Devices **must** follow the manifest to emit stable keys; human UIs **may** render °C/°F for display only.

```
{
"timestamp_utc": "2025-09-24T12:00:00Z",
"e_T": <number>,
"a_T": <number, optional>, # only if emit_a_T
"a_phase": <number, optional>, # single-pivot phase dial
"a_phase_fused": <number, optional>, # fused multi-pivot dial (prefer
exactly one of a_phase or a_phase_fused)
"Q_phase": <number, optional>, # only if emit_Q_phase
"T_m_tag": "<string, optional>", # when using single pivot
"T_m_tag_list": ["<string>", ... , optional], # when using fused multi-
pivot
"manifest_id": "<string>",
"health": { "range_ok": true/false, "drift": true/false, "sensor_ok":
true/false },
"oor": false | "below_min" | "above_max",
"uncertainty": { # optional; see Addendum L.2
"e_T_std": <number, optional>,
"a_T_std": <number, optional>,
"method": "delta"
}
}
```

• **Required:** `timestamp_utc`, `e_T`, `manifest_id`, `health.range_ok`.
• **Phase fields:** Prefer **exactly one** of `a_phase` or `a_phase_fused`. If `a_phase_fused` is present, `T_m_tag_list` **should** be present and non-empty; if `a_phase` is used with a single pivot, `T_m_tag` **should** be present.
• **Conformance:** Emit `a_T`, `a_phase`, `a_phase_fused`, and `Q_phase` **only** if the manifest outputs enable them.
• **Time format:** `timestamp_utc` **must** be ISO-8601 UTC (with `Z`).
• **Out-of-range:** `oor` **must** be `false`, `"below_min"`, or `"above_max"` per validity semantics (see **2.6**).
• **Uncertainty block (optional):** If present, fields follow **Addendum L.2**; omit if sensor variance is unavailable or implausible.

Optional (informative) fields for experiments/integrations (non-normative).
These **may** be included in logs or pilots and do not affect SSMT conformance.
• `g_t: <number>` — env-gate per Section **3.1** (bounded in `(0,1)`).
• **Domain outputs** (e.g., chemistry): include downstream metrics like `RSI_env` in a separate app payload; do **not** treat them as SSMT core keys.

MQTT mapping (copy-paste example).

```
topic: ssmt/{station_id}/s1
payload:
{
"timestamp_utc": "2025-09-24T12:00:00Z",
"e_T": 0.0331,
"a_phase": -0.12,
"Q_phase": 0.31,
"uncertainty": { "e_T_std": 0.004, "method": "delta" },
"manifest_id": "SSMT-CITY-2025-09-001",
```

```
"health": { "range_ok": true, "sensor_ok": true, "drift": false },
"oor": false
}
```

• **Note:** Keep MQTT payloads in **pure ASCII JSON**; avoid including raw °C/°F values in machine payloads (**UI-only** if needed).

2.6 Validity and domain-violation semantics (must)

• **Out-of-range Kelvin:** set `health.range_ok := false`, set `oor` accordingly, and compute with **saturated** `T_K_eff` (no crashes):
`T_K_eff := min(max(T_K, T_min), T_max)`
• **Lens domain violations:**
• **log/beta:** if `T_K <= 0` or `T_ref <= 0`, set `health.sensor_ok := false`.
• **linear:** if `DeltaT <= 0`, set `health.sensor_ok := false`.
• **kBT:** if `E_unit <= 0`, set `health.sensor_ok := false`.
• **qlog:** if `T_ref <= 0` or `alpha <= 0`, set `health.sensor_ok := false`.
• **Kelvin floor:** apply `T_K := max(T_K, eps_TK)` after conversion to improve numerical safety; still honor flags above.
• **No coercion of outputs:** flags inform downstream logic; thresholds remain in **symbol space**.

2.7 Keeping it simple (anti over-engineering guardrails)

• Use **S1** unless you **demonstrably** need phase or pooling.
• If freezing matters, upgrade to **S2** (add `a_phase`, soft hysteresis).
• Add `a_T` (**S3**) only for **pooling/fleet dials** or **ML priors** that benefit from bounded features.
• Document the **reason** for each upgrade in `notes`.

2.8 Versioning and compatibility

• **New fields** must be **optional** with sensible defaults.
• **Deprecations** must be announced and supported for **>= 12 months**.
• **Consumers** should **ignore unknown fields**.

2.9 Example manifests (copy-ready)

A) Weather/city analytics (S1, log lens)

```
SSMT_manifest_v1:
version: "1.1"
compliance_level: "S1"
```

```
lens: "log"
anchors: { T_ref: 295.00, DeltaT: null, E_unit: null, tau: null, alpha:
null }
guards: { clamp_a_eps: 1e-6, eps_TK: 1e-6, T_valid_range_K: [230.0, 330.0]
}
phase_block: { T_m_list: [], rho: 0.90, k_side: 2.0 }
outputs: { emit_e_T: true, emit_a_phase: false, emit_a_T: false }
notes: "City anomaly; unitless e_T only"
# optional: privacy: { mode: "P1", step: 0.02 } # Addendum J
```

B) Cold chain (S2, linear lens + phase dial)

```
SSMT_manifest_v1:
version: "1.1"
compliance_level: "S2"
lens: "linear"
anchors: { T_ref: 277.15, DeltaT: 2.0, E_unit: null, tau: null, alpha: null
}
guards: { clamp_a_eps: 1e-6, eps_TK: 1e-6, T_valid_range_K: [260.0, 320.0]
}
phase_block:
T_m_list: [ { tag: "product", T_m: 273.15, DeltaT_m: 2.0, c_m: 1.2 } ]
rho: 0.90
k_side: 2.0
outputs: { emit_e_T: true, emit_a_phase: true, emit_a_T: false,
emit_Q_phase: true }
notes: "Phase-aware excursion; no alignment channel"
# optional: gate: { enabled: true, lane: "Q_phase", L: 60, kappa: 1.0, mu:
3.0, rho_g: 0.9, theta_g: 0.2 } # Section 3.1
```

C) Space ops (S3, mixed subsystems; hybrid lens)

```
SSMT_manifest_v1:
version: "1.1"
compliance_level: "S3"
lens: "hybrid"
anchors: { T_ref: 200.0, DeltaT: 10.0, E_unit: null, c_T: 0.7, tau: 5.0,
alpha: null }
guards: { clamp_a_eps: 1e-6, eps_w: 1.2e-12, eps_TK: 1e-6, T_valid_range_K:
[3.0, 500.0] }
phase_block:
T_m_list: [ { tag: "propellant", T_m: 65.0, DeltaT_m: 3.0, c_m: 1.2 } ]
rho: 0.95
k_side: 3.0
pooling: { weight_rule: "health" }
health_flags: { enable_range_ok: true, enable_sensor_ok: true,
enable_drift: true }
validation: { dataset_ref: "url-or-id", test_vectors_ref: "url-or-id" }
outputs: { emit_e_T: true, emit_a_phase: true, emit_Q_phase: true,
emit_a_T: true }
notes: "Mixed regimes; bounded fleet dials and validation hooks"
# optional: privacy: { mode: "P2", b: 1.0, eps: 1.0 }
# optional: gate: { enabled: true, lane: "w1*abs(a_T)+w2*Q_phase", w:
{w1:0.5,w2:0.5}, L:60, kappa:1.0, mu:3.0, rho_g:0.9, theta_g:0.25 }
```

2.10 Device passport snippet (ships with firmware)

```
SSMT_passport:
manifest_id: "SSMT-ACME-2025-09-001"
```

```
version: "1.1"
lens: "log"
anchors: { T_ref: 298.15, DeltaT: null, E_unit: null, c_T: null, tau: null,
alpha: null }
phase_block: { T_m_list: [], rho: 0.90, k_side: 2.0 }
guards: { clamp_a_eps: 1e-6, eps_TK: 1e-6, T_valid_range_K: [180.0, 500.0]
}
outputs: { emit_e_T: true, emit_a_phase: false, emit_a_T: false }
# optional: privacy: { mode: "P1", step: 0.02 }
# optional: gate: { enabled: false }
```

2.11 Optional — declared auto lens policy (fleet-level, deterministic)

If mixed regimes must be covered in one fleet, declare a **deterministic** auto policy at commissioning (publish thresholds; do **not** switch per record):

```
auto_lens_policy:
enabled: false
rules:
- if: { min_K: "<= 1.0" } then: { lens: "qlog" }
- if: { span_K: ">= 500" } then: { lens: "log" }
- if: { min_K: "< 100" } then: { lens: "beta" }
- if: { span_K: "<= 50" } then: { lens: "linear" }
- else: { lens: "hybrid", tau: 5.0 }
```

Record the **resolved lens once per stream**; avoid per-sample switching to maintain **auditability**.

---

3) Benefits & Operational Coupling

Why SSMT's **representation-first** approach is simple enough for everyday use and strong enough for **fleets**, **safety**, and **ML**.

3.1 Unified coupling to g_t (env-gate)

Purpose. Turn SSMT outputs into a bounded environmental gate `g_t` that modulates any downstream bounded index, for example `RSI_env := g_t * RSI`.

Lane construction (bounded `S_t` in `[0,1]`).
`S_t := clip( w1*abs(a_T) + w2*Q_phase + w3*abs(a_phase) , 0 , 1 )`
• **Knobs.** `w1, w2, w3 >= 0` with `w1 + w2 + w3 <= 1` (default `w1 = 0.5`, `w2 = 0.5`, `w3 = 0`).
• **Alternative.** `S_t := abs(a_T)` or `S_t := Q_phase` (pick one; declare in manifest).
• **Optional env bundle.** If enabled, add `w_env*abs(a_env)` to the sum (see Addendum L.1).

Volatility (windowed).
`Z_t := log(1 + Var_{t-L+1:t}(S_t))`
• **Knob.** `L >= 2`. Variance is nonnegative by definition.

Antagonist term and contrast.
```
A_t := 1 / (1 + Z_t)
Delta_t := abs(Z_t - A_t)
```

Calm memory (risk accumulation).
`Q_t := rho_g*Q_{t-1} + (1 - rho_g)*clip(A_t - Z_t, 0, 1)`
• **Knob.** `0 < rho_g < 1` (default `0.9`).
• **Init.** `Q_0 := clip(A_0 - Z_0, 0, 1).`

Unified gate (bounded, monotone in risk).
`g_t := ( 1 / (1 + Z_t + kappa*Delta_t) ) * ( 1 - exp(-mu*Q_t) )`
• **Knobs.** `kappa >= 0` (contrast weight), `mu >= 0` (risk gain). Typical: `kappa = 1.0`, `mu = 3.0`.
• **Range.** `0 < g_t < 1` for `Z_t >= 0` and `mu > 0`. If `mu = 0`, then `g_t = 0`.

Downstream coupling (examples).
```
RSI_env := g_t * RSI
phi_phase := clip((a_phase + 1)/2, 0, 1) // optional phase mask in [0,1]
RSI_env := g_t * RSI * phi_phase // use when kinetics depend on phase side
```
Alert when `g_t <= theta_g` (declare threshold `theta_g`).
Throttle factor `:= g_t` (for example scale actuator setpoints by `g_t`).

Notes.
• **Determinism.** Choose one `S_t` recipe per study; keep all knobs in the manifest.
• **Privacy.** When needed, compute `S_t` using quantized or DP-noised `a_T/a_phase/Q_phase` as declared elsewhere (P1/P2).
• **Interoperability.** If Addendum L.1 (`a_env`) is used, declare `w_env` and keep the fuse rule fixed.

3.2 First-order benefits (what you get on day one)

• **Zero unit drama.** One to-K step, then `e_T` means the same everywhere.
• **Comparable by construction.** `e_T = 0` at `T_ref`; sign gives hot/cold; magnitude is a unitless "how far."
• **Phase-aware safety.** `a_phase` is a smooth near-pivot dial; soft hysteresis reduces flicker.
• **ML-ready features.** `e_T` is centered; `a_T` and `a_phase` are bounded in `(-1, +1)` for stable inputs.
• **Portable thresholds.** Example: **"Heat Advisory if `e_T >= +0.8` for `3 h`."**
• **Auditability.** A tiny **manifest** explains the transform; decisions are replayable and vendor-neutral.
• **Cost relief.** One symbol stream collapses dual-unit code paths, QA matrices, and an incident class.

3.3 Why zero-centric with lenses (the math intuition)

All lenses produce a **unitless**, **monotone** contrast `e_T`, centered at `T_ref`. Pick one, publish it, and keep it fixed.

• **Log lens (default):** scale-invariant across wide ranges.
```
e_T := ln(T_K / T_ref)
```

• **Linear lens (narrow windows):** intuitive near baseline.
```
e_T := (T_K - T_ref) / DeltaT
```

• **Beta lens (cold-sensitive):** emphasizes the cold side.
```
e_T := (T_ref / T_K) - 1
```

• **kBT lens (energy-linked):** ties temperature to an energy scale.
```
e_T := (R*T_K)/E_unit - (R*T_ref)/E_unit
```

• **Hybrid lens (mixed regimes; piecewise):** linear near baseline, log for larger departures.
```
if abs(T_K - T_ref) > tau: e_T := ln(T_K / T_ref) else: e_T := (T_K -
T_ref) / DeltaT
```

• **Quantum-safe log (near 0 K; zero at `T_ref`):** avoids undefined log while keeping
```
e_T(T_ref) = 0.
e_T := ln( (T_K/T_ref + alpha) / (1 + alpha) ) # alpha > 0
```

Design choice. One lens per study or mission keeps analytics stable and removes tuning ambiguity.

3.4 Why bounded alignments (and safe pooling)

Bounded dials simplify dashboards and averaging across devices.

• **Alignment channel (optional).**
```
a_T := tanh(c_T * e_T)
a_T := clamp_a(a_T, eps_a)
```
// enforce `|a_T| <= 1 - eps_a`

• **Fleet or array pooling (associative, order-invariant).**
```
U := sum_i w_i * atanh(a_i)
W := max(sum_i w_i, eps_w)
a_pool := tanh(U / W)
```

This **rapidity mean** preserves bounds, behaves well under regrouping, and de-sensitizes outliers.

3.5 Phase proximity and soft hysteresis (why it is gentler and safer)

A single near-pivot dial removes brittle `0 C` vs `32 F` logic.

• **Dial (single pivot).**
```
d_m := (T_K - T_m) / DeltaT_m
a_phase := tanh(c_m * d_m)
```

• **Optional fused multi-pivot dial (e.g., freezing + boiling).**
```
a_phase_fused := tanh( sum_i( c_m_i * (T_K - T_m_i) / DeltaT_m_i ) )
```
Emit **either** `a_phase` **or** `a_phase_fused`, not both.

• **Soft hysteresis (reduces chatter).**
```
p_side := 0.5 * (1 + tanh(k_side * (T_K - T_m)))
Q_phase := rho*Q_prev + (1 - rho)*clip(p_side, 0, 1)
```

Outcome. Fewer false toggles when hovering around freezing, clearer **risk** and **clear** states.

3.6 Keeping it simple (anti over-engineering guardrails)

• Start with **S1 (Lite):** emit only `e_T` and `manifest_id`.
• If freezing matters, move to **S2:** add `a_phase` and optional `Q_phase`.
• Add **S3** (`a_T`, pooling) only when bounded fleet dials or ML priors help.
• Document each upgrade reason in manifest `notes`. No change to existing `e_T` streams.

3.7 Empirical validation (how to prove it quickly)

Minimal additions let you build evidence without changing runtime logic. Always report **baseline vs SSMT** with identical windows and `dt`.

Baselines (declare once).
• **Raw-freeze:** flag if `T_K <= T_m` (no hysteresis).
• **Raw-hot (log lens comparable):** `T_hot := T_ref*exp(E_hot)`; flag if `T_K >= T_hot`.
• **Raw-anomaly:** `A_K(t) := T_K(t) - median_{30d,same hour}(T_K)`.

Bench A — Unit incidents (ops metric).
Track incident rate before/after SSMT adoption. Expect fewer "wrong unit / wrong threshold" events.
Metrics: `R_inc` (before) → `R_inc'` (after); time-to-triage; `% incidents` attributable to unit mixups.

Bench B — Freeze alert stability (safety metric).
Compare alert chatter near freezing using `a_phase +` soft hysteresis vs raw-freeze baseline.

Metrics: `flicker_rate` (toggles/hour), false positive rate, `dwell_time_risk` (seconds/hour).
Definitions (SSMT).

```
flicker_rate := 3600 * (flips_phase + flips_Q) / window_seconds
dwell_time_risk := sum( dt * 1{ Q_phase >= Q_thr and |T_K - T_m| <= B_K } )
```

Bench C — Threshold portability (analytics/ML metric).
Apply one global hot threshold across stations.
SSMT: flag if `e_T >= E_hot`.
Baseline: flag if `T_K >= T_hot` with `T_hot := T_ref*exp(E_hot)`.
Metrics: per-station `hot_fraction`; spread across stations (std or IQR); number of re-tuning events avoided.

Bench D — Model hygiene (ML metric).
Swap raw temperature with `e_T` (and `a_phase` if relevant) in existing models.
Metrics: outlier rate, feature-scaling ops removed, retrain frequency, drift alarms, AUROC/accuracy deltas.

Bench E — Env-gate benefit for chemistry (optional, cross-spec).
Gated index: `RSI_env := g_t * RSI` (optionally `* phi_phase` with `phi_phase := clip((a_phase + 1)/2, 0, 1)`).
Metrics: variance reduction in RSI-driven decisions; false positives reduced when phase is unfavorable; stability of alerts over time.

Validation hooks.
Keep **dataset** and **test-vector** references in the manifest `validation` block; fix windows and `dt`; compute **baselines and SSMT** with identical slices. Runtime logic stays **purely symbolic**.

3.8 Limitations and failure modes (be explicit)

• **Lens mis-specification.** Poor lens choice can distort scale. **Mitigation:** use log by default; publish a short lens decision note; consider hybrid when regimes mix.
• **Anchor policy drift.** Changing `T_ref` mid-run breaks comparability. **Mitigation:** freeze per study; if policy changes (`seasonal/diurnal`), document upfront.
• **Domain guards.**
– log/beta: require `T_K > 0` and `T_ref > 0`
– linear: require `DeltaT > 0`
– kBT: require `E_unit > 0`
– qlog: require `T_ref > 0` and `alpha > 0`
• **Extremes and numerical safety.** Near `0 K` or very high `T_K`, declare `T_valid_range_K` and apply a small Kelvin floor:
`T_K := max(T_K, eps_TK)` // `eps_TK > 0`
Flag range or guard violations; compute with saturated `T_K_eff` for stability only (no threshold coercion).
• **Multi-pivot overfitting.** Too many pivots can overreact in noise. **Mitigation:** publish a small, named `T_m_list` with rationale; prefer single pivot unless needed.

• **Privacy invertibility.** If anchors are public, `e_T` can be inverted to `T_K`. **Mitigation:** coarser `e_T` precision or bucketed anchors when needed.

3.9 Human vs machine separation (non-negotiable)

• **Machine logic:** consumes only `{ e_T, a_phase, a_T? }` and manifest flags.
• **Human UI:** may render `°C/°F` for comfort, display-only; never feed that back into logic.

3.10 Why this strikes the right balance

SSMT keeps **physics** untouched and simplifies everything downstream: one **stable symbol space** for rules and models, a gentle **phase dial** for safety, and a **manifest** that makes intent obvious. Start with **S1** and add only what is needed while enabling **rigorous, reproducible validation**.

3.11 (Optional) Adoption quick-wins

• Publish **S1** manifests for current sensors; flip machine rules to `e_T`.
• Enable **S2** only on freeze-critical assets; log flicker metrics.
• Add a one-page **lens decision note** per deployment; keep it fixed.
• Record **validation refs** (dataset and test-vectors) in the manifest; no runtime changes required.

---

4) Worked Examples (copy-ready, unitless)

4.1 Unit invariance (same state, different units)

Config. `lens = log`, `T_ref = 298.15 K (25 C)`
Physical state. `35 C = 95 F = 308.15 K`
Compute. `e_T = ln(308.15 / 298.15) = 0.03298996`
Meaning. One number travels across `°C/°F/K` and vendors.

4.2 Near-freezing dial (water)

Config. `T_m = 273.15 K`, `DeltaT_m = 2.0`, `c_m = 1.2`, `eps_a = 1e-6`
`T = 271.15 K -> d_m = -1.0 -> a_phase = tanh(-1.2) = -0.83365461`
`T = 273.15 K -> d_m = 0.0 -> a_phase = 0.0`
`T = 275.15 K -> d_m = +1.0 -> a_phase = tanh(+1.2) = +0.83365461`
Meaning. One smooth dial for "how close to freeze" and which side you are on.

4.3 Soft hysteresis around the pivot (reduced flicker)

Config. `rho = 0.90`, `k_side = 2.0`
Update each step.
```
p_side = 0.5 * (1 + tanh(k_side * (T_K - T_m)))
Q_phase = rho * Q_prev + (1 - rho) * clip(p_side, 0, 1)
```
Meaning. Brief dips below `T_m` do not instantly flip state; clears require sustained warmth.

4.4 Cold-chain excursion (symbolic, unitless)

Config. `lens = linear`, `T_ref = 277.15 K (4 C)`, `DeltaT = 2.0`, `c_T = 0.7`
`T = 281.15 K (8 C) -> e_T = +2.00 -> a_T = tanh(1.4) = +0.88535165`
`T = 275.15 K (2 C) -> e_T = -1.00 -> a_T = tanh(-0.7) = -0.60436778`

Daily symbolic cooling degree-day (threshold `e* = +0.5`).
```
S-CDD := sum_t max(e_T(t) - 0.5, 0)
```
Policy example. "Reject if `S-CDD > 1.5`."

4.5 City-to-city comparability (different climates, same anomaly)

Config (linear lens). both declare `DeltaT = 5.0 K`
`City A: T_ref = 295 K, T = 301 K -> e_T = (301 - 295)/5 = 1.2`
`City B: T_ref = 285 K, T = 291 K -> e_T = (291 - 285)/5 = 1.2`
Meaning. Identical symbolic anomaly despite different baselines.

4.6 Rail “sun-kink” guard (hot excursion)

Config (linear lens). `T_ref = 308.15 K (35 C), DeltaT = 5 K`
`Measured T_rail = 328.15 K (55 C) -> e_T = (328.15 - 308.15)/5 = 4.0`
Rule example. “Speed restrict if `e_T >= 3.0` for `>= 10 min`.”

4.7 Diesel gelling (cloud point pivot)

Config. `T_cloud = 258.15 K (-15 C), DeltaT_m = 2.0, c_m = 1.2`
`Ambient 255.15 K (-18 C) -> d_m = (255.15 - 258.15)/2 = -1.5`
`a_phase_fuel = tanh(1.2 * -1.5) = -0.94680601`
Rule example. “Pre-heat if `a_phase_fuel <= -0.10` for `>= 10 min`.”

4.8 Space ops (cold-sensitive subsystem via beta lens)

Config. `lens = beta, T_ref = 90 K, c_T = 0.7`
`Cryo reading T = 80 K -> e_T = (90/80) - 1 = 0.125`
`a_T = tanh(0.7 * 0.125) = 0.08727737`
Meaning. Beta lens emphasizes cold-side changes without changing physics.

4.9 ML feature swap (hygiene, no re-tuning per unit)

Swap. Replace raw temperature with `e_T` (and `a_phase` if freezing matters).
Immediate effects. Centered feature, bounded dial, fewer outliers, stable thresholds across sources.

4.10 Fault detection (unit flip shock)

If a stream flips `°C↔°F` by mistake, the one-time **to-K** step isolates it as a sudden level shift in `e_T`; drift monitors catch it.
Action. Set `health.sensor_ok = false`, flag `drift = true`, investigate source; thresholds remain in **symbol space**.

4.11 Empirical snapshot (public data, symbol space)

Setup. `log` lens; `T_ref = 298.15 K`; phase dial near freezing with `T_m = 273.15 K`, `DeltaT_m = 2.0 K`, `c_m = 1.2`, `eps_a = 1e-6`; soft hysteresis `rho = 0.90`, `k_side = 2.0`.
Rules. Freeze (symbolic): `a_phase <= -Phi_freeze` with `Phi_freeze = 0.10`. Hot (symbolic): `e_T >= E_hot` with `E_hot = 0.8`.
Results. Across 5 stations, phase-aware rules reduced alert flicker by **10.45%** (`134 -> 120`), with 4/5 stations showing equal or lower flicker. A synthetic `C->F` unit flip produced a detectable step in `e_T` and set `sensor_ok = false`, `drift = true`; symbol-only rules remained stable. In this slice, hot-threshold portability was neutral (no exceedances).
Reproducibility. See **Addendum C** for data license, methods, and how to rerun the slice in symbol space.

---

5) Validation — Tier S1 (Unit Tests & Runnable Spec)

Goal. Prove correctness of the **representation** (math, guards, outputs) without any real-world data. All tests are **synthetic**, **deterministic**, and suitable for **CI**.

5.1 Coverage (no data required)

• **Unit standardization.** `degC/degF/K -> T_K`; Kelvin floor applied: `T_K := max(T_K, eps_TK)`.
• **Lenses.** Zero at `T_ref`, monotone in `T_K`; **hybrid** is deterministic (linear near baseline, log for larger departures); **qlog** stays zero-centric at `T_ref` and avoids `log(0)`.
• **Domain guards.** log/beta require `T_K > 0` and `T_ref > 0`; linear requires `DeltaT > 0`; kBT requires `E_unit > 0`; hybrid requires `tau > 0` (and inherits branch guards); qlog requires `T_ref > 0` and `alpha > 0`.
• **Alignment bounds.** `a_T` and `a_phase` remain strictly in `(-1, +1)` with clamps.
• **Phase dial.** Correct sign and symmetry around `T_m`; optional fused multi-pivot dial stays bounded and interpretable.
• **Soft hysteresis.** Memory dial `Q_phase` evolves smoothly and reduces flicker.
• **Validity handling.** Out-of-range Kelvin sets flags and computes with saturation.
• **Pooling (if used).** Rapidity pooling is bounded and associative.

5.2 Test vectors (copy-paste, minimal)

A) Unit invariance (to-K and lens)
Case A1. `lens = log`, `T_ref = 298.15 K`
Inputs. `35 C`, `95 F`, `308.15 K`
Expect. All yield identical `e_T = ln(308.15/298.15) ≈ 0.03298996` (tol `1e-6`).
Case A2 (Kelvin floor). `to_kelvin(-273.16, "C") -> T_K = eps_TK` (with `eps_TK = 1e-6`).

B) Lens zero and monotonicity
Case B1. Any lens, `T_K = T_ref ⇒ e_T = 0`.
Case B2. `T1 = 290 K < T2 = 300 K ⇒ e_T(T2) > e_T(T1)`.

C) Lens domain guards
Case C1 (log/beta). `T_K <= 0` or `T_ref <= 0 ⇒ health.sensor_ok = false`.
Case C2 (linear). `DeltaT <= 0 ⇒ reject config at init`.
Case C3 (kBT). `E_unit <= 0 ⇒ reject config at init`.
Case C4 (hybrid). `tau <= 0 ⇒ reject config at init`.
Case C5 (qlog). `T_ref <= 0` or `alpha <= 0 ⇒ reject config at init`.

D) Alignment/channel clamps
Given `c_T = 0.7`, `eps_a = 1e-6`.
Case D1. `e_T = +1000 ⇒ a_T = tanh(700) -> clamp to < +1 by eps_a`.
Case D2. `e_T = -1000 ⇒ a_T = tanh(-700) -> clamp to > -1 by eps_a`.

E) Phase dial sign and symmetry
Given `T_m = 273.15`, `DeltaT_m = 2.0`, `c_m = 1.2`, `eps_a = 1e-6`.
Case E1. `T = 271.15 K ⇒ d_m = -1.0 ⇒ a_phase ≈ -0.83365461`.
Case E2. `T = 275.15 K ⇒ d_m = +1.0 ⇒ a_phase ≈ +0.83365461`.
Case E3. `T = 273.15 K ⇒ d_m = 0.0 ⇒ a_phase = 0`.
Case E4 (fused multi-pivot symmetry).
Let `P1 = {T_m=273.15, DeltaT_m=2.0, c_m=1.2}`, `P2 = {T_m=373.15, DeltaT_m=2.0, c_m=1.2}`.
At `T = (273.15 + 373.15)/2 = 323.15 K`, standardized distances are `+25` and `-25` ⇒ sum `= 0` ⇒ `a_phase_fused = tanh(0) = 0`.

F) Soft hysteresis evolution
`rho = 0.90`, `k_side = 2.0`, start `Q_prev = 0.30`.
Sequence of sides `s = [+1,+1,+1,+1,+1,-1,-1,-1,+1,+1]`.
Expect. `Q_phase` increases on warm steps, decays on cold, no oscillatory overshoot.

G) Validity and saturation
`T_valid_range_K = [180, 500]`.
Case G1. `T_K = 170 ⇒ health.range_ok = false`, `oor = "below_min"`; compute with `T_K_eff = 180`.
Case G2. `T_K = 510 ⇒ health.range_ok = false`, `oor = "above_max"`.

H) Pooling safety (if pooling is enabled)
Given `a = [0.2, 0.4, -0.1]`, `w = [1, 1, 1]`.
`a_pool = tanh( mean(atanh(a_i)) )`.
Checks. Associativity (regrouping yields identical `a_pool`); bounds `|a_pool| < 1`.

I) Hybrid branch selection (deterministic)
Config: `lens = hybrid`, `T_ref = 300 K`, `DeltaT = 10 K`, `tau = 5 K`.
Case I1 (linear branch). `T = 302 K` (`|delta| = 2 K <= tau`) ⇒ `e_T = (302 - 300)/10 = 0.2`.
Case I2 (log branch). `T = 310 K` (`|delta| = 10 K > tau`) ⇒ `e_T = ln(310/300) ≈ 0.032789`.

J) qlog zero and monotonicity
Config: `lens = qlog`, `T_ref = 298.15 K`, `alpha = 1.0`.
Case J1. `T = T_ref` ⇒ `e_T = ln((1 + alpha)/(1 + alpha)) = 0`.
Case J2. `T1 = 295 K`, `T2 = 305 K` ⇒ `e_T(T2) > e_T(T1)`.

5.3 Runnable harness (ASCII pseudo-code)

```
# Use absolute/relative tolerances as shown; fail on any mismatch.
tol_abs = 1e-6
tol_pool = 1e-12
R = 8.314462618
eps_TK = 1e-6

# To Kelvin + floor
assert to_kelvin(35, "C") == 308.15
assert abs(to_kelvin(95, "F") - 308.15) < tol_abs
assert to_kelvin(308.15, "K") == 308.15
assert to_kelvin(-273.16, "C") == eps_TK # floor applied

# Lens core (zero and monotone)
assert abs( ln(298.15/298.15) - 0.0 ) < tol_abs
assert ( (300 - 298.15) / 10.0 ) > ( (290 - 298.15) / 10.0 )

# Hybrid branch selection
T_ref = 300.0
DeltaT = 10.0
tau = 5.0
assert abs( encode_eT(302.0, lens="hybrid", T_ref=T_ref, DeltaT=DeltaT,
tau=tau) - 0.2 ) < tol_abs
assert abs( encode_eT(310.0, lens="hybrid", T_ref=T_ref, DeltaT=DeltaT,
tau=tau) - ln(310.0/300.0) ) < tol_abs

# qlog zero and monotone
alpha = 1.0
T_ref = 298.15
assert abs( encode_eT(T_ref, lens="qlog", T_ref=T_ref, alpha=alpha) - 0.0 )
< tol_abs
assert encode_eT(305.0, lens="qlog", T_ref=T_ref, alpha=alpha) >
encode_eT(295.0, lens="qlog", T_ref=T_ref, alpha=alpha)
```

```
# Domain guards
assert invalid_log (T_K=0.0, T_ref=298.15) # sets sensor_ok=false or raises
assert invalid_beta (T_ref=0.0, T_K=298.15)
assert invalid_linear(DeltaT=0.0)
assert invalid_kBT (E_unit=0.0)
assert invalid_hybrid(tau=0.0)
assert invalid_qlog (T_ref=0.0, alpha=1.0) or invalid_qlog(T_ref=298.15,
alpha=0.0)

# Alignment clamps
a_hi = clamp_a( tanh(0.7*1000), 1e-6 )
a_lo = clamp_a( tanh(-0.7*1000), 1e-6 )
assert a_hi <= 1 - 1e-6
assert a_lo >= -1 + 1e-6

# Phase dial
a1 = clamp_a( tanh(1.2 * ((271.15 - 273.15)/2)), 1e-6 )
a2 = clamp_a( tanh(1.2 * ((275.15 - 273.15)/2)), 1e-6 )
assert abs(a1 + 0.83365461) < tol_abs
assert abs(a2 - 0.83365461) < tol_abs

# Fused multi-pivot (symmetry case)
a_fused = tanh( 1.2*((323.15 - 273.15)/2) + 1.2*((323.15 - 373.15)/2) )
assert abs(a_fused - 0.0) < tol_abs

# Soft hysteresis (shape check)
Q = 0.30
for s in [+1,+1,+1,+1,+1,-1,-1,-1,+1,+1]:
p_side = 0.5 * (1 + tanh(2.0 * s))
Q = 0.90*Q + 0.10*clip(p_side, 0, 1)
assert 0.0 <= Q <= 1.0

# Validity
range_ok, oor, e_T_eff = validity_encode(T_K=170, T_min=180, T_max=500,
...)
assert (range_ok == False) and (oor == "below_min")

# Pooling (if used)
a_pool_1 = pool([0.2, 0.4], [1, 1])
a_pool_2 = pool([a_pool_1, -0.1], [2, 1]) # regrouped
a_pool_3 = pool([0.2, pool([0.4, -0.1],[1,1])], [1, 2]) # regrouped
assert abs(a_pool_2 - a_pool_3) < tol_pool
assert abs(a_pool_2) < 1
```

5.4 CI integration

• Run **Tier S1** tests on every change to firmware, pipelines, or manifests.
• **Fail the build** on any domain-guard violation or clamp failure.
• Store **golden outputs** for A–E and compare with tolerances.

5.5 Acceptance

All **asserts pass** under fixed tolerances; any change flags a **spec-breaking edit**. No external data required.

---

6) Mini-Metrics Library (unitless, copy-ready)

A small, opinionated toolkit for portable KPIs built directly on symbols (`e_T`, `a_phase`)—no raw units, no site-specific retuning. These metrics let you compare behavior across devices, buildings, cities, or planets with the same thresholds and windows, and they slot cleanly into dashboards and CI. The library is intentionally simple: robust medians/quantiles instead of fragile means; rolling windows that respect local solar time; and definitions that remain valid whether your cadence is 1-min, 10-min, or hourly (pick `L` accordingly).

What's inside.
• Anomaly and normalization against diurnal or seasonal baselines (unitless).
• Volatility and stability to track variance and time in risk bands.
• Symbolic excursion budgets (degree-day analogs) for policy.
• Regime and flicker KPIs to quantify chattering vs true events.
• Fleet KPIs that aggregate safely across heterogeneous sensors.
• Governance metrics to monitor anchor stability and threshold portability.
All formulas are ASCII, representation-only, and manifest-aware. Publish chosen `E_hot`, `E_cold`, `Phi_freeze`, windows.

6.1 Notation and defaults

`e_T(t)` : unitless contrast at time `t`
`a_phase_star(t)` : chosen phase dial at time `t` (use `a_phase` for single pivot or `a_phase_fused` for multi-pivot)
`lst(t)` : local solar time (or local clock hour)
`L` : window length (default `24 h` for hourly data)
`Q` : quantile operator; median is `Q_0.5`
`P_t(•)` : fraction of time in `[t−L, t]` for which a predicate holds
`MAD(x)` : median absolute deviation `= median(|x − median(x)|)`
Defaults. diurnal window `= 24 h`, volatility window `= 24 h`, rolling medians `= 30 d` (same hour-of-day)

6.2 Core anomaly and normalization

Anomaly vs seasonal or diurnal norm.

```
A_T(t) := e_T(t) − median_{d in past 30 days, same hour}( e_T(d, hour(lst(t))) )
```

Diurnal-normalized anomaly (if maintaining a diurnal mean).

```
e_T_star(t) := e_T(t) − mu_diurnal( hour(lst(t)) )
```

Robust z-score.

```
Z_T(t) := ( e_T(t) − median_{30d}(e_T | same hour) ) / MAD_{30d}(e_T | same hour)
```

Optional normal scaling: divide `MAD` by `k_MAD = 1.4826` if you want approximately `N(0,1)` under Gaussian assumptions.

6.3 Volatility and stability

Rolling volatility over window `L`.

```
V_T(t) := sqrt( Var_{tau in [t−L, t]}( e_T(tau) ) )
```

Phase-band dwell near pivot.

```
TimeInPhaseBand := fraction_{tau in [t−L, t]}( |a_phase_star(tau)| <= phi_star )
```

Use `phi_star = Phi_freeze` unless specified.

6.4 Symbolic excursion budgets (degree-day analogs)

Cooling-like excursions above a symbolic threshold `e*`.

```
S-CDD(t1..t2) := sum_{tau=t1..t2} max( e_T(tau) − e* , 0 )
```

Heating-like excursions below `−e*`.

```
S-HDD(t1..t2) := sum_{tau=t1..t2} max( −e_T(tau) − e* , 0 )
```

6.5 Regimes and flicker

Regime tagging.

```
Regime(t) :=
  "HOT"    if e_T(t) >= E_hot
  "COLD"   if e_T(t) <= -E_cold
  "NORMAL" otherwise
```

Flicker count over `[t−L, t]`.
`Flicker := number_of_changes_in( Regime(tau) for tau in [t−L, t] )`
Target: with soft hysteresis applied to decisions, `Flicker` decreases while true positives remain.

6.6 Fleet KPIs (portable across sites)

```
HotFraction := P_t( e_T >= E_hot )
ColdFraction := P_t( e_T <= -E_cold )
RiskPhaseFraction := P_t( a_phase_star <= -Phi_freeze )
MaxPhaseIntrusion := max_{tau in [t−L, t]}( -a_phase_star(tau) )
```
`ClearLatency := min { Δt >= 0 : a_phase_star(t+Δt) >= +Phi_clear }` (measured after entering risk)

6.7 Stability and governance metrics

Anchor stability (for declared seasonal or diurnal policies).
`DriftFlag := 1 if | median_{[t−L, t]}( e_T ) | > E_drift else 0`

Threshold portability across sites `s` (using common `E_hot`, `E_cold`, `Phi_freeze`).
`PortabilitySpread := max_s( Precision_s ) − min_s( Precision_s )`

Guard or quality monitors (symbol pipeline health).
```
GuardViolationRate := P_t( (health.sensor_ok == false) OR (health.range_ok
== false) )
OORFraction := P_t( oor != false )
LensConsistency := 1 if count_unique(resolved_lens over [t−L, t]) == 1 else
0
```
Smaller `PortabilitySpread` and `GuardViolationRate` are better. `LensConsistency` should remain `1`.

6.8 Suggested defaults (tunable)

`E_hot = 0.8`, `E_cold = 0.8`, `Phi_freeze = 0.10`, `Phi_clear = 0.10`, `e* = 0.5`, `L = 24 h`.
Publish chosen values in policy or manifest notes.

6.9 Presentation guidance

• Always label axes in symbol space (`e_T`, `a_phase`).
• For human-facing displays, you may show `°C/°F` alongside symbols; logic remains symbolic.
• Prefer medians and quantiles over means for robustness across mixed sensors.

6.10 Implementation notes (optional, practical)

• **Missing data.** Compute medians or quantiles with at least `N_min` samples per hour-of-day; otherwise mark KPI as `insufficient_data`.
• **Cadence changes.** Resample to a common cadence before computing windows; use forward-fill with a `gap_max` guard, or drop stretches that exceed `gap_max`.
• **Window alignment.** Use half-open intervals `[t-L, t)` for streaming safety; document this in dashboards.
• **Manifest awareness.** Store exact `E_hot`, `E_cold`, `Phi_freeze`, `Phi_clear`, `e*`, `L`, and the resolved lens with parameters (for example `tau` for hybrid, `alpha` for qlog) alongside KPIs for audit replay.
• **Multi-pivot phase.** Define `a_phase_star := a_phase_fused` when a multi-pivot phase dial is configured; otherwise use `a_phase`.

6.11 Reference pseudo-code (copy-ready)

```
# Choose the phase dial for KPIs
phase_dial(t):
  if multi_pivot_enabled: return a_phase_fused(t)
  else:                   return a_phase(t)

# Fleet KPIs (window [t-L, t))
HotFraction(t, L, E_hot):
  return frac{ tau in [t-L, t) : e_T(tau) >= E_hot }

RiskPhaseFraction(t, L, Phi_freeze):
  return frac{ tau in [t-L, t) : phase_dial(tau) <= -Phi_freeze }

MaxPhaseIntrusion(t, L):
  return max_{tau in [t-L, t)}( -phase_dial(tau) )

# Governance
GuardViolationRate(t, L):
  return frac{ tau in [t-L, t) : (health.sensor_ok(tau)==false) or
(health.range_ok(tau)==false) }

OORFraction(t, L):
  return frac{ tau in [t-L, t) : oor(tau) != false }

LensConsistency(t, L):
  return 1 if count_unique({ resolved_lens(tau) for tau in [t-L, t) }) == 1
else 0
```

7) Deployment Patterns

Practical, copy-ready blueprints for rolling out SSMT across common domains. Each pattern lists **Inputs**, the symbol **Emit** you should produce, and a minimal **Rules** set expressed entirely in **symbol space**.

7.1 Weather and climate operations

Inputs. Raw temps from stations; optional seasonal `T_ref(lat, lon, doy)`.
Emit. `e_T` (log lens default); optionally `a_phase` for freezing.
Rules (examples).
• Heat Advisory: `e_T >= +0.8` for `>= 3 h`
• Cold Advisory: `e_T <= -0.8` for `>= 3 h`
• Freeze Risk (surface): `a_phase <= -0.10` for `>= 20 min`
Notes. One lens per deployment (publish and keep fixed). Use **log** by default; if your domain mixes small and large departures around baseline, consider **hybrid** and publish `tau`. If operating near absolute zero, consider **qlog** and publish `alpha`. Publish how `T_ref` is set (fixed vs seasonal). Keep UI in `°C/°F` for people; machine logic uses **symbols**.

7.2 Cities and mobility (roads, rail, aviation)

Inputs. Pavement/rail/wing surface temps; material pivots with `T_m_tag_list` (water, brine, diesel cloud, alloy neutral).
Emit. `e_T`, `a_phase` (or `a_phase_fused` if using multiple pivots), `Q_phase`.
Road example.
• Black Ice Risk: `a_phase_road <= -0.15` for `>= 20 min`
• Clear Ice Risk: `CLEAR_FREEZE` when `a_phase_road >= +0.10` for `>= 20 min`
Rail example.
• Define `T_neutral` and linear lens with `DeltaT`.

```
e_T_rail := ( T_rail - T_neutral ) / DeltaT
```

• Speed Restrict: `e_T_rail >= +3.0` for `>= 10 min`
Aviation example.
• De-ice Hold: `a_phase_wing <= -0.05` ; **Release:** `Q_phase >= 0.70`

7.3 Supply chain, food safety, and cold chain

Inputs. Logger temps; product-specific `T_m` where relevant.
Emit. `e_T` (linear lens common), optional `a_phase`, `Q_phase`.

Rules.
• Daily symbolic cooling degree-day: `S-CDD := sum_t max( e_T(t) - e* , 0 )` → **Reject** if `S-CDD > 1.5`
• Freeze excursion: `a_phase_product <= -0.05` for `> 15 min`
Notes. Include `manifest_id` in every record for audit replay.

7.4 Healthcare and pharma (storage, transport, wards)

Inputs. Device temps; choose lens per appliance; optional `T_m` for biologics.
Emit. `e_T` everywhere; `a_phase` where freeze or overheating matters.
Rules.
• Cold-chain cabinet: `e_T in [-1.0, +0.5]` most of day; **alert** if outside for `>= 10 min`
• Bio warm-chain: `e_T >= +0.8` for `>= 15 min` → **quarantine**

7.5 Buildings and HVAC

Inputs. Indoor/outdoor temps; choose seasonal or diurnal-corrected `T_ref`.
Emit. `e_T` for indoor and outdoor; compute `S-CDD`, `S-HDD` for planning.
Rules.
• Comfort alarm: `|e_T_indoor| >= 0.7` for `>= 15 min`
• Demand forecast driver: use `S-CDD` and `S-HDD` in unitless planning models.

7.6 Utilities and data centers

Inputs. Inlet/outlet temps per rack/zone.
Emit. `e_T`, optional `a_T` for bounded dashboards, `V_T` (rolling volatility).
Rules.
• Thermal excursion: `e_T_inlet >= +0.6` **OR** `V_T(10 min) >= v*`
• Clear: `e_T_inlet <= +0.6 - E_hyst` for `>= 10 min`
Note. Publish `v*` and `E_hyst > 0` in policy.

7.7 Space and aerospace

Inputs. Subsystem-specific temps; pick lens per regime (`beta` for cold-sensitive cryo, `log` for avionics; `qlog` near absolute-zero safety).
Emit. `e_T` per subsystem; `a_phase` for propellants; `a_T` if pooling across sensors.
Rules.
• Cryo fill inhibit: `a_phase_prop <= -0.02` **OR** `e_T_beta >= E_cold*`
• Avionics hot alert: `e_T >= +0.8` for `>= 5 min`

7.8 IoT and firmware

Inputs. Device sensor readings; unit standardization happens on-device.
Core on-device transforms (copy-ready).

```
T_K = to_kelvin(T_raw, unit_raw)
T_K = max(T_K, eps_TK) # Kelvin floor; publish eps_TK if non-default
e_T = ln(T_K / T_ref) # log lens (S1 default)
# Optional (enable only if manifest says so)
a_phase = tanh( c_m * (T_K - T_m) / DeltaT_m )
p_side = 0.5 * ( 1 + tanh( k_side * (T_K - T_m) ) )
Q_phase = rho * Q_prev + (1 - rho) * clip(p_side, 0, 1)
a_T = tanh( c_T * e_T )
```

Emit (minimal stable payload).

```
{ timestamp_utc, e_T, a_T?, a_phase? (or a_phase_fused?), Q_phase?,
T_m_tag_list?, manifest_id, health:{range_ok, drift, sensor_ok}, oor? }
```

• Machine logic consumes **symbols only**; UIs **MAY** render `°C/°F` for display.
• Emit optional uncertainty only if available (see Addendum L.2).

MQTT mapping (copy-paste).

```
topic pattern: ssmt/{station_id}/{profile} # profile = s1 | s2 | s3
```

Recommended: `QoS 1`, `retained=false`, UTF-8 JSON

S1 (Lite) payload.

```
{
  "timestamp_utc": "2025-09-24T12:00:00Z",
  "e_T": 0.0331,
  "manifest_id": "SSMT-CITY-2025-09-001",
  "health": { "range_ok": true, "sensor_ok": true, "drift": false },
  "oor": false
}
```

S2 (with phase + soft hysteresis).

```
{
  "timestamp_utc": "2025-09-24T12:00:00Z",
  "e_T": 0.0331,
  "a_phase": -0.12,
  "Q_phase": 0.31,
  "T_m_tag": "water",
  "manifest_id": "SSMT-COLDCHAIN-2025-09-007",
  "health": { "range_ok": true, "sensor_ok": true, "drift": false },
  "oor": false
}
```

Timing & ordering.
• `timestamp_utc` MUST be ISO-8601 with `Z`. Use RTC + NTP/GNSS sync; monotone non-decreasing timestamps per stream.
• If offline, buffer in order and publish on reconnect; **do not resample symbols**.

Determinism & manifests.
• One lens per firmware profile; **do not switch per record**.
• Every payload carries `manifest_id` that resolves to the fixed recipe (Section 2).
• If a manifest hash is used, log it locally with firmware build info.

Safety & guards (on-device).
• Apply domain checks before emit (see 2.6):
```
log/beta: T_K > 0 and T_ref > 0
linear: DeltaT > 0
kBT: E_unit > 0
qlog: T_ref > 0 and alpha > 0
```
• Set `health.range_ok = false` and `oor = "below_min" | "above_max"` when outside `T_valid_range_K`; compute with saturated `T_K_eff` only for stability.

Performance notes.
• Use IEEE-754 float or fixed-point; precompute constants such as `1/DeltaT`, `1/T_ref`.
• Avoid per-sample dynamic allocation; keep `Q_phase` and any gate memory in a tiny state struct.
• Emit at sensor cadence; **do not downsample symbols** unless declared.

7.9 Research and ML

Inputs. Historical tables with temperature columns.
Emit/Use. Replace raw temperature features with `e_T`; add `a_phase` if freeze matters.
Practices.
• Standardize thresholds across datasets **in symbol space**.
• Track `PortabilitySpread` of precision/recall across sites with a fixed `E_hot`, `E_cold`.

7.10 Policy snippets (copy-paste)

• Heat Advisory: `e_T >= +E_hot` for `>= T_hot_min`
• Cold Advisory: `e_T <= -E_cold` for `>= T_cold_min`
• Freeze Risk: `a_phase <= -Phi_freeze` for `>= T_freeze_min`
• Clear states:
`CLEAR_HEAT` when `e_T <= +E_hot - E_hyst` for `>= T_clear`
`CLEAR_COLD` when `e_T >= -E_cold + E_hyst` for `>= T_clear`
`CLEAR_FREEZE` when `a_phase >= +Phi_clear` for `>= T_clear`

7.11 Commissioning checklist per site

• Pick **one lens**; declare anchors and `T_valid_range_K`. If **hybrid**, publish `tau`; if **qlog**, publish `alpha`.
• Resolve the lens once at commissioning (fixed window) and **freeze** it for the run.

• Decide `T_ref` policy (fixed, seasonal, or diurnal-corrected).
• Declare pivots `T_m` where relevant; set `DeltaT_m`, `c_m`, `k_side`, `rho`.
• Publish thresholds (`E_hot`, `E_cold`, `Phi_freeze`, `E_hyst`, dwell times).
• Attach `manifest_id` to every stream; enable health flags; publish `eps_TK` if not default.

7.12 Safe rollout pattern (shadow mode -> cutover) (optional but recommended)

Phase A (shadow). Compute `e_T/a_phase` alongside legacy rules; log only. Compare KPIs (`Flicker`, precision/recall).
Phase B (hybrid). Gate alerts with `legacy OR SSMT`, require human ack; monitor misses and false positives.
Phase C (cutover). Retire unit-based thresholds; keep a read-only shadow of legacy for `2–4 weeks` for audit.
Artifacts. Store validation refs in the manifest validation block; publish change log.

7.13 Edge–cloud topology (reference) (optional)

• **Edge (device/gateway).** `to_kelvin` → `T_K = max(T_K, eps_TK)` → `encode_eT` (+ optional `a_phase`, `Q_phase`) → emit JSON payload with `manifest_id`.
• **Backhaul.** Compress + sign payload; deliver over MQTT/HTTPS with retry.
• **Cloud (ingest).** Schema-validate against manifest; compute KPIs (`S-CDD`, `V_T`, `Flicker`).
• **Serving.** Dashboards render symbols; any `°C/°F` is display-only.
• **Governance.** Daily job writes KPI snapshots + manifest hash; alerts if drift/guards trip.

---

8) Guardrails & Governance

Normative rules and operational practices that keep SSMT simple, auditable, and safe at scale. Terms **“MUST”**, **“SHOULD”**, and **“MAY”** are used in their standard normative sense.

8.1 Principles (normative)

• **Single lens per study.** A deployment **MUST** choose exactly one lens and keep it fixed.
• **Symbol-only logic.** Machine logic **MUST** consume only `{ e_T, a_phase (or a_phase_fused), a_T? }` and manifest flags; human UIs **MAY** render `°C/°F` for display only.

• **Publish anchors.** `T_ref` (and `DeltaT` or `E_unit` if applicable) **MUST** be published. If **hybrid**, publish `tau`; if **qlog**, publish `alpha`.
• **Validity first.** `T_valid_range_K` **MUST** be declared; out-of-range readings **MUST** set `health.range_ok = false`.
• **Kelvin floor.** After unit conversion, a small floor **MUST** be applied: `T_K = max( T_K , eps_TK )`. If non-default, `eps_TK` **MUST** be published.
• **Clamps are explicit.** If emitting `a_T` or `a_phase`, `eps_a` **MUST** be published and clamps enforced.
• **Auditability.** Every record **MUST** carry `manifest_id` that resolves to the full recipe.
• **Simplicity by default.** Teams **SHOULD** start at **S1** (`e_T` only) and upgrade only for explicit needs.

8.2 Compliance levels

• **S1 (Minimal).** `e_T` + `manifest_id` + `health.range_ok`. No phase dial, no alignment channel.
• **S2 (Phase-aware).** S1 + `a_phase` (+ optional `Q_phase`) where freezing/melting matters (single pivot or fused multi-pivot).
• **S3 (Full).** S2 + `a_T` (bounded alignment) and optional pooling.
Upgrade rule. A higher level **MUST NOT** change the meaning of existing fields; it only adds fields.

8.3 Change control and versioning

• **Manifest version.** Include `version: "1.1"` (string).
• **No mid-run lens changes.** Changing lens or anchor policy mid-run is prohibited; start a new study with a new `manifest_id`.
• **Backward compatibility.** New fields **MUST** be optional with sensible defaults.
• **Deprecations.** Announce at least 12 months before removing a field; keep shims during the window.

8.3.1 Manifest integrity (normative, recommended)

• Emit `manifest_hash` (SHA-256 or better) alongside `manifest_id`; consumers **SHOULD** log both for replayability.
• Distinguish **spec version** (document, e.g., `vX.Y`) from **schema version** (e.g., `"1.1"`); both **MAY** appear in notes.

8.3.2 On-chain registry (optional, informative)

Purpose. Provide a tamper-evident audit trail for manifests by recording a compact fingerprint on an append-only ledger (public chain or internal append-only log).

What to publish (metadata only).
• `manifest_hash` (SHA-256 or better)
• `anchors_policy` (e.g., `"full"` | `"bucketed"` | `"withheld"`)
• `spec_version` (document, e.g., `vX.Y`)
• `schema_version` (e.g., `"1.1"`)
• `manifest_id_prefix` (vendor/tenant scope; no PII)
• `timestamp_utc` (ISO-8601, `Z`)
• `repo_ref` (e.g., commit SHA or release tag)

Emitter payload (copy-ready JSON).

```
{
  "manifest_hash": "sha256:...hex...",
  "anchors_policy": "bucketed",
  "spec_version": "vX.Y",
  "schema_version": "1.1",
  "manifest_id_prefix": "ACME-WX",
  "timestamp_utc": "2025-09-26T00:00:00Z",
  "repo_ref": "github:org/repo@vX.Y"
}
```

Ledger write and local record.
• Write the payload to the ledger and obtain `tx_id`.
• In your manifest notes (or a small registry block), record:
`registry: { system: "append-only-ledger", tx_id: "<txid>", network: "<name-or-url>" }`

Consumer verification (normative steps).

1. Recompute SHA-256 of the exact manifest bytes → `manifest_hash'`.
2. Fetch the ledger payload for `tx_id`.
3. Check `manifest_hash' == manifest_hash` in ledger record.
4. Check `spec_version` and `schema_version` match what the device claims.
5. Log verification outcome alongside `manifest_id`.

Privacy & scope (important).
• Do **not** publish raw `e_T`, `a_T`, `a_phase`, or any PII.
• Publishing `anchors_policy` can aid audit without exposing anchors themselves (see Addendum J).
• If anchors are withheld, the ledger still proves when and what policy governed the stream.

Failure and reconciliation.
• If hash mismatch: quarantine the stream, re-fetch the manifest from source control, and re-encode hash; if still mismatched, treat as integrity incident.
• If ledger unavailable: continue with local verification; queue a retry and mark status `"ledger_pending"`.

Governance (recommended).
• Restrict who can post (signing key or org account).
• Rate-limit and monitor for replay attempts.
• Rotate keys with overlap; publish rotation events on the same ledger.

Alternative (if no blockchain).
• Use an internal append-only Merkle log (same fields; expose root hash per release).

8.4 Domain guards and numeric safety (must)

• **Lens domains.**
`log` and `beta`: require `T_K > 0` and `T_ref > 0`.
`linear`: require `DeltaT > 0`.
`kBT`: require `E_unit > 0`.
`hybrid`: require `tau > 0`; linear branch requires `DeltaT > 0`; log branch requires `T_K > 0` and `T_ref > 0`.
`qlog`: require `T_ref > 0` and `alpha > 0` (zero at `T_ref`, safe as `T_K -> 0`).
• **Clamps.** Enforce `|a_T| <= 1 - eps_a` and `|a_phase| <= 1 - eps_a`, with `eps_a in (0, 1e-3]`.
• **Saturation on OOR.** Compute with `T_K_eff = min( max( T_K , T_min ) , T_max )`; set `oor` to `"below_min"` or `"above_max"`.
• **Kelvin floor.** After conversion, enforce `T_K = max( T_K , eps_TK )` to avoid numerical hazards near `0 K`.
• **Hysteresis.** If used, `rho in (0,1)`; `k_side > 0`.

8.4.1 Extreme regimes (limits, normative)

• **Applicability.** Cryogenic (`< 100 K`), very hot (`> 1000 K`), plasma/exotic media, high-vacuum, or radiation-intense environments.
• **Lens choice.** Teams **SHOULD** prefer `beta` or `kBT` in these regimes; `qlog` **MAY** be used near absolute zero for numeric safety (publish `alpha`); `log/linear` **MAY** be used only within an explicitly published `T_valid_range_K` where monotonicity and sensitivity are acceptable.
• **Guards (MUST).** Declare `T_valid_range_K = [T_min, T_max]` per subsystem. For `kBT`, publish `E_unit`. Add a short `lens_valid_note` explaining the chosen range and trade-offs.
• **Saturation + flag (MUST).** Compute with `T_K_eff = min( max( T_K , T_min ) , T_max )` and set `oor` to `"below_min"` or `"above_max"`. Thresholds **MUST NOT** be coerced; decisions should honor the `oor` flag.
• **Phase pivots.** Each material pivot `T_m` **MUST** be physically justified. If phase behavior is non-classical or poorly known, omit `a_phase` or replace with a banded proxy dial.
• **Precision.** Publish `precision_K` and `precision_e_T` (or an `e_T_quantize` policy) so consumers do not over-interpret digits in extreme regimes.
• **Validation (MUST).** CI includes: lens monotonicity over `[T_min, T_max]`, pivot symmetry at `T_m` (if used), and failure injection for sensor lag/self-heating.

• **Documentation.** Manifest notes **MUST** state “extreme regime” and reference the validation artifacts.

8.5 CI and conformance (must)

• **Tier S1 tests.** Unit invariance, lens zero/monotonicity, domain guards, clamps, phase symmetry, soft hysteresis shape, validity/saturation, Kelvin floor present, and (if hybrid/qlog selected) branch/parameter guards.
• **Golden vectors.** Store expected outputs for key cases; CI **MUST** fail on mismatch beyond tolerances.
• **Static checks.** Manifests **MUST** pass a schema validator before deployment, including `resolved_lens` constancy across the run.

8.5.1 Tolerances (normative, copy-ready)

`tol_abs = 1e-6` for value comparisons in symbol space; `tol_pool = 1e-12` for pooling associativity checks.

8.6 Security and privacy

• **No PII in `manifest_id`.**
`manifest_id` must never contain personal data, location-identifying strings, device owner name, or anything that can be tied back to an individual. It is an internal reference to a declared manifest (lens, pivots, ranges, clamps), not an identity tag.

• **Attribution of raw units.**
Device-side conversion to Kelvin is allowed, and strongly encouraged. Raw °C/°F MUST NOT be persisted in machine-logic stores unless it is strictly required for regulated human reporting. Machine-facing logic, alerting, routing, and governance SHOULD operate purely on `T_K`, `e_T`, `a_phase`, `a_phase_fused`, `a_T`, and related symbolic fields.

• **Invertibility notice.**
Because `e_T` is monotone with respect to `T_K` under a declared lens (for example, `e_T := ln( T_K / T_ref )`), and because manifests publish anchors like `T_ref`, `DeltaT`, `E_unit`, and lens type, `T_K` can be approximately reconstructed from `e_T`. This is expected.
If external sharing must limit reconstruction of raw temperature, teams MAY:
– Reduce precision of shared `e_T` (for example, rounding or bucketing), and/or
– Withhold or bucket anchors (for example, publish `T_ref` as a band instead of an exact value), and/or
– Emit only bounded dials such as `a_phase` or `a_phase_fused` for public dashboards.
These steps reduce direct invertibility but also reduce absolute auditability. This trade-off must be explicitly declared in policy.

• **Time integrity.**
All emitted records MUST include `timestamp_utc` with synchronized clocks (for example, NTP or GNSS disciplined). Downstream audit, replay, and incident review depend on this. Missing or drifting time makes symbol streams legally and operationally weaker.

• **Transport and integrity.**
Telemetry SHOULD be signed and transmitted over TLS or an equivalent secure transport. Consumers SHOULD verify signatures and `manifest_hash` (or equivalent manifest integrity field) before trusting incoming data. The integrity check must bind the symbol stream (`e_T`, `a_phase`, etc.) to the specific manifest that declared its lens, pivots, and ranges.

8.6.1 Privacy note (normative)

SSMT reduces surface exposure of raw temperatures, but it is not a privacy system.

`e_T` is intentionally auditable and therefore intentionally reconstructable if anchors are public. For example, with the default log lens:

```
e_T := ln( T_K / T_ref )
```
can be rearranged as
```
T_K := T_ref * exp( e_T ).
```

Because of that, you MUST NOT market SSMT as "privacy by default." It is not encryption and it is not anonymization.

For data sharing outside your trust boundary:

• Prefer emitting `a_phase`, `a_phase_fused`, or a banded/rounded `e_T` when the exact `T_K` value is sensitive (for example, patient-adjacent telemetry, secure facility thermal signatures, mission hardware states).
• If you intentionally obscure `e_T` (for example, `e_pub := e_T + salt_bias`) or obscure anchors (for example, bucketed `T_ref`), you MUST declare that policy. Do not claim full auditability if you are publishing only obfuscated or lossy signals.

• You MUST clearly state which streams are raw/auditable vs which are privacy-shifted.

In plain terms: **SSMT gives you a common language. It does not remove your obligation to govern who hears it.**

8.6.2 Data retention (recommended)

• **Raw Kelvin retention.**
Keep raw Kelvin `T_K` values only as long as necessary for calibration evidence, regulatory reporting, and human-facing obligations. After that retention window, purge them according to policy.

• **Symbol stream retention.**
Retain the emitted symbolic streams (`e_T`, `a_phase`, `a_phase_fused`, `a_T`, `Q_phase`, `health`, `timestamp_utc`, `manifest_id`) and the corresponding manifests (including declared pivots, declared lens, clamps, ranges, and any privacy offsets) for the full regulatory window.

• **Rationale.**
Long-term traceability should rely on the symbolic stream plus its manifest, not on bulk raw °C/°F logs. This protects privacy, reduces storage cost, and keeps review focused on the governance surface (for example, "Did we violate our declared safe band? When? For how long?").

8.6.3 Accessibility & inclusion (non-normative)

Goal.
Make SSMT usable by every operator, auditor, policymaker, and responder — including those using screen readers, plain-text terminals, low-vision tools, or translated UIs — without changing the math.

• **ASCII-first formulas (copy-ready).**
```
e_T := ln( T_K / T_ref )
a_phase := tanh( c_m * ( (T_K - T_m) / DeltaT_m ) )
```
Speakable cue for screen readers:
"e underscore T equals natural log of T sub K over T sub ref."
"a underscore phase equals tanh of c sub m times open parenthesis T sub K minus T sub m close parenthesis divided by Delta T sub m."

• **Screen-reader structure.**
Keep definitions short and self-contained:

1. define the symbol,
2. give the formula in plain ASCII,
3. describe what positive, negative, or zero means.
   Present these in logical order (definition → example → guard). Provide a plain-text "Policy Pack" alongside any figure or dashboard.

• **Figures with text equivalents.**
Every plot, band graph, or dial must ship with alt-text, a compact key-value table, and direct numeric labels for thresholds. Do not rely on a legend that requires color vision.

• **Color and contrast.**
Do not encode safety states using color alone. Use line style, markers, and explicit numeric labels such as "`a_phase = -0.7 cold-side risk`" and "`a_phase = +0.8 hot-side risk`."

• **Numeric presentation.**
Machine logic uses full precision internally. For human display:
– Temperatures (if shown) may be rounded to 0.1 C.
– Symbolic quantities (`e_T`, `a_phase`, etc.) should be shown with at least 6 decimal places if they drive policy or audit.
– Timestamps must be ISO-8601 in UTC (`timestamp_utc`) so humans and machines refer to the same moment.

• **Plain-text friendliness.**
Avoid exotic glyphs in required fields. Keep variable names short, repeatable, and spelled the same way across documents, dashboards, and policy.

• **Internationalization.**
Payloads MUST use dot decimals and UTC timestamps. User interfaces MAY localize units or language for display, but the underlying stream must remain machine-stable and language-agnostic.

• **Example alt-texts (copy/paste).**
– "e_T vs T_K for multiple lenses: all curves cross e_T = 0 at T_ref = 298.15 K. The log lens scales smoothly across a wide range. The beta lens exaggerates cold-side deviation. The quantum-safe log lens remains stable near 0 K."
– "a_phase near freezing: bounded tanh dial centered at T_m = 273.15 K with a ±2 K softness band. Output is always between -1 and +1. Negative values mean cold-side risk (below pivot). Positive values mean hot-side risk (above pivot)."

Accessibility checklist (release gate).
[ ] ASCII formulas are present and speakable.
[ ] Alt-text and a small numeric key accompany every figure.
[ ] No color-only encodings for safety states; thresholds are explicitly labeled.
[ ] All timestamps are ISO-8601 UTC; dot decimals are used in payloads.
[ ] A plain-text Policy Pack (what each dial means and when to escalate) is included.
[ ] Glossary variables (`T_K`, `e_T`, `a_phase`, `a_phase_fused`, `Q_phase`, `rho`, etc.) match exactly across spec, dashboard, and policy.

8.7 Simplicity and anti over-engineering

• Start **S1**; remain there unless one of these is true:
– **Freezing matters.** Then adopt **S2** (`a_phase`).
– **Fleet pooling or bounded dashboards needed.** Then adopt **S3** (`a_T`).
• Every upgrade **MUST** include a one-line rationale in the manifest notes.
• Avoid adding unused pivots (`T_m_list`) or dormant fields.

8.8 Audit and traceability

• **Replayability.** Given `manifest_id` and raw Kelvin, a third party **MUST** be able to reproduce `e_T` (and `a_phase/a_T` if present) bit-for-bit within declared tolerances.
• **Event provenance.** Alerts and decisions **MUST** reference the exact symbol thresholds (e.g., `E_hot`, `Phi_freeze`) and the `manifest_id` (and **SHOULD** log `manifest_hash`).
• **Drift policy.** Define a drift guard such as `|median_{24h}(e_T)| <= E_drift`; emit `health.drift = true` if violated.
• **Artifact binding.** Store `validation.dataset_ref`, `test_vectors_ref`, and `manifest_hash` with each KPI snapshot.
• **Lens consistency.** Systems **SHOULD** record the `resolved_lens` and confirm it remains constant for the run.

8.9 Procurement and contract snippet (copy-paste)

Devices MUST emit SSMT payloads:
`{ timestamp_utc, e_T, a_phase?, Q_phase?, a_T?, T_m_tag_list?, manifest_id, health:{ range_ok, sensor_ok, drift }, oor? }`
Machine logic SHALL consume only SSMT symbols.
Human interfaces MAY render local units for display; display-only.
Manifests SHALL declare:

```
{
  version,
  lens, # "log" | "linear" | "beta" | "kBT" | "hybrid" | "qlog"
  anchors:{ T_ref, DeltaT?, E_unit?, c_T?, tau?, alpha? },
  guards:{ clamp_a_eps, T_valid_range_K, eps_w?, eps_TK? },
  phase_block:{ T_m_list?, rho?, k_side? },
  outputs:{ emit_e_T, emit_a_phase?, emit_Q_phase?, emit_a_T? }
}
```

8.10 Governance roles and RACI

• **Spec owner (A/R).** Maintains schema, lenses, and compliance levels.
• **Data steward (R).** Curates manifests, validates anchors and pivots.
• **Ops owner (R).** Enforces CI, monitors health flags, manages incidents.
• **Security (C).** Reviews privacy posture and time integrity.
• **Partners (I).** Receive stable contracts in symbol space.

8.11 Incident playbook (symbol-first)

• **Unit flip suspected.** Look for sudden `e_T` level shift; set `health.sensor_ok = false`; quarantine source; replay with correct to-K mapping.

• **Flicker near freezing.** Enable soft hysteresis (`rho`, `k_side`) and dwell-based clear rules; verify `a_phase` banding.
• **Out-of-range readings.** Rely on saturation + flags; do not coerce thresholds.
• **Lens mis-specification.** If discovered, stop the study, archive with notes, and relaunch with a new `manifest_id` and lens decision memo.
• **Near-zero anomalies.** Verify qlog parameters (`alpha`) and `eps_TK` floor; confirm guards for `T_ref > 0`.

8.12 Sunset and archive

• At study end, freeze the manifest and archive symbol thresholds and validation vectors.
• New studies **SHOULD** receive new `manifest_ids` even if parameters are unchanged, to preserve provenance.

---

9) Disaster-Prevention Playbook

Copy-ready, symbol-only recipes for common temperature-related hazards. Each entry names the **risk**, specifies **inputs**, defines a **guard in symbol space**, sets an **action rule with dwell time**, and states clear **conditions with hysteresis**. Replace placeholders (e.g., `E_hot`, `Phi_freeze`, `T_hot_min`) with your policy values.

Playbook template (fill and reuse)
Risk: `<short name>`
Inputs: `<sensors/material pivots>`
Guard (symbol space): `<e_T / a_phase expression>`
Action (enter): `<condition for N minutes>`
Clear (exit): `<condition for M minutes>`
Notes: `<sensor placement, dwell, human comms>`

9.1 Road and runway icing (water/brine)

Risk: black ice / surface refreeze
Inputs: surface temperature; `T_m_tag = water` or `brine`
Guard: `a_phase_surface = tanh( c_m * ( (T_surface - T_m) / DeltaT_m ) )`
Action (enter): `a_phase_surface <= -Phi_freeze` for `>= T_freeze_min`
Clear (exit): `a_phase_surface >= +Phi_clear` for `>= T_clear`
Notes: pick `T_m` for water (`273.15 K`) or local brine; mount sensors where shade persists.
Multi-pivot option: if both water and brine matter, you **MAY** use a fused dial `a_phase_fused = tanh( sum_i c_m_i * ( (T_surface - T_m_i) / DeltaT_m_i ) )` and apply the same enter/clear rules to `a_phase_fused`.

9.2 Aviation de-icing holdover

Risk: loss of anti-ice effectiveness on wings
Inputs: wing surface temperature; `T_m_tag = water`
Guards: `a_phase_wing`, `Q_phase_wing`
Action (enter): `a_phase_wing <= -0.05` **OR** `Q_phase_wing <= 0.30`
Clear (exit): `Q_phase_wing >= 0.70` **AND** `a_phase_wing >= +0.05` for `>= T_clear`
Notes: use soft hysteresis (`rho`, `k_side`) to avoid churn during taxi delays.

9.3 Rail "sun-kink" (track buckling)

Risk: hot compressive buckling
Inputs: rail temp; neutral temp `T_neutral`; linear lens with `DeltaT`
Guard: `e_T_rail = ( T_rail - T_neutral ) / DeltaT`
Action (enter): `e_T_rail >= +E_hot` for `>= T_hot_min`
Clear (exit): `e_T_rail <= +E_hot - E_hyst` for `>= T_clear`
Notes: calibrate `T_neutral` per track stress state; measure at mid-span, sun-facing.

9.4 Cold-chain pharmaceuticals

Risk: potency loss from warm excursion or freeze
Inputs: product logger; optional `T_m_tag = product`
Guards:
• **Warm:** `e_T` (linear lens around storage setpoint)
• **Freeze:** `a_phase_product` near `T_m`
Action (enter):
• **Warm:** `S-CDD = sum_t max( e_T(t) - e* , 0 ) > S_CDD_max`
• **Freeze:** `a_phase_product <= -Phi_freeze` for `> T_freeze_min`
Clear (exit): `e_T <= e*` **AND** `a_phase_product >= +Phi_clear` for `>= T_clear`
Notes: keep `manifest_id` on every record for audit; place probe at product core.

9.5 Food logistics (frozen vs tempered SKUs)

Risk: partial thaw or unintended freezing
Inputs: SKU-specific `T_m_tag` (ice cream, meat, produce)
Guard: `a_phase_sku`
Action (enter): `|a_phase_sku| > Phi_sku` for `>= T_sku_min`
Clear (exit): `|a_phase_sku| <= Phi_clear` for `>= T_clear`

Notes: separate policies for **frozen** (protect against warming, watch `a_phase_sku -> +1`) vs **tempered** (protect against re-freeze, watch `a_phase_sku -> -1`).

9.6 Data centers (hot-aisle excursions)

Risk: thermal runaway / throttling
Inputs: rack inlet temps
Guards: `e_T_inlet` and short-window volatility `V_T(10 min)`
Action (enter): `e_T_inlet >= +E_hot` **OR** `V_T >= V_star` for `>= T_hot_min`
Clear (exit): `e_T_inlet <= +E_hot - E_hyst` for `>= T_clear`
Notes: monitor worst-of inlets; tie actions to workload-shedding tiers.

9.7 Spaceflight cryogenic operations

Risk: propellant freezing or unsafe fills
Inputs: line/tank temperature; `T_m_tag = propellant`
Guards: `a_phase_prop`, optional cold-sensitive contrast via **beta** or **qlog** lens
• `e_T_beta = ( T_ref / T_K ) - 1` (cold emphasis)
• `e_T_qlog = ln( ( T_K/T_ref + alpha ) / ( 1 + alpha ) )` (near-zero safe; publish `alpha > 0`)
Action (enter): `a_phase_prop <= -Phi_freeze` **OR** `e_T_beta >= E_cold*` for `>= T_min`
Clear (exit): `a_phase_prop >= +Phi_clear` for `>= T_clear`
Notes: isolate by segment; prefer fiber sensors with low thermal lag; publish chosen lens and parameters.

9.8 Diesel fuel gelling (gensets, towers)

Risk: filter blockage at cloud point
Inputs: fuel line temp; `T_m_tag = cloud_point`
Guard: `a_phase_fuel`
Action (enter): `a_phase_fuel <= -0.10` for `>= 10 min` → pre-heat / switch blend
Clear (exit): `a_phase_fuel >= +0.10` for `>= 10 min`
Notes: adjust `T_m` by seasonal diesel grade; insulate exposed lines.

9.9 Maritime sea-spray icing

Risk: superstructure icing
Inputs: surface temp; `T_m_tag = brine`
Guards: `a_phase_surface`, `V_T(15 min)`

Action (enter): `a_phase_surface <= -Phi_freeze` **AND** `V_T >= V_star`
Clear (exit): `a_phase_surface >= +Phi_clear` for `>= T_clear`
Notes: combine with wind/sea state externally; temperature side stays symbolic.

9.10 Greenhouses and precision agriculture

Risk: frost at leaves/fruit
Inputs: canopy/leaf temps; `T_m_tag = plant_tissue`
Guard: `a_phase_leaf`
Action (enter): `a_phase_leaf <= -0.05` for `>= 10 min` → heaters/misting
Clear (exit): `a_phase_leaf >= +0.05` for `>= 10 min`
Notes: canopy sensors at coldest microclimates (near vents, edges).

9.11 District heating / heat-pump fleets

Risk: underheating in cold snaps
Inputs: outdoor `e_T_outdoor`, supply `e_T_supply`
Guard: `e_T_outdoor`, target band for `e_T_supply`
Action (enter): Stage N if `e_T_outdoor <= -E_cold` **AND** `e_T_supply < target_e`
Clear (exit): de-stage when `e_T_outdoor >= -E_cold + E_hyst` **OR** `e_T_supply >= target_e`
Notes: avoids unit retuning across cities; thresholds travel.

9.12 Pipeline freeze protection (heat tracing)

Risk: stagnant segment freezing
Inputs: coldest-segment temp; `T_m_tag = fluid`
Guard: `a_phase_fluid`
Action (enter): **Trace ON** while `a_phase_fluid <= 0.00`
Clear (exit): **Trace OFF** when `a_phase_fluid >= +0.10`
Notes: use redundancy; verify CTS load limits before automatic ON.

9.13 Server room / small facilities (quick policy)

Risk: overheating with limited telemetry
Inputs: single ambient sensor
Guard: `e_T_room`
Action (enter): `e_T_room >= +0.7` for `>= 10 min` → notify + open loop to HVAC
Clear (exit): `e_T_room <= +0.7 - 0.2` for `>= 10 min`
Notes: S1-only deployment; no phase dial needed.

9.14 Vaccine clinics (handoff windows)

Risk: warm exposure during staging
Inputs: tray probe
Guards: `e_T_tray`, `S-CDD`
Action (enter): `e_T_tray >= +0.5` for `>= 5 min` **OR** `S-CDD > 1.5`
Clear (exit): `e_T_tray <= +0.5 - 0.2` for `>= 5 min`
Notes: document thresholds on signage in symbol space.

9.15 Warehousing (mixed SKUs)

Risk: co-mingled goods with different pivots
Inputs: per-zone temps; per-SKU `T_m_tag`
Guards: `a_phase_zone`, SKU-specific bands
Action (enter): `a_phase_zone <= -Phi_freeze_sku` for `>= T_min_sku`
Clear (exit): `a_phase_zone >= +Phi_clear_sku` for `>= T_clear_sku`
Notes: route-sensitive SKUs away from cold zones during picks.

9.16 Battery cabinets & BESS (thermal runaway precursor) (optional)

Risk: precursor overheating / instability
Inputs: cell or module temps; choose lens per design (linear near setpoint, log for wide spans)
Guards: `e_T_module`, short-window volatility `V_T(5–10 min)`
Action (enter): `e_T_module >= +E_hot` **OR** `V_T >= V_star` for `>= T_hot_min` → throttle/isolator trip per SOP
Clear (exit): `e_T_module <= +E_hot - E_hyst` for `>= T_clear`
Notes: keep per-string manifests; pool `a_T` across redundant sensors if S3 is enabled.

9.17 Museums & archives (materials preservation) (optional)

Risk: thermal stress from swings
Inputs: ambient temps by gallery/case
Guards: rolling volatility `V_T(L)` and robust anomaly `Z_T`
Action (enter): `V_T >= V_star` **OR** `|Z_T| >= Z_star` for `>= T_min` → slow-control correction

Clear (exit): `V_T < V_star - V_hyst` **AND** `|Z_T| < Z_star` for `>= T_clear`
Notes: symbol space ensures portable triggers across buildings without retuning units.

Suggested default knobs (tunable)
`Phi_freeze = 0.10`, `Phi_clear = 0.10`, `E_hot = 0.8`, `E_cold = 0.8`, `E_hyst = 0.2`, `T_hot_min = 10 min`, `T_freeze_min = 20 min`, `T_clear = 20 min`. Publish your chosen values in policy docs.

Operational drills (recommended)
• **Tabletop.** Run through each entry with last month's telemetry; confirm enter/clear timing.
• **Failover.** Simulate sensor loss; verify `health.sensor_ok = false` triggers manual checks.
• **Hysteresis tuning.** Sweep `rho` and `k_side`; choose values that minimize flicker without late clears.

Human communications (ready phrases)
• "Heat Advisory triggered in symbol space: `e_T` exceeded `+E_hot` for `T_hot_min`."
• "Freeze Risk cleared: `a_phase` exceeded `+Phi_clear` for `T_clear`."
• "Audit trail: `manifest_id`, symbol thresholds, and dwell times attached to the event."

Bottom line
Each rule here is **unitless, portable, and auditable**. Keep machine logic in **symbol space**; reserve `°C/°F` for **human display only**.

---

10) Conclusion & Release Guide

SSMT turns temperature into a **single, unitless language** for analytics, safety, and ML. Fix a **lens** and **anchors**, emit **`e_T`** (and, when needed, **`a_phase`** or **`a_phase_fused`** and **`a_T`**), and carry a **small manifest**. The result is **zero unit drama**, **phase-aware safety**, and **portable thresholds** across cities, vendors, and devices without changing physics or downstream models.

10.1 For adopters (external): what to ship first

• **Start with S1 (Lite):** emit `e_T` and `manifest_id` (add health flags if available).
• **Add S2 when freezing matters:** include `a_phase` (or `a_phase_fused` for multiple pivots) and **soft hysteresis** (`rho`, `k_side`).
• **Add S3 only if needed:** include `a_T = tanh( c_T * e_T )` for bounded fleet dials or ML priors; use **pooling** if aggregating many sensors.
• **Keep logic symbolic:** rules use `e_T` and `a_phase`; **°C/°F are display-only**.
• **Kelvin floor:** after unit conversion, apply `T_K = max( T_K , eps_TK )` and publish `eps_TK` if non-default.

Typical payload (S1 to S3):

```
{ timestamp_utc, e_T, a_phase?, Q_phase?, a_T?, T_m_tag_list?, manifest_id,
health:{ range_ok, drift, sensor_ok }, oor? }
```

Typical policies (symbol space):

```
HEAT_ADVISORY : e_T >= +E_hot for >= T_hot_min
COLD_ADVISORY : e_T <= -E_cold for >= T_cold_min
FREEZE_RISK : a_phase <= -Phi_freeze for >= T_freeze_min
CLEAR_* : dwell-based clears with hysteresis bands
```

10.2 For maintainers (internal): implementation checklist

• **Tests (Tier S1 in CI):** unit invariance; **lens guards** for log/linear/beta/kBT and, if used, **hybrid** with `tau > 0` and **qlog** with `alpha > 0`; **clamps**; **phase symmetry**; **soft-hysteresis evolution**; **validity/saturation**; **pooling bounds**; **Kelvin floor** present.
• **Manifests:** freeze **one lens** and **anchors** per study; publish `T_valid_range_K`; attach `manifest_id` (and optionally `manifest_hash`) to every record; record `resolved_lens` once per run.
• **Policies:** define **symbol-only** alert thresholds and dwell times; document rationale.
• **Governance: no mid-run lens changes**; version manifests; keep new fields **optional**.
• **Optional evidence:** maintain a short empirical appendix showing **freeze-zone flicker reduction** and **threshold portability**.

10.3 Path to standardization (optional roadmap, copy-ready)

Phase 1 — Starter kit (OSS).
• Publish minimal refs (Python/R): `to_kelvin`, `encode_eT` for log|linear|beta|kBT|hybrid|qlog, `a_phase`, `clamp_a`, **pooling**; **JSON Schemas**.
• Include **golden vectors (Tier S1)**, tiny CLI, and a notebook that regenerates key tables.
• Document lenses and knobs: `tau` (hybrid), `alpha` (qlog), `eps_TK`, `eps_a`, `eps_w`.
• Ship a **Policy Pack** (`E_hot`, `E_cold`, `Phi_freeze`, `E_hyst`, dwell windows) and a **Lens Decision Note** template.

• **Accessibility & privacy starters:** ASCII-only formulas, alt-text guidance, **invertibility note** with symbol precision policy.

Phase 2 — Pilot deployments (3 flavors).
• **Weather API (S1):** public **`e_T`** stream; symbol-only advisories; manifest publishes **`T_ref`** policy and **`eps_TK`**.
• **Cold-chain vendor (S2):** **`a_phase`** + **soft hysteresis**; audit-ready manifests; publish **`T_m`** pivots.
• **Data center (S3):** bounded **`a_T`** + **pooling**; dashboard dials; publish **`c_T`** and pooling rule.
• **Optional cryo/space pilots: beta** or **qlog** where applicable; document **`tau/alpha`**.

Phase 3 — Evidence & conformance.
• Run **Tier S1 in CI** for all pilots; store **golden outputs** and **manifest hashes**.
• Report **KPIs (symbol space):** unit-incident reduction, freeze flicker reduction, **portability spread** across sites for fixed **`E_hot`** and **`E_cold`**.
• Record **failure injections** and outcomes; capture **privacy posture** and **accessibility checks**.

Phase 4 — Community & registry.
• Launch a **Challenge Set:** public datasets + manifests; contributors submit **symbol-only** results.
• Stand up an **SSMT Registry:** manifests with hashes, declared lenses/knobs, and domain tags.
• Maintain a **public ledger** of Lens Decision Notes and golden vectors per entry.

Phase 5 — External alignment.
• Present pilots and KPIs to standards bodies for **data representation profiles**.
• Reserve **lens names** and **field semantics**.
• Keep changes **backward-compatible:** new fields optional; deprecations with **≥ 12-month** windows.

Governance checkpoints.
• **Quarterly spec review.**
• **Accessibility & privacy audit.**
• **Versioning discipline:** manifests pinned, no mid-run lens changes, **manifest hashes** published.

Adoption one-liners.
• "**One to-K step, one symbol stream, thresholds that travel.**"
• "**Safer near pivots with symmetric bands.**"
• "**Interop by construction** across vendors, cities, and units."

10.4 Release checklist (copy-ready)

• Tag **document spec version** `v2.3` and **schema version** `"1.1"`.
• Publish example **S1/S2/S3 manifests** and **golden test vectors**.
• Ship a minimal **reference harness:** `to_kelvin`, `encode_eT`, `a_phase`, **clamps**, **pooling**.

• Run **CI** on target hardware/pipeline; archive results and **`manifest_hash`**; record **`resolved_lens`**.
• Announce **deprecation timelines** and **migration notes**.
• Provide a **Policy Pack** with default knobs: **`E_hot`**, **`E_cold`**, **`Phi_freeze`**, **`E_hyst`**, **`T_*`**, include **`eps_TK`** if non-default.

10.5 Changelog template (copy-paste)

SSMT v2.3 — YYYY-MM-DD
• **Spec:** non-breaking update; four focused deltas.
• **Schema:** version `"1.1"` unchanged.
• **Addition 1 — Smooth Hybrid lens (continuity-safe):**

```
e_T = s * ln( T_K / T_ref ) + ( 1 - s ) * ( T_K - T_ref ) / DeltaT,
s = 1 / ( 1 + exp( -( abs( T_K - T_ref ) - tau ) / width ) ).
```

• **Addition 2 — Normalized multi-pivot fusion:**

```
z_i = c_m_i * ( T_K - T_m_i ) / DeltaT_m_i;
W = max( sum_i w_i , eps_w );z_bar = ( 1 / W ) * sum_i( w_i * z_i );
a_phase_fused = tanh( z_bar ).
```

• **Addition 3 — Adaptive hysteresis (rho by noise):**

```
sigma_T = std( T_K[t-W+1:t] );sigma_norm = sigma_T / DeltaT_m;
rho = clip( 1 - exp( - sigma_norm / sigma0 ) , rho_min , rho_max );
Q_phase = rho * Q_{t-1} + ( 1 - rho ) * clip( p_side , 0 , 1 ).
```

• **Addition 4 — Privacy modes & starter manifests:**
Addendum J (P0 raw, P1 quantized, P2 DP-lite; anchors policy) and Addendum K (M1/M2/M3); Section 2 cross-refs for optional privacy and gate blocks.

10.6 Version matrix (for clarity)

• **Document spec:** `vX.Y`
• **Manifest/schema:** `"1.1"`
• **Compliance levels:** `S1` / `S2` / `S3`

Bottom line
Start simple, stay symbolic, scale only as needed. This keeps deployments easy for domain users while giving maintainers the **safeguards** and **tests** needed to keep behavior stable over time.

Addendum A — SSMT-Lite (Quick Start)

Minimal adoption path. Encode once, use everywhere. Lite emits only the unitless contrast **`e_T`** and, where freezing matters, the phase dial **`a_phase`** with **soft hysteresis**. **Default lens = log.** No **`a_T`** in Lite.

A.1 When to use Lite

• Weather and dashboards • City analytics • Audits • Simple IoT and firmware shims

---

A.2 Encoding (Lite)

1. **Standardize to Kelvin**

```
to_kelvin(T_raw, unit):
  u = lower(unit)                  # accept "C","c","celsius", etc.
  if u in {"k","kelvin"}     : return T_raw
  if u in {"c","celsius"}    : return T_raw + 273.15
  if u in {"f","fahrenheit"} : return (T_raw - 32) * 5/9 + 273.15
  raise "unknown unit"
```

• **Precision hint:** keep ≥ **0.01 K** internally; round only for human display.
• **Kelvin floor (numeric safety):** after conversion, apply **`T_K = max( T_K , eps_TK )`**. Default **`eps_TK = 1e-6`** (publish if non-default).

2. **Choose lens (fixed per study; default = log)**
   All maps are unitless and monotone in **`T_K`**.
   • **Log (recommended):** `e_T = ln( T_K / T_ref )`
   • **Linear (narrow bands):** `e_T = ( T_K - T_ref ) / DeltaT`
   • **Beta (cold-sensitive):** `e_T = ( T_ref / T_K ) - 1`
   • **kBT (energy-linked):** `e_T = ( R*T_K )/E_unit - ( R*T_ref )/E_unit`
   • **(Optional, advanced) Hybrid (piecewise):** linear near baseline, log for larger departures; publish **`tau > 0`**.
   • **(Optional, near-zero safety) qlog:** `e_T = ln( ( T_K/T_ref + alpha ) / ( 1 + alpha ) )`; publish **`alpha > 0`**.
   Domain guards: log/beta require **`T_K > 0`** and **`T_ref > 0`**; linear requires **`DeltaT > 0`**; kBT requires **`E_unit > 0`**; hybrid requires **`tau > 0`**; qlog requires **`T_ref > 0`** and **`alpha > 0`**.
3. **Phase proximity (optional, near pivots like freezing)**
   `d_m = ( T_K - T_m ) / DeltaT_m`
   `a_phase = tanh( c_m * d_m ) # in (-1, +1)`
   `a_phase = clamp_a( a_phase , eps_a ) # |a_phase| <= 1 - eps_a`
4. **Soft hysteresis (reduce flicker near `T_m`)**
   `p_side = 0.5 * ( 1 + tanh( k_side * ( T_K - T_m ) ) )`
   `Q_phase = rho * Q_prev + (1 - rho) * clip( p_side , 0 , 1 )`

Knobs: `c_m > 0`, `DeltaT_m > 0`, `k_side > 0`, `rho in (0,1)`, `eps_a ~ 1e-6`.
Note: `clamp_a(x, eps) = min( max(x, -1+eps), 1-eps ).`

A.3 Emitted record (Lite)

Unitless, portable, self-describing:

```
{
  "timestamp_utc": "...",
  "e_T": <number>,
  "a_phase": <number, optional>,
  "Q_phase": <number, optional>,
  "T_m_tag": "<string, optional>",
  "manifest_id": "<string>",
  "health": { "range_ok": true/false, "drift": true/false, "sensor_ok":
true/false },
  "oor": false | "below_min" | "above_max"
}
```

A.4 Minimal manifest (Lite)

Publish once; do not change mid-run.

```
SSMT_manifest_v1:
  version: "1.1"
  compliance_level: "S1"
  lens: "log"
  anchors:
    T_ref: 298.15
    DeltaT: null
    E_unit: null
  guards:
    clamp_a_eps: 1e-6
    T_valid_range_K: [180.0, 500.0]
    eps_TK: null                  # optional; set if overriding default 1e-
6
  phase_block:
    T_m_list: [ { tag:"water", T_m:273.15, DeltaT_m:2.0, c_m:1.2 } ]   #
optional
    rho: 0.90
    k_side: 2.0
  outputs:
    emit_e_T: true
    emit_a_phase: false           # set true (and consider S2) only if
freezing matters
    emit_Q_phase: false
  notes: "Lite emits e_T; add a_phase only if freezing matters"
```

A.5 Worked examples (Lite)

• Unit invariance (log lens, `T_ref = 298.15 K`)
`35 C = 95 F = 308.15 K → e_T = ln(308.15 / 298.15) = 0.03298996`

• **Water freezing dial (`T_m = 273.15 K`, `DeltaT_m = 2.0`, `c_m = 1.2`)**
`T = 271.15 K → a_phase = tanh(1.2 * -1.0) = -0.83365461`
`T = 275.15 K → a_phase = tanh(1.2 * +1.0) = +0.83365461`
• **Soft hysteresis sketch (`rho = 0.90`, `k_side = 2.0`)**
Short dips below `T_m` no longer thrash alerts; clears require sustained warmth.

A.6 Symbol-only alert grammar (Lite)

```
HEAT_ADVISORY : e_T >= +E_hot for >= T_hot_min
COLD_ADVISORY : e_T <= -E_cold for >= T_cold_min
FREEZE_RISK : a_phase <= -Phi_freeze for >= T_freeze_min

CLEAR_HEAT when e_T <= +E_hot - E_hyst for >= T_clear
CLEAR_COLD when e_T >= -E_cold + E_hyst for >= T_clear
CLEAR_FREEZE when a_phase >= +Phi_clear for >= T_clear
```

A.7 Upgrade path

Start in **Lite**. If you need **bounded pooling**, **ML priors**, or **unified fleet dials**, add the full alignment channel `a_T = tanh(c_T * e_T)` later (no changes to existing `e_T` streams).

A.8 Anti over-engineering note

Default to **S1**. Add `a_phase` only for freeze/phase problems. Keep everything else **off** until the use case proves it out.

Appendix B — JSON Schemas (concise, copy-ready)

Schemas use **JSON Schema draft-07**. They're strict (`additionalProperties: false`) to keep streams stable and auditable. Field names and semantics match the spec.

B1) Manifest schema (v1.1)

```
{
  "$schema": "https://json-schema.org/draft-07/schema",
  "$id": "https://ssmt.shunyaya.blog/schema/manifest/1.1/schema.json",
  "title": "SSMT Manifest v1.1",
  "type": "object",
  "additionalProperties": false,
```

```
  "required": ["version", "compliance_level", "lens", "anchors", "guards",
"outputs"],
  "properties": {
    "version": { "type": "string", "const": "1.1" },
    "compliance_level": { "type": "string", "enum": ["S1", "S2", "S3"] },

    "lens": { "type": "string", "enum": ["log", "linear", "beta", "kBT",
"hybrid", "qlog"] },

    "anchors": {
      "type": "object",
      "additionalProperties": false,
      "required": ["T_ref"],
      "properties": {
        "T_ref":  { "type": "number", "exclusiveMinimum": 0 },
        "DeltaT": { "type": ["number", "null"], "exclusiveMinimum": 0 },
        "E_unit": { "type": ["number", "null"], "exclusiveMinimum": 0 },
        "c_T":    { "type": ["number", "null"], "exclusiveMinimum": 0 },
        "tau":    { "type": ["number", "null"], "exclusiveMinimum": 0 },
        "alpha":  { "type": ["number", "null"], "exclusiveMinimum": 0 }
      }
    },

    "guards": {
      "type": "object",
      "additionalProperties": false,
      "required": ["T_valid_range_K"],
      "properties": {
        "T_valid_range_K": {
          "type": "array",
          "minItems": 2, "maxItems": 2,
          "items": { "type": "number" }
        },
        "clamp_a_eps": {
          "type": ["number", "null"],
          "minimum": 0.0, "maximum": 0.001
        },
        "eps_w": {
          "type": ["number", "null"],
          "exclusiveMinimum": 0.0, "maximum": 1.0
        },
        "eps_TK": {
          "type": ["number", "null"],
          "exclusiveMinimum": 0.0
        }
      }
    },

    "phase_block": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "T_m_list": {
          "type": "array",
          "items": {
            "type": "object",
            "additionalProperties": false,
            "required": ["tag", "T_m", "DeltaT_m", "c_m"],
            "properties": {
              "tag":       { "type": "string", "minLength": 1 },
              "T_m":       { "type": "number" },
```

```
              "DeltaT_m":  { "type": "number", "exclusiveMinimum": 0 },
              "c_m":       { "type": "number", "exclusiveMinimum": 0 }
            }
          }
        },
        "rho":    { "type": ["number", "null"], "exclusiveMinimum": 0,
"exclusiveMaximum": 1 },
        "k_side": { "type": ["number", "null"], "exclusiveMinimum": 0 }
      }
    },

    "pooling": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "weight_rule": { "type": "string", "enum": ["equal", "inv_var",
"health"] }
      }
    },

    "health_flags": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "enable_range_ok": { "type": "boolean" },
        "enable_sensor_ok": { "type": "boolean" },
        "enable_drift": { "type": "boolean" }
      }
    },

    "validation": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "dataset_ref": { "type": "string" },
        "test_vectors_ref": { "type": "string" },
        "metrics": { "type": "object", "additionalProperties": true }
      }
    },

    "auto_lens_policy": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "enabled": { "type": "boolean" },
        "rules": {
          "type": "array",
          "items": {
            "type": "object",
            "additionalProperties": true
          }
        }
      }
    },

    "outputs": {
      "type": "object",
      "additionalProperties": false,
      "required": ["emit_e_T"],
      "properties": {
        "emit_e_T":     { "type": "boolean", "const": true },
```

```
        "emit_a_phase": { "type": ["boolean", "null"] },
        "emit_Q_phase": { "type": ["boolean", "null"] },
        "emit_a_T":     { "type": ["boolean", "null"] }
      }
    },

    "notes": { "type": "string" }
  }
}
```

Notes (validation intent, non-enforceable cross-field rules):
• If `lens=="linear"` then `anchors.DeltaT` should be non-null and > 0.
• If `lens=="kBT"` then `anchors.E_unit` should be non-null and > 0.
• If `lens=="hybrid"` then `anchors.tau` should be non-null and > 0.
• If `lens=="qlog"` then `anchors.alpha` should be non-null and > 0 and `anchors.T_ref > 0`.
• If `outputs.emit_a_T==true` then `anchors.c_T` should be non-null and > 0.
• `T_valid_range_K = [T_min, T_max]` with `0 < T_min < T_max` (enforce in CI).
• If `guards.eps_TK` is omitted, default to `1e-6` at runtime.

B2) Payload schema — Full SSMT (S1/S2/S3)

```
{
  "$schema": "https://json-schema.org/draft-07/schema",
  "$id": "https://ssmt.shunyaya.blog/schema/payload/full/1.0/schema.json",
  "title": "SSMT Payload (Full)",
  "type": "object",
  "additionalProperties": false,
  "required": ["timestamp_utc", "e_T", "manifest_id", "health"],
  "properties": {
    "timestamp_utc": { "type": "string", "format": "date-time" },
    "e_T":           { "type": "number" },

    "a_T":           { "type": "number" },
    "a_phase":       { "type": "number" },
    "a_phase_fused": { "type": "number" },
    "Q_phase":       { "type": "number" },

    "T_m_tag":       { "type": "string" },
    "T_m_tag_list":  {
      "type": "array",
      "items": { "type": "string" }
    },

    "manifest_id":   { "type": "string", "minLength": 1 },

    "health": {
      "type": "object",
      "additionalProperties": false,
      "required": ["range_ok"],
      "properties": {
        "range_ok":  { "type": "boolean" },
        "sensor_ok": { "type": "boolean" },
        "drift":     { "type": "boolean" }
      }
```

```
    },

    "oor": { "enum": [false, "below_min", "above_max"] }
  }
}
```

Notes (validation intent): Prefer exactly one of `a_phase` or `a_phase_fused`. If `a_phase_fused` is present, `T_m_tag_list` SHOULD be present and non-empty.

B3) Payload schema — Lite (S1)

```
{
  "$schema": "https://json-schema.org/draft-07/schema",
  "$id": "https://ssmt.shunyaya.blog/schema/payload/lite/1.0/schema.json",
  "title": "SSMT Payload (Lite, S1)",
  "type": "object",
  "additionalProperties": false,
  "required": ["timestamp_utc", "e_T", "manifest_id", "health"],
  "properties": {
    "timestamp_utc": { "type": "string", "format": "date-time" },
    "e_T":           { "type": "number" },

    "a_phase":       { "type": "number" },
    "a_phase_fused": { "type": "number" },
    "Q_phase":       { "type": "number" },

    "T_m_tag":       { "type": "string" },
    "T_m_tag_list":  {
      "type": "array",
      "items": { "type": "string" }
    },

    "manifest_id":   { "type": "string", "minLength": 1 },

    "health": {
      "type": "object",
      "additionalProperties": false,
      "required": ["range_ok"],
      "properties": {
        "range_ok":  { "type": "boolean" },
        "sensor_ok": { "type": "boolean" },
        "drift":     { "type": "boolean" }
      }
    },

    "oor": { "enum": [false, "below_min", "above_max"] }
  }
}
```

Notes (validation intent): Lite typically omits `a_phase`; if included, treat as S2 behavior. Prefer exactly one of `a_phase` or `a_phase_fused`.

B4) Examples (copy-paste)

B4.1 Common temperatures micro-table (`e_T`, `a_T`, `a_phase`)

Parameters: `T_ref = 298.15 K` (lens = log), `c_T = 1.0`; `T_m = 273.15 K`, `DeltaT_m = 10.0 K`, `c_m = 1.0`.
Columns: `Temp_C, T_K, e_T, a_T, a_phase`

Temp_C	T_K	e_T	a_T	a_phase
-10	263.150000	-0.12487250	-0.12422748	-0.76159416
0	273.150000	-0.08757562	-0.08735242	0.00000000
25	298.150000	0.00000000	0.00000000	0.98661430
37	310.150000	0.03945934	0.03943887	0.99877824
100	373.150000	0.22438377	0.22069233	1.00000000

• Note: `e_T = ln( T_K / T_ref );a_T = tanh( c_T * e_T );a_phase = tanh( ( T_K - T_m ) / DeltaT_m ).`
• CSV artifact: `common_temps.csv` (same columns; append as needed).

B4.2 Full S3 with phase + alignment

```
{
  "timestamp_utc": "2025-09-24T12:05:00Z",
  "e_T": 0.92,
  "a_T": 0.56,
  "a_phase": -0.12,
  "Q_phase": 0.31,
  "T_m_tag": "water",
  "manifest_id": "SSMT-FLEET-2025-09-007",
  "health": { "range_ok": true, "sensor_ok": true, "drift": false },
  "oor": false
}
```

B4.3 Multi-pivot (fused phase) example (optional)

```
{
  "timestamp_utc": "2025-09-24T12:10:00Z",
  "e_T": -0.08,
  "a_phase_fused": -0.21,
  "T_m_tag_list": ["water","brine"],
  "manifest_id": "SSMT-ROADS-2025-09-011",
  "health": { "range_ok": true, "sensor_ok": true, "drift": false },
  "oor": false
}
```

B5) Validator tips (practical)

• Enforce cross-field guards in CI (lens↔anchors, `T_min < T_max`, etc.).
• If `lens=="linear"`, require `DeltaT > 0`; if `lens=="kBT"`, require `E_unit > 0`; if `lens=="hybrid"`, require `tau > 0`; if `lens=="qlog"`, require `alpha > 0` and `T_ref > 0`.
• Treat `timestamp_utc` as UTC (Z suffix); reject local timestamps.

• Log both `manifest_id` and a `manifest_hash` to pin behavior over time.
• For S1, **reject payloads carrying `a_T`** (keeps tiers clean).
• Prefer exactly one of `a_phase` or `a_phase_fused`; if `a_phase_fused` is present, ensure `T_m_tag_list` is non-empty.
• If `eps_TK` is published, verify `T_K = max(T_K, eps_TK)` was applied upstream (spot-check via golden vectors).
• Detect degC/degF flips: monitor `e_T` for sudden affine-like discontinuities consistent with a raw-unit swap. Rule of thumb: flag when `|Δe_T|` exceeds a manifest-bounded jump `J_flip` over one step while derived `T_K(t)` would be more consistent under the alternate unit mapping. Validate by replay on golden vectors with synthetic `C↔F` swaps.
• Near-pivot flicker audit: define a band `B_K` around `T_m` (e.g., ±2 K). Count sign changes of `a_phase` or threshold crossings of `Q_phase` within the band per hour. Require `flicker_rate ≤ F_max` and `dwell_time_risk ≥ D_min`; if violated, tighten `k_side` or increase `rho` (or enable adaptive `rho`).

Appendix C — Empirical Snapshot (Tier 2, optional)

Small, public-style benchmarks demonstrating SSMT's benefits without changing the spec. All computations are in **symbol space**.

Data provenance (applies to C1, C2, C9). Results are computed from the public "Hourly Weather Data in Ireland (from 24 stations)" dataset (Met Éireann; CC BY 4.0). **C4** uses the same dataset with a synthetic °C↔°F flip injected. **C5** and **C6** are runnable specifications (no dataset required). No derived CSV artifacts are distributed; results are reproducible from the public dataset referenced at the end of this appendix.

Methods (summary).
• Lens: `log`. Anchors: `T_ref = 298.15 K`.
• Phase dial: `T_m = 273.15 K`, `DeltaT_m = 2.0 K`, `c_m = 1.2`, `eps_a = 1e-6`.
• Soft hysteresis: `rho = 0.90`, `k_side = 2.0`.
• Freeze rule (symbolic): `a_phase <= -Phi_freeze`, with `Phi_freeze = 0.10`.
• Hot threshold (symbolic): `e_T >= E_hot`, with `E_hot = 0.8`.
• Flicker metric: count of state toggles at the dataset cadence (dwell approximated by time step).

Baselines to compute (for fair comparisons).
• Raw-freeze baseline (no hysteresis): flag if `T_K <= T_m`.
• Raw-hot baseline (log-lens comparable): `T_hot = T_ref * exp(E_hot)`; flag if `T_K >= T_hot`.
• Raw-anomaly baseline (for C3): `A_K(t) = T_K(t) - median_{30d, same hour}( T_K )`.

MQTT streaming example (copy-paste).
Topic (example): `ssmt/{station_id}/s1`
Payload (JSON):

```
{
  "timestamp_utc": "2025-09-24T12:00:00Z",
  "e_T": 0.0331,
  "a_phase": -0.12,
  "Q_phase": 0.31,
  "manifest_id": "SSMT-CITY-2025-09-001",
  "health": { "range_ok": true, "sensor_ok": true, "drift": false }
}
```

C1. Freeze-zone alert stability (flicker reduction).

Baseline vs SSMT.
• Baseline: raw-freeze rule (`T_K <= T_m`), no hysteresis.
• SSMT: symbolic `a_phase` with soft hysteresis (`Q_phase`) and threshold `Phi_freeze`.

Result (this slice).
Across 5 stations, total flicker fell from **134 (baseline)** to **120 (SSMT)**, an aggregate reduction of **10.45%**. Mean flicker dropped from **26.80** to **24.00**; median from **32.00** to **30.00**. Improvement was observed in **4 of 5 stations (80.0%)**.

C2. Threshold portability across stations.

Baseline vs SSMT.
• Baseline: raw-hot rule with `T_hot = T_ref * exp(E_hot).`
• SSMT: `e_T >= E_hot` (single global threshold).

Result (this slice).
Fraction of time flagged "hot" showed a spread of **0.0000 (symbolic)** vs **0.0000 (raw)** here. Interpretation: no hot exceedances in this window; to show non-trivial portability, use a warmer period or lower `E_hot` (e.g., `E_hot = 0.6`).

C3. Symbolic anomaly ROC (optional).

Definitions.
• SSMT: `A_T(t) = e_T(t) - median_{30d, same hour}( e_T ).`
• Baseline (raw): `A_K(t) = T_K(t) - median_{30d, same hour}( T_K ).`
Decision: positive if `A_*(t) >= tau` (sweep `tau` on a grid).

Report.
ROC (TPR vs FPR) and AUROC per station; compare spread across stations to demonstrate portability.

Status in this snapshot: method provided; curves/values may be added later without changing the spec.

C4. Failure-case injection (unit mislabelling).

A synthetic **C->F** flip injected into a continuous window produced a median step change in `e_T` of **0.001792**; drift and sensor plausibility flags were asserted (`sensor_ok=false`, `drift=true`). Decision rules remained stable in symbol space.

C5) Near-pivot flicker audit (runnable spec).

Objective. Quantify and suppress alert "chatter" when temperature hovers near a material pivot `T_m`.

Inputs (declare in manifest or test block).
• Stream: `{(t, T_K[t])}`, sampling interval `dt` (seconds).
• Pivot: `T_m (K)`, `DeltaT_m > 0`, `c_m > 0`.
• Hysteresis: `k_side > 0 (1/K)`, `rho in (0,1)` or adaptive-rho recipe.
• Band width: `B_K > 0 (K)`, e.g., `B_K = 2.0`.
• Risk threshold: `Q_thr in (0,1)`, e.g., `Q_thr = 0.5`.
• Window for rates: `W_eval` (minutes).
• Limits: `F_max` (max flickers/hour), `D_min` (min dwell in risk per hour).

Derived dials.

```
d_m = ( T_K - T_m ) / DeltaT_m
a_phase = tanh( c_m * d_m )
p_side = 0.5 * ( 1 + tanh( k_side * ( T_K - T_m ) ) )
Q_phase[t] = rho * Q_phase[t-1] + ( 1 - rho ) * clip( p_side , 0 , 1 )
```

Procedure.

1. Define near-pivot band: `in_band[t] = 1{ |T_K[t] - T_m| <= B_K }`.
2. Count `a_phase` sign flips within band:
   `flips_phase = # { t : in_band[t]=1 and sign(a_phase[t]) ≠ sign(a_phase[t-1]) }`.
3. Count `Q_phase` threshold crossings within band:
   `flips_Q = # { t : in_band[t]=1 and 1{Q_phase[t] >= Q_thr} ≠ 1{Q_phase[t-1] >= Q_thr} }`.
4. Rate per hour over each evaluation window:
   `flicker_rate = 60 * ( flips_phase + flips_Q ) / W_eval_minutes.`
5. Risk dwell within band:
   `dwell_time_risk = sum_t ( dt * 1{ in_band[t]=1 and Q_phase[t] >= Q_thr } ).`

Metrics to report.
• `flicker_rate` (per hour)
• `dwell_time_risk` (seconds per hour)

• band_coverage = (time in band) / (window time)
• suggestions = { tighten k_side, increase rho or enable adaptive-rho, widen B_K cautiously }

Pass / action.
• Pass if flicker_rate <= F_max and dwell_time_risk >= D_min.
• Else: auto-recommend parameter nudges:
– If flicker_rate high: increase rho by +0.05 (cap at 0.95) or use adaptive-rho; increase k_side by +0.1 1/K.
– If dwell_time_risk too low (over-sticky): decrease rho by -0.05 (floor 0.5) or reduce k_side.

CSV schema (example, one row per window).

```
window_start_iso,window_end_iso,B_K,Q_thr,k_side,rho,flicker_rate_per_hr,dwell_time_risk_s,band_coverage,suggest_k_side,suggest_rho
2025-09-24T12:00:00Z,2025-09-24T13:00:00Z,2.0,0.5,0.5,0.80,3.0,1200,0.35,0.6,0.85
```

Pseudocode (reference).

```
for each eval window:
  flips = 0
  dwell = 0
  for t from 1..N_win:
    in_band = (abs(T_K[t]-T_m) <= B_K)
    if in_band:
      if sign(a_phase[t]) != sign(a_phase[t-1]): flips += 1
      if (Q_phase[t] >= Q_thr) != (Q_phase[t-1] >= Q_thr): flips += 1
      if Q_phase[t] >= Q_thr: dwell += dt
  flicker_rate = 3600 * flips / window_seconds
  emit row with metrics and suggestions
```

Baseline comparison (report alongside SSMT).
• Baseline flips = sign(T_K - T_m) toggles within band (no hysteresis).
• Compare flicker_rate_baseline vs flicker_rate_SSMT and dwell_time_risk_baseline vs dwell_time_risk_SSMT.

C6) °C/°F flip safety (runnable spec).

Objective. Detect accidental upstream unit flips (C↔F) that can corrupt continuity before the to-K step or mis-encode e_T.

Two operating modes.
• **Mode A (preferred):** raw sensor stream provides T_raw and unit flag unit_raw ∈ {"C","F","K"}.
• **Mode B (e_T-only):** only e_T is stored; reconstruct T_K via declared lens to test plausibility.

Reference transforms.

```
T_C = T_K - 273.15
T_F = (9/5) * T_C + 32
T_C_fromF = (5/9) * ( T_F - 32 )
T_K_fromC = T_C + 273.15
```

Invert e_T to T_K (by lens).

- log: `T_K = T_ref * exp( e_T )`
- linear: `T_K = T_ref + DeltaT * e_T`
- beta: `T_K = T_ref / ( 1 + e_T )`
- kBT: `T_K = T_ref + ( E_unit / R ) * e_T`
- qlog: `T_K = T_ref * ( ( 1 + alpha ) * exp( e_T ) - alpha )`

Flip-detection heuristic (windowed).

Let `J_obs[t] = | e_T[t] - e_T[t-1] |`.
Let `S_loc[t] = sqrt( Var_{t-L+1:t}( e_T ) ) + eps`, `eps > 0`.
Define `jump_ratio[t] = J_obs[t] / S_loc[t]`.
Candidate flip at `t*` if `jump_ratio[t*] >= J_flip` (e.g., `J_flip = 6`).

Confirmation test (reduce-to-smoother under alternate mapping).
Mode A (with T_raw).

1. If `unit_raw[t*] == "C"`, build an alternate series where samples `t*..t*+H` are reinterpreted as Fahrenheit then converted to Kelvin:
   `T_K_alt = T_K_fromC( T_C_fromF( T_F_alt ) )`, where `T_F_alt = (9/5) * T_raw + 32`.
   (Analogous if `unit_raw[t] == "F"`: reinterpret as Celsius.)
2. Recompute `e_T_alt` at `t*..t*+H` using the declared lens.
3. Local variance ratio: `R_alt = Var( e_T_alt[t*-h..t*+H] ) / Var( e_T[t*-h..t*+H] )`.
4. Confirm flip if `R_alt <= r_thr` (e.g., `r_thr = 0.5`) and `jump_ratio[t*] >= J_flip`.

Mode B (e_T-only).

1. Invert to `T_K` via lens; obtain `T_K[t]`.
2. Synthesize an alternate remap at `t*..t*+H` as if those samples were encoded after a `C↔F` swap before to-K:
   `T_C_alt = T_C_fromF( T_F = (9/5) * ( T_K[t] - 273.15 ) + 32 )`
   `T_K_alt = T_C_alt + 273.15`
3. Recompute `e_T_alt` from `T_K_alt` using the declared lens.
4. Same variance-ratio test as Mode A.

Decision and action.

- If confirmed: flag **FLIP** at `t*`, annotate window `[t*-h, t*+H]`, and recommend quarantining those samples or reprocessing with correct unit.
- Else: flag as outlier-jump; leave to general QC.

CSV schema (events).

```
t_star_iso,mode,J_obs,jump_ratio,J_flip,R_alt,r_thr,decision,notes
2025-09-24T12:05:00Z,A,0.41,7.3,6.0,0.34,0.50,CONFIRMED_FLIP,"C->F swap
suspected; recommend reprocess"
```

Pseudocode (reference).

```
for t from L+1..N:
  J_obs = abs(e_T[t]-e_T[t-1])
  S_loc = sqrt(var(e_T[t-L+1..t])) + eps
  if J_obs / S_loc >= J_flip:
    build e_T_alt via Mode A or B over [t-h..t+H]
    R_alt = var(e_T_alt_window) / var(e_T_window)
    decision = (R_alt <= r_thr) ? "CONFIRMED_FLIP" : "NO_FLIP"
    emit event row
```

Parameter starting points.
`L = 60` samples; `eps = 1e-6`; `J_flip = 6.0`; `h = 5` samples; `H = 10` samples; `r_thr = 0.5`.

Notes.
• Guards: ensure `T_K > 0`, `T_ref > 0`, `DeltaT > 0`, `E_unit > 0`, `alpha > 0` as required by the lens.
• If raw sensor units are guaranteed correct and immutable, **Mode B** alone suffices to catch residual pathologies (e.g., sensor resets).

C7) Reproducibility notes.

• Kelvin floor and lens: `T_K = max( T_K , eps_TK );` `e_T = ln( T_K / T_ref ).` Declare `T_ref` and `eps_TK`.
• Phase dial (single pivot): `a_phase = tanh( c_m * ( T_K - T_m ) / DeltaT_m );` clamp to `(-1 + eps_a , +1 - eps_a).`
• Soft hysteresis: `p_side = 0.5 * ( 1 + tanh( k_side * ( T_K - T_m ) ) );` `Q_phase_t = rho * Q_phase_{t-1} + ( 1 - rho ) * clip( p_side , 0 , 1 ).`
• Event counting: count state toggles at the data cadence; report window length and `dt`.
• Cross-spec check (SSM-Chem): compute `RSI_env = g_t * RSI` using identical windows; report pre/post metrics and include `manifest_id` with dataset refs for replay.
• Seeds/knobs: publish `c_m, DeltaT_m, k_side, rho, eps_a, eps_TK` (fixed over the run).
• Time: timestamps MUST be ISO-8601 UTC (`Z`).
• Optional: if using Smooth Hybrid, declare `tau` and `width`; if using privacy P1/P2, declare step or `( b , eps ).`

C8) Phase-aware env-gate example (SSM-Chem coupling).

Purpose. Show how temperature gating modulates a chemistry index without unit conversions.

Setup.
• From SSMT: `{ e_T, a_T?, a_phase, Q_phase }`.
• Lane (pick one and freeze per study):
`S_t = clip( w1 * abs(a_T) + w2 * Q_phase + w3 * abs(a_phase) , 0 , 1 )`
(default `w1=0.5, w2=0.5, w3=0.0`)
• Volatility and gate (window `L >= 2`):
`Z_t = log( 1 + Var_{t-L+1:t}( S_tau ) )`
`A_t = 1 / ( 1 + Z_t )`
`Delta_t = abs( Z_t - A_t )`
`Q_t = rho_g * Q_{t-1} + ( 1 - rho_g ) * clip( A_t - Z_t , 0 , 1 )` // `0 < rho_g < 1`
`g_t = ( 1 / ( 1 + Z_t + kappa * Delta_t ) ) * ( 1 - exp( -mu * Q_t ) )` // `kappa, mu >= 0`

Phase-aware chemistry gate (example).
• Phase mask `(0..1)`: `phi_phase = clip( ( a_phase + 1 ) / 2 , 0 , 1 )`
• Gated chemistry index: `RSI_env = g_t * RSI * phi_phase`

Tiny numeric illustration (one record).
Given `RSI = 0.60`, `g_t = 0.70`, `a_phase = -0.12` ⇒ `phi_phase = ( -0.12 + 1 ) / 2 = 0.44`
Then `RSI_env = 0.70 * 0.60 * 0.44 = 0.184`

Payload note (optional fields).
Emit `g_t` and `RSI_env` in experiment logs (not required for SSMT conformance).

C9) Warm-season hot exceedance portability (real dataset).

Objective. Show that a single symbolic hot threshold (`e_T`) yields identical decisions to an absolute-Kelvin threshold across heterogeneous stations, enabling portable alert rules.

Setup (fixed for this run).
• Lens: `log`. Anchors: `T_ref = 298.15 K`.
• Threshold in symbol space: `E_hot = 0.0166`.
• Mapping to Kelvin: `T_hot = T_ref * exp(E_hot) ≈ 303.140597 K (≈ 29.9906 C)`.
• Period: full available period in the aggregated hourly dataset (no month filter).
• Guards: `T_K = max( T_K , eps_TK )` with `eps_TK = 1e-6`.
• Encoding: `e_T = ln( T_K / T_ref )`.

Decision rules (evaluated per station).
• Symbolic: flag "hot" if `e_T >= E_hot`.
• Absolute reference: flag "hot" if `T_K >= T_hot`.

Results — per station (summary).
All stations produced `hot_frac_ssmt == hot_frac_raw`, hence `delta = 0.000000` per station (as expected from the monotone mapping `e_T = ln( T_K / T_ref )`).

station_id	hot_frac_ssmt	hot_frac_raw	delta
175	0	0	0
275	0	0	0
375	0	0	0
518	0.000113	0.000113	0
532	0	0	0
575	0	0	0
675	0	0	0
775	0	0	0
875	0	0	0
1075	0	0	0
1175	0	0	0
1275	0	0	0
1375	0	0	0
1475	0	0	0
1575	0	0	0
1775	0	0	0
1975	5.67E-05	5.67E-05	0
2075	0	0	0
2175	8.10E-06	8.10E-06	0
2275	0	0	0
2375	0	0	0
3723	0	0	0
3904	0	0	0
4935	0	0	0

Aggregate summary (across stations).

- `stations = <count from your run>`
- `ssmt_std = <std of hot_frac_ssmt>` ; `ssmt_iqr = <IQR of hot_frac_ssmt>`
- `raw_std = <std of hot_frac_raw>` ; `raw_iqr = <IQR of hot_frac_raw>`
- `mean_delta = 0.000000` (95% CI: `[0.000000 , 0.000000]`)

Interpretation.

• The symbolic rule (`e_T >= E_hot`) and the absolute-Kelvin rule (`T_K >= T_hot`) match exactly per station (`delta = 0`), confirming portability of a single threshold across heterogeneous sensors/sites.

• Near-zero hot fractions are consistent with the climate slice and a ~30 C threshold; using a warmer subset or a lower `E_hot` (e.g., `0.00952` for ~28 C) will yield non-trivial exceedance rates while preserving equivalence.

Changes by the authors: selection, filtering, and transformation to unitless SSMT symbols (`e_T, a_phase, Q_phase`). No endorsement implied.
Accessed on: 2025-09-24.

Appendix D — Indoor Comfort & Demand Planning (house-as-fleet)

Purpose.
A single-home, multi-room case study showing how to treat a house like a small fleet: rooms become "devices," and we apply one symbolic policy everywhere. We demonstrate comfort alerts, excursion budgets, freeze-risk gating, and manager-facing KPIs — all expressed in symbol space instead of °C/°F.

Methods (summary)

• **Data columns.** Indoor temperatures `T1..T9` (C), facade outdoor `T6` (C), and airport outdoor `T_out` (C), all timestamped.
• **Lens.** Log lens.
`e_T := ln( T_K / T_ref )`
with `T_ref = 298.15 K`.
• **Kelvin conversion.**
`T_K := max( to_kelvin(T_raw, unit_flag) , eps_TK )`
where `eps_TK > 0` (for example, `1e-6`) and `unit_flag` is `"C"` here.
• **Optional phase dial for outdoor freeze risk.**
`T_m = 273.15 K`, `DeltaT_m = 2.0 K`, `c_m = 1.2`, `eps_a = 1e-6`.
• **Soft hysteresis (outdoor, optional).**
`rho = 0.90`, `k_side = 2.0`.
• **Comfort threshold (symbolic).**
Choose `E_hot_room = 0.0` to represent "too warm indoors" (roughly corresponds to ~25 C in warmer rooms under this lens and anchor), and if needed define `E_cold_room` (for example, `0.10..0.20`) to capture "uncomfortably cool".

• **Windowing.**
Sampling cadence is 10 minutes. We recommend a dwell of 10–15 minutes (one or two samples) before calling a comfort alert. Daily totals (24h windows) are used for excursion budgets.

These definitions let all rooms be judged by the same symbolic dial — no per-room °C retuning, no °F vs °C logic forks.

D1. Comfort portability across rooms (no unit retuning)

Goal.
Use one comfort policy across all rooms `T1..T9` without rewriting thresholds per room or per unit.

Procedure.

1. Convert each room's raw reading to Kelvin and clamp:
   `T_K(room) := max( to_kelvin(T_room_raw, "C") , eps_TK ).`
2. Compute the room's symbolic contrast:
   `e_T_room := ln( T_K(room) / 298.15 ).`
3. Mark a room as HOT at time `t` if
   `e_T_room(t) >= E_hot_room`
   continuously for the dwell window (10–15 min).
4. Compute per-room hot exposure:
   `HotFraction_room := P_t( e_T_room(t) >= E_hot_room ).`
   This is the fraction of samples (or minutes) that the room spent in symbolic "too warm" territory.
5. Compute fleet spread across rooms:
   `PortabilitySpread := max_room(HotFraction_room) - min_room(HotFraction_room).`

Outcome (illustrative).
With `E_hot_room = 0.0`, different rooms show different `HotFraction_room` values under the exact same symbolic threshold. A spread on the order of `~0.10` (for example, `≈0.1061`) indicates meaningful thermal imbalance between rooms — discovered without ever rewriting a Celsius threshold per room.

Why this matters.
A single symbolic comfort rule can govern an entire building (or fleet) without manual per-room tuning. You get fairness, comparability, and actionable imbalance signals in one pass.

D2. Freeze-aware gating for outdoor exposure (optional)

Goal.
Generate a gentle, interpretable cold-risk signal from an outdoor channel (for example, `T_out` or facade `T6`). This can drive entryway policies, asset staging, or cold-chain handling.

Procedure.

1. Convert the chosen outdoor reading to Kelvin and clamp:
   `T_K(out) := max( to_kelvin(T_out_raw, "C") , eps_TK ).`
2. Compute standardized offset from freezing:
   `d_m := ( T_K(out) - 273.15 ) / 2.0.`
3. Map to a bounded cold/hot dial:
   `a_phase_out := tanh( 1.2 * d_m ).`

Then clamp to `(-1 + eps_a , 1 - eps_a)` to enforce numeric safety.
Interpretation:
• Negative `a_phase_out` means "cold-side stress / freeze exposure risk".
• Positive `a_phase_out` means "warm-side stress".

4. Optional short-term memory (soft hysteresis):
   `p_side := 0.5 * ( 1 + tanh( 2.0 * ( T_K(out) - 273.15 ) ) )`
   `Q_out := rho*Q_prev + (1 - rho)*clip( p_side , 0 , 1 )`
   where `rho = 0.90`.
   Interpretation: `Q_out` holds a smoothed memory of "we stayed on the freezing side" without flapping if the reading jitters around 0 C.

Outcome (illustrative).
Define a freeze-risk condition such as
`a_phase_out <= -0.10`.
A practical building run can show a nontrivial `FreezeRiskFraction := P_t( a_phase_out <= -0.10 )` (for example, `≈5.34%`). That number tells you "how often we were in meaningful cold stress," using a single symbolic rule instead of ad-hoc °C cutoffs.

Why this matters.
Cold ingress risk becomes machine-visible, schedulable, and auditable. You can tie staffing, HVAC staging, or door management to a symbolic cold stress signal instead of gut feel.

D3. Symbolic excursion budgets (degree-day analogs)

Goal.
Measure "how much too warm" (or "how much too cold") over time in a way that is unitless and directly comparable across rooms, homes, or sites.

Definition (per room).
For a chosen comfort line `e*`, define:

```
S-CDD(t1..t2) := sum_{tau in [t1..t2]} max( e_T_room(tau) - e* , 0 )
```

Interpretation:
• `S-CDD` is a symbolic cooling demand day analog.
• Higher values mean "we spent more time above comfort in that interval."
• You can compute this per day, per shift, or per billing cycle.
• Because `e_T_room` is already unitless and mapped to the same `T_ref`, **you can compare two different rooms or buildings directly.**

Practical note.
Choose `e*` to reflect the point where you consider a room uncomfortably warm. For example, `e* = 0.05` might correspond to "slightly above desired indoor comfort."

D4. Manager-friendly KPIs (compact, portable)

Let `Delta_min` be the sampling interval in minutes (here `Delta_min = 10`).

1. **ComfortViolationMinutes/day (per room)**
   `CVM_day := Delta_min * count_{t in day}( |e_T_room(t)| >= 0.7 )`
   Interpretation:
   Total minutes per day where the room drifted into "unacceptable thermal deviation," regardless of whether that deviation was too hot or too cold.
   This is a single KPI you can email to a building manager.
2. **MaxExcursion_eT/day (per room)**
   `MaxExcursion_eT_day := max_{t in day}( |e_T_room(t)| )`
   Interpretation:
   The single worst symbolic deviation that day.
   High values signal either a comfort failure or an HVAC control issue.
3. **PortabilitySpread (fleet stability)**
   `PortabilitySpread_hot := max_room( HotFraction_room ) - min_room( HotFraction_room )`
   (Similarly, define `PortabilitySpread_cold` using `E_cold_room`.)
   Interpretation:
   This measures fairness: how uneven the thermal experience is across rooms under the **same** symbolic comfort rule.
   A lower spread means more equitable comfort delivery with no retuning.
4. **Reporting defaults.**
   – Compute these KPIs once per day.
   – Use `E_hot_room = 0.0`.
   – Use `E_cold_room = 0.10..0.20` if you are also tracking "too cool".
   – Require a dwell of 10–15 min before flagging a HOT or COLD event.

Why this matters.
You can brief operations, facilities, finance, and compliance with the same KPIs, even if they manage different buildings in different climates. The KPIs are portable, numeric, and auditable.

D5. Optional “before/after” (Celsius vs. symbol) sanity check

When migrating an existing building rule ("Alert if any room exceeds T_thr_C") into SSMT:

1. Compute the symbolic equivalent threshold:
   `e_thr := ln( ( T_thr_C + 273.15 ) / 298.15 )`
2. For any room:
   `T_room >= T_thr_C <=> e_T_room >= e_thr`
   because `ln(T_K / T_ref)` is monotone in `T_K`.

Result.
Your old comfort alarm and your new symbolic alarm are mathematically aligned. But the symbolic alarm is now portable: it drops into KPIs, pooling, hysteresis, and cross-room fairness metrics without per-room °C tuning.

Why this matters.
This is how you migrate without political friction. You can prove that the symbolic policy is not "stricter" or "looser" — it is just **more transferable, more auditable, and easier to govern.**

D6. Reference encoder (non-normative, copy-ready)

This block shows how a simple device-side or gateway-side script could emit an SSMT-compatible record.

Given:
• Raw reading `T_raw` in °C.
• Kelvin floor `eps_TK`.
• Declared reference `T_ref`.
• Declared pivot `T_m`, softness `DeltaT_m`, and sharpness `c_m`.
• Dwell/hysteresis state `Q_prev`.
• Alignment sharpness `c_T`.

Steps (ASCII):

1. Convert to Kelvin and clamp:
   `T_K := max( T_raw + 273.15 , eps_TK )`
2. Compute contrast using the declared lens (log lens shown):
   `e_T := ln( T_K / T_ref )`
3. Optional bounded alignment dial:
   `a_T := tanh( c_T * e_T )`
4. Phase proximity around a pivot (for example, freezing / comfort / warp point):
   `d_m := ( T_K - T_m ) / DeltaT_m`
   `a_phase := tanh( c_m * d_m )`
5. Optional soft hysteresis memory:
   `p_side := 0.5 * ( 1 + tanh( k_side * ( T_K - T_m ) ) )`
   `Q_phase := rho*Q_prev + (1 - rho)*clip( p_side , 0 , 1 )`
6. Emit a minimal record (illustrative structure):
   `{ timestamp_utc, e_T, a_phase, Q_phase, manifest_id, health }`

Notes.
• `manifest_id` refers to a published manifest that declares `T_ref`, lens type, pivots (`T_m`, `DeltaT_m`, `c_m`), ranges, clamps, and privacy policy.
• If you also emit `a_phase_fused`, include the ordered list of pivots that were fused.
• If you also emit `a_T`, you MUST publish `c_T` and any clamp parameters (for example, `eps_a`).
• This encoder is informative. Your production encoder MUST keep the same math for a given manifest so that downstream consumers can audit it.

Attribution and license

Addendum E — Multi-City Threshold Portability & Freeze Hysteresis (Meteostat Hourly)

A cross-city case study demonstrating **(1)** hot/cold threshold portability in **symbol space**, and **(2)** freeze-risk flicker reduction via **soft hysteresis** near `0 C`. Files: `10381.csv.gz` (Berlin–Dahlem), `10382.csv.gz` (Berlin–Tegel), `72494.csv.gz` (San Francisco Intl).

Methods (summary).
• **Cadence & coverage:** each station has **8,784 hours** (2024, hourly, leap year).
• **Columns used:** `time (UTC)`, `temp (C)`.
• **Lens:** `log`; **anchors:** `T_ref = 298.15 K`.
• **Symbol:** `e_T = ln( T_K / 298.15 )`, where `T_K = temp_C + 273.15`.
• **Freeze dial (optional):** `a_phase = tanh( 1.2 * ( T_K - 273.15 ) / 2.0 )`, clamped to `(-1+1e-6, +1-1e-6)`.
• **Memory (optional):**
`p_side = 0.5 * ( 1 + tanh( 2.0 * ( T_K - 273.15 ) ) )`
`Q_t = 0.90 * Q_{t-1} + 0.10 * clip( p_side , 0 , 1 )` (init `Q_0 = 0.5`).
• **Hot thresholds (illustrative):** `E_hot ∈ {0.00, 0.02, 0.05, 0.10}`. For intuition:
– `E_hot=0.02` ⇒ ~`31.02 C`; `E_hot=0.05` ⇒ ~`40.29 C`; `E_hot=0.10` ⇒ ~`56.36 C` (relative to `25 C` baseline).

• **Freeze policy (symbolic):** enter risk if `a_phase <= -0.10`; clear with either **band hysteresis** (`a_phase >= +0.10`) or **memory** (`Q_phase >= 0.70`).
• **Flicker metric:** number of state changes over the year (no dwell applied here).
• **Policy mapping (exact equivalence):** old Celsius threshold `T_thr_C` ↔ `e_thr = ln( ( T_thr_C + 273.15 ) / 298.15 )`.

E1. Threshold portability across cities (results).
`HotFraction = fraction of hours with e_T >= E_hot.`

E_hot = 0.00 *(at/above baseline 25 C; "warm day" band)*

Station	HotHours/Total	HotFraction
10381	423 / 8784	0.048156
10382	390 / 8784	0.044399
72494	118 / 8784	0.013434

PortabilitySpread = 0.034722

E_hot = 0.02 *(~31.0 C; "truly hot" band)*

Station	HotHours/Total	HotFraction
10381	34 / 8784	0.003871
10382	17 / 8784	0.001935
72494	12 / 8784	0.001366

PortabilitySpread = 0.002505

Higher bands for reference
• `E_hot = 0.05` (~40.3 C): `HotFraction = 0.000000` for all three stations →
PortabilitySpread = 0.000000
• `E_hot = 0.10` (~56.4 C): `HotFraction = 0.000000` for all three stations →
PortabilitySpread = 0.000000

Takeaway. One global, unitless threshold applies uniformly; the spread reflects true climatology rather than unit conversions or site-specific retuning.

E2. Freeze-risk flicker reduction (results).
Compare three regime detectors per station over all **8,784 hours**:
• **Raw Kelvin toggle:** risk if `T_K <= 273.15`.
• **Band hysteresis (symbolic):** enter if `a_phase <= -0.10`, clear if `a_phase >= +0.10`.
• **Memory hysteresis (symbolic):** enter if `a_phase <= -0.10`, clear if `Q_phase >= 0.70`.

For each: report **RiskFraction** and **Flicker** (state changes).

10381 (Berlin–Dahlem)
• RiskFraction: raw `0.049066`, band `0.047700`, memory `0.061248`
• Flicker: raw `156`, band `134` (**−14.10%**), memory `144` (**−7.69%**)

10382 (Berlin–Tegel)
• RiskFraction: raw `0.038821`, band `0.037341`, memory `0.044854`
• Flicker: raw `114`, band `98` (**−14.04%**), memory `102` (**−10.53%**)

72494 (San Francisco Intl)
• RiskFraction: raw `0.000000`, band `0.000000`, memory `0.000000`
• Flicker: raw `0`, band `0`, memory `0`

Interpretation.
• ~**14%** flicker reduction with a **symmetric band** on `a_phase` *(Dahlem: 156→134; Tegel: 114→98)*.
• **Memory hysteresis** keeps risk "sticky" (slightly higher RiskFraction, lower flicker) — desirable when sustained warmth should confirm clearance.
• SFO shows **zero flicker** (no freezing), validating a single policy that's portable and climate-appropriate.

Reproducibility notes.
• `e_T = ln( T_K / 298.15 )` with `T_K = temp_C + 273.15`.
• `a_phase = tanh( 1.2 * ( T_K - 273.15 ) / 2.0 )`, clamp to `( -1 + 1e-6 , +1 - 1e-6 )`.
• `Q_t = 0.90 * Q_{t-1} + 0.10 * clip( 0.5 * ( 1 + tanh( 2.0 * ( T_K - 273.15 ) ) ), 0, 1 )`.
• Tables provided: **Coverage**, **HotFraction by E_hot**, **PortabilitySpread**, **Freeze-risk flicker & fractions**.
• Dwell windows can be layered on top (e.g., **3 h** for weather advisories) without changing the symbol definitions.

Attribution and license.

Addendum F — Lens Decision Note (one-pager template)

Why this exists: a concise, sign-off aid for choosing a lens, anchors, pivots, and guards before deployment. Keeps decisions auditable and stable.

F.1 How to use (2 minutes).

1. Fill the template below (ASCII).
2. Paste the manifest snippet into your repo/config.
3. Attach golden vectors and run Tier S1 tests.

4. Record sign-offs and date; publish alongside the manifest (include `manifest_id` and `manifest_hash`).

F.2 Template (fill-in).

Study/Deployment: <name>
Owner(s): <names>
Date: <YYYY-MM-DD>
Cadence & Scope: <hourly / 10 min / per-device>; **Domains:** <weather / HVAC / cold-chain / space>
Data Columns: <temp unit & field names>
To-K step: `T_K = max( to_kelvin(T_raw, unit) , eps_TK )` with `eps_TK = <1e-6 default or chosen>`

Chosen lens (exactly one): `<log | linear | beta | kBT | hybrid | qlog>`
Rationale (1–3 lines): <wide ranges / narrow band / cold-sensitive / energy-linked / mixed-regime / near-zero safety>

Anchors (publish):
• `T_ref = <...> K`
• `DeltaT = <...> K` *(linear only)*
• `E_unit = <...> J/mol` *(kBT only)*
• `tau = <...> K` *(hybrid only; linear inside* `|T_K - T_ref| <= tau`*, log outside)*
• `alpha = <...>` *(qlog only;* `alpha > 0`*)*

Phase pivots (optional):
• `T_m_list = [ { tag:"<material>", T_m:<...> K, DeltaT_m:<...> K, c_m:<...> } ]`
• **Soft hysteresis (if used):** `rho = <0..1>`, `k_side = >0`
• **(Optional) Multi-pivot fused dial:** define how `a_phase_fused` is computed from `T_m_list` and publish `T_m_tag_list`.

Validity range (must):
• `T_valid_range_K = [ T_min , T_max ]` with `0 < T_min < T_max`
• **Precision:** `precision_K = <...>`, `precision_e_T = <...>` *(optional)*

Emissions (compliance level):
• **S1:** `e_T` ; **S2:** `e_T + a_phase (+Q_phase)` ; **S3:** `+ a_T`
• If `a_T` emitted: `c_T = <...>`, `eps_a = 1e-6 (or chosen)`, `eps_w = 1e-12 (if pooling)`

Guards (domain & clamps):
• **log/beta:** require `T_K > 0`, `T_ref > 0`
• **linear:** `DeltaT > 0` ; **kBT:** `E_unit > 0`
• **hybrid:** `tau > 0` *(publish)*
• **qlog:** `alpha > 0`, `T_ref > 0` *(publish)*
• **Kelvin floor:** `eps_TK >= 0` *(publish if not default)*
• **Clamp:** `|a| <= 1 - eps_a`

Policy thresholds (symbol space):
• `E_hot = <...>`, `E_cold = <...>`, `Phi_freeze = <...>`, `E_hyst = <...>`
• **Dwell windows:** <values matched to cadence>

Mapping note (for legacy °C thresholds):
• `e_thr = ln( ( T_thr_C + 273.15 ) / T_ref )` // exact, monotone

Tests to run (Tier S1): unit invariance; lens zero/monotone; domain guards; clamps; phase symmetry; soft-hysteresis shape; validity/saturation; pooling bounds *(if S3)*; Kelvin floor applied; hybrid switch *(tau)* and qlog *(alpha)* behave as declared.

Risk/assumption notes (1–2 lines): <extreme regime? near 0 K? sensor lag? non-classical phase?>

Sign-off:
• **Spec owner (A/R):** <name>
• **Data steward (R):** <name>
• **Ops (R):** <name>
• **Security/Privacy (C):** <name>
• **Partners (I):** <names>

F.3 Mini decision guide (quick).

• **log:** `e_T = ln( T_K / T_ref )`
Use when ranges are wide / cross-fleet / cross-planet. Robust to multiplicative changes.

• **linear:** `e_T = ( T_K - T_ref ) / DeltaT`
Use for tight bands near a setpoint (comfort, cabinets). Easy interpretability.

• **beta:** `e_T = ( T_ref / T_K ) - 1`
Use when cold-side sensitivity matters (cryo, radiative regimes).

• **kBT:** `e_T = ( R*T_K )/E_unit - ( R*T_ref )/E_unit`
Use when gating by energy scale (chem/bio coupling, reaction windows).

• **hybrid (piecewise):**
`e_T = ( T_K - T_ref ) / DeltaT` if `|T_K - T_ref| <= tau`, else `e_T = ln( T_K / T_ref )`
Use when you need linear interpretability near baseline and log robustness for larger departures. Publish `tau > 0` *(and `DeltaT`)*.

• **qlog (near-zero safety):**
`e_T = ln( ( T_K/T_ref + alpha ) / ( 1 + alpha ) )` with `alpha > 0`
Use when operating near absolute zero to avoid `log(0)` issues while preserving monotonicity. Publish `alpha`.

Anti over-engineering: start **S1**; add **S2** if freezing matters; add **S3** only for bounded pooling/ML priors.

F.4 Manifest snippet (copy-paste skeleton).

```
SSMT_manifest_v1:
  version: "1.1"
  compliance_level: "<S1|S2|S3>"
  lens: "<log|linear|beta|kBT|hybrid|qlog>"
  anchors: { T_ref: <...>, DeltaT: <null|...>, E_unit: <null|...>, c_T:
<null|...>, tau: <null|...>, alpha: <null|...> }
  guards:  { clamp_a_eps: 1e-6, eps_w: 1e-12, eps_TK: <null|1e-6>,
T_valid_range_K: [ <T_min>, <T_max> ] }
  phase_block:
    T_m_list: [ { tag:"<material>", T_m:<...>, DeltaT_m:<...>, c_m:<...> }
]
    rho: <0.90>
    k_side: <2.0>
  pooling: { weight_rule: "<equal|inv_var|health>" }
  outputs: { emit_e_T: true, emit_a_phase: <bool>, emit_Q_phase: <bool>,
emit_a_T: <bool> }
  auto_lens_policy:
    enabled: <false>
    rules:
      - if: { span_K: ">= 500" }   then: { lens: "log" }
      - if: { min_K: "< 100" }     then: { lens: "beta" }
      - if: { span_K: "<= 50" }    then: { lens: "linear" }
      - else:                      { lens: "log" }
  notes: "<1–2 line rationale; include resolved lens at commissioning>"
```

F.5 Golden vectors (attach).

List a few `(unit, T_raw) -> T_K -> e_T -> a_phase?` rows *(include edge cases, near pivot, OOR, and—if used—hybrid switch around `tau` and qlog behavior with `alpha`)*. Store as a small CSV in your repo and pin expected outputs `(tol 1e-6)`.

F.6 Acceptance (tick-box).

[ ] Single lens fixed *(or auto policy resolved once)*
[ ] Anchors published *(incl. `tau`/`alpha` if applicable)*
[ ] Validity range declared; Kelvin floor (`eps_TK`) set
[ ] Guards pass Tier S1 *(domains, clamps, hysteresis shape)*
[ ] Policy thresholds in symbol space
[ ] Manifest + lens note committed *(with `manifest_hash`)*
[ ] Sign-offs completed

Addendum G — Data Sources Catalog (types • file patterns • licenses)

A practical catalog of temperature data sources you can plug into SSMT. For each source we note: **What it is**, **How to fetch** (high-level patterns), **Units/fields**, and a **License summary**. Always verify the current license and attribution text before publishing.

G1) Public, no sign-up (bulk CSV)

Meteostat — Hourly Bulk (per station/year)

- **What:** Global hourly observations aggregated from national weather services and aviation networks.
- **Fetch (pattern):** Station/year `.csv.gz` files organized by year and station ID.
- **Units/fields:** `time` (UTC), `temp` (°C), plus standard weather variables.
- **License (summary):** CC BY-NC 4.0 (non-commercial).
- **SSMT ingest hint:** `T_K = temp_C + 273.15`; timestamps are UTC.

NOAA / NCEI — ISD (Integrated Surface Database), Hourly

- **What:** Global hourly/synoptic surface observations, multi-decade coverage.
- **Fetch:** Bulk archives and programmatic access via official portals or public mirrors.
- **Units/fields:** Air temperature typically in °C (sometimes scaled integers).
- **License (summary):** U.S. Government data (generally open; check use constraints).
- **SSMT ingest hint:** Standardize to UTC; `T_K = T_C + 273.15`.

G2) Public, sign-up/API

Copernicus / ECMWF — ERA5 Hourly (2 m temperature)

- **What:** Global reanalysis, hourly; gridded fields suitable for modeling.
- **Fetch:** Climate Data Store API; request spatiotemporal subsets.
- **Units/fields:** `2m_temperature` in Kelvin.
- **License (summary):** Copernicus open data licence (attribution required).
- **SSMT ingest hint:** Use Kelvin directly; pick `T_ref` and encode `e_T`.

NASA POWER (API)

- **What:** Global analysis-ready meteorology/solar parameters (incl. 2 m air temperature).
- **Fetch:** REST API returning JSON/CSV; specify coordinates/time range.
- **Units/fields:** 2 m air temperature in °C; timestamps commonly UTC.
- **License (summary):** NASA open data; review service terms.
- **SSMT ingest hint:** `T_K = T_C + 273.15`; record coordinates and data vintage.

ERA5-Land (via geospatial platforms)

- **What:** Higher-resolution land reanalysis, hourly.
- **Fetch:** Platform APIs or export tools; tile/grid selection required.
- **Units/fields:** 2 m temperature in Kelvin.
- **License (summary):** Copernicus open data licence.
- **SSMT ingest hint:** Use `T_K` directly; log lens works well for wide ranges.

G3) Curated portals / academic

Met Éireann — Ireland hourly (curated packs)

- **What:** Multi-station hourly observations consolidated over many years.
- **Fetch:** Curated bundles via research portals.
- **Units/fields:** °C with station/time columns.
- **License (summary):** Commonly CC BY 4.0 on curated releases; verify per bundle.
- **SSMT ingest hint:** Good for freeze-dial demonstrations across sites.

UCI ML Repository — "Appliances Energy Prediction"

- **What:** Single home, multi-room temps at 10-min cadence (`energydata_complete.csv`).
- **Fetch:** Direct CSV download.
- **Units/fields:** Indoor `T1..T9` (°C), outdoor fields (°C), timestamp column.
- **License (summary):** CC BY 4.0 (attribution required).
- **SSMT ingest hint:** Per-room `e_T` + optional outdoor `a_phase` for comfort and `S-CDD` budgets.

Berkeley Earth — Gridded Surface Temperature (daily/monthly)

- **What:** Global gridded fields for climatology (not hourly).
- **Fetch:** Downloadable archives; documentation provided.
- **Units/fields:** °C anomalies and/or absolute temperatures by product.
- **License (summary):** CC BY-NC 4.0 (non-commercial).
- **SSMT ingest hint:** Use daily `e_T` for climatology or KPI rollups.

G4) Quick mapping to SSMT (ingest cheatsheet)

- **CSV column to Kelvin:**
  `T_K = temp_C + 273.15` *(observational CSVs)*
  `T_K = 2m_temperature` *(reanalysis in Kelvin)*
- **Symbol encoding (log lens):**
  `e_T = ln( T_K / T_ref )` // publish `T_ref` in manifest
- **Phase dial near freezing (optional):**
  `a_phase = tanh( c_m * ( T_K - T_m ) / DeltaT_m )` with `T_m = 273.15 K`
- **Time:** prefer UTC for joins/audit. Document timezone if not UTC.

G5) Governance & license reminders (must-read)

- **Attribution text:** retain each provider's required wording (e.g., CC BY, Copernicus attribution).
- **Redistribution:** some sources restrict commercial re-hosting (e.g., non-commercial clauses).
- **Competition-gated data:** avoid sources that require joining a competition to view/use.
- **Manifest notes:** record dataset name, access path/pattern, and license in `validation.dataset_ref` for reproducibility.

Addendum H — Repro Harness (minimal, deterministic, copy-ready)

A tiny, ASCII-only toolkit to regenerate SSMT numbers, figures, and tables exactly—suitable for CI and notebooks. Ships with core math, a CLI, and golden vectors from Tier S1.

H.1 Scope & guarantees (what this harness promises)

• **Deterministic** (float64 end-to-end; no locale dependence).
• **ASCII-only** formulas and outputs (screen-reader friendly).
• **Stable** clamping and guard semantics (see 1.12).
• **Kelvin floor supported:** `T_K = max( to_kelvin(...), eps_TK )` (publish `eps_TK` if non-default).
• **Lenses supported:** `log | linear | beta | kBT | hybrid | qlog`.
• **Optional multi-pivot** phase fusion via rapidity pooling (`a_phase_fused`).
• **Golden vectors** (Tier S1) with exact expected values and tolerances.
Outputs use ISO-8601 UTC and dot decimals. All formulas are plain ASCII.

H.2 Reference layout (drop-in structure)

```
ssmt/
  ssmt_core.py        # pure functions: to_kelvin, encode_eT, a_phase,
a_phase_fused, q_phase_update, clamp_a, pool_alignments
  ssmt_cli.py         # tiny CLI wrapper (argparse)
  golden/
    vectors_A_to_E.csv          # unit tests for Tier S1 (A–E)
    expected_A_to_E.csv         # expected outputs (6+ dp)
    vectors_optional_HQ.csv     # OPTIONAL: hybrid/qlog/Kelvin-floor checks
    expected_optional_HQ.csv
  manifests/
    sample_S1.yaml
```

```
    sample_S2.yaml
    sample_S3.yaml
  tests/
    test_tier_S1.py    # asserts with tol=1e-6 (or tighter where noted)
README.md
```

H.3 Core math (ASCII, copy-ready)

Python-like reference; keep numbers in float64. `ln(x)` is the natural log.

```
def to_kelvin(T_raw, unit):
  if unit == "K": return float(T_raw)
  if unit == "C": return float(T_raw) + 273.15
  if unit == "F": return (float(T_raw) - 32.0) * 5.0/9.0 + 273.15
  raise Exception("unknown unit")

def enforce_kelvin_floor(T_K, eps_TK=0.0):
  # eps_TK >= 0; publish if not default
  return max(float(T_K), float(eps_TK))

def encode_eT(T_K, lens, T_ref, DeltaT=None, E_unit=None, R=8.314462618,
tau=None, alpha=None):
  # domain guards (must) are enforced by caller:
  # log/beta/qlog: require T_K > 0 and T_ref > 0
  # linear: require DeltaT > 0
  # kBT: require E_unit > 0
  # hybrid: require DeltaT > 0 and tau > 0
  if lens == "log"   : return ln(T_K / T_ref)
  if lens == "linear": return (T_K - T_ref) / DeltaT
  if lens == "beta"  : return (T_ref / T_K) - 1
  if lens == "kBT"   : return (R*T_K)/E_unit - (R*T_ref)/E_unit
  if lens == "hybrid":
    # linear near baseline, log for larger departures
    if abs(T_K - T_ref) <= tau:
      return (T_K - T_ref) / DeltaT
    else:
      return ln(T_K / T_ref)
  if lens == "qlog":
    # near-zero-safe log; alpha > 0
    return ln( ( (T_K / T_ref) + alpha ) / (1.0 + alpha) )
  raise Exception("unknown lens")

def clamp_a(a, eps_a):
  return min(1.0 - eps_a, max(-1.0 + eps_a, a))

def a_T(e_T, c_T, eps_a):
  return clamp_a( tanh(c_T * e_T), eps_a )

def a_phase(T_K, T_m, DeltaT_m, c_m, eps_a):
  d = (T_K - T_m) / DeltaT_m
  return clamp_a( tanh(c_m * d), eps_a )

def a_phase_fused(T_K, pivots, weights=None, eps_a=1e-6, eps_w=1e-12):
  # pivots: list of tuples (T_m, DeltaT_m, c_m)
  # compute individual a_phase_i and fuse via rapidity pooling
  A = []
  for (T_m, dTm, c_m) in pivots:
    A.append( a_phase(T_K, T_m, dTm, c_m, eps_a) )
  if weights is None:
```

```
    weights = [1.0 for _ in A]
  return pool_alignments(A, weights, eps_a=eps_a, eps_w=eps_w)

def q_phase_update(T_K, T_m, k_side, rho, Q_prev):
  p_side = 0.5 * (1.0 + tanh( k_side * (T_K - T_m) ))
  return rho * Q_prev + (1.0 - rho) * clip(p_side, 0.0, 1.0)

def pool_alignments(a_list, w_list, eps_a=1e-6, eps_w=1e-12):
  U = 0.0
  W = 0.0
  for (a, w) in zip(a_list, w_list):
    U += w * atanh( clamp_a(a, eps_a) )
    W += w
  W = max(W, eps_w)
  return tanh(U / W)
```

Domain guards (must) — enforce before calling `encode_eT`:

```
log, beta, qlog: T_K > 0 and T_ref > 0
linear: DeltaT > 0
kBT: E_unit > 0
hybrid: DeltaT > 0 and tau > 0
```

Kelvin floor (optional, recommended to publish if used):

```
T_K = enforce_kelvin_floor( to_kelvin(T_raw, unit), eps_TK )
```

H.4 Tiny CLI (examples; use argparse)

Commands return plain numbers or CSV with headers. All examples are ASCII-only.

```
# Kelvin conversion (+ optional floor)
ssmt to-k --val 35 --unit C --epsTK 0.0
# -> 308.15

# Contrast (log lens)
ssmt eT --TK 308.15 --Tref 298.15 --lens log
# -> 0.03298996

# Contrast (hybrid lens: DeltaT + tau required)
ssmt eT --TK 301.0 --Tref 298.15 --lens hybrid --DeltaT 5.0 --tau 2.0

# Contrast (qlog lens: alpha required)
ssmt eT --TK 5.0 --Tref 298.15 --lens qlog --alpha 0.01

# Phase dial (near freezing)
ssmt aphase --TK 271.15 --Tm 273.15 --dTm 2.0 --cm 1.2 --epsa 1e-6
# -> -0.83365461

# Fused phase (multi-pivot; weight list optional)
ssmt aphase-fused --TK 271.5 --pivots
"water:273.15:2.0:1.2,brine:268.0:2.5:1.1" --weights "1,1" --epsa 1e-6

# Soft hysteresis over a file (CSV with column TK)
ssmt qphase --csv in.csv --col TK --Tm 273.15 --k_side 2.0 --rho 0.90 --out
qphase.csv
# -> writes qphase.csv with Q_phase column

# Pooling (alignments + weights)
```

```
ssmt pool --a 0.2 0.4 -0.1 --w 1 1 1
# -> pooled alignment in (-1,+1), eps-guarded
```

H.5 Golden vectors (Tier S1, copy-ready)

Save as `golden/vectors_A_to_E.csv`; expected outputs in `golden/expected_A_to_E.csv`.
Tolerance unless stated: `abs(err) <= 1e-6`.

A) Unit invariance & lens (log, T_ref=298.15 K)

```
case, unit, T_raw, T_K, lens, e_T_expected
A1a, "C", 35.0, 308.15, log, 0.03298996
A1b, "F", 95.0, 308.15, log, 0.03298996
A1c, "K", 308.15,308.15, log, 0.03298996
```

B) Lens zero & monotonicity

```
B1, K, 298.15, 298.15, log, 0.0
B2_lo, K, 290.00, 298.15, linear: DeltaT=10.0, -0.815
B2_hi, K, 300.00, 298.15, linear: DeltaT=10.0, 0.185
# test compares B2_hi > B2_lo, not exact values
```

C) Domain guards (flag or reject config at init)

```
C1, log , T_K=0.0, T_ref=298.15 -> sensor_ok=false
C2, beta, T_K=298.15, T_ref=0.0 -> sensor_ok=false
C3, linear, DeltaT=0.0 -> invalid config
C4, kBT, E_unit=0.0 -> invalid config
```

D) Alignment clamps (c_T=0.7, eps_a=1e-6)

```
D1, e_T=+1000 -> a_T = tanh(700) -> <= +0.999999
D2, e_T=-1000 -> a_T = tanh(-700)-> >= -0.999999
```

E) Phase dial symmetry (T_m=273.15, DeltaT_m=2.0, c_m=1.2, eps_a=1e-6)

```
E1, TK=271.15 -> d=-1.0 -> a_phase=-0.83365461
E2, TK=275.15 -> d=+1.0 -> a_phase=+0.83365461
E3, TK=273.15 -> d= 0.0 -> a_phase=0.0
```

Hysteresis shape check (sequence; rho=0.90, k_side=2.0, Q0=0.30)

`Input sides s= [+1,+1,+1,+1,+1, -1,-1,-1, +1,+1]`
via `p_side = 0.5*(1 + tanh(2.0*s));`
assert `0.0 <= Q <= 1.0` and monotone response on runs (no oscillatory overshoot).

Pooling associativity

```
a_pool_1 = pool([0.2, 0.4], [1, 1])
a_pool_2 = pool([a_pool_1, -0.1], [2, 1])
a_pool_3 = pool([0.2, pool([0.4, -0.1],[1,1])], [1, 2])
```

Assert `|a_pool_2 - a_pool_3| < 1e-12` and `|a_pool_2| < 1`.

Optional add-ons *(separate files `vectors_optional_HQ.csv`, `expected_optional_HQ.csv`):*

F) Kelvin floor check

```
F1, eps_TK=1e-4, T_raw=-273.15 C -> to_kelvin=0.0 -> T_K=1e-4 after floor
```

G) Hybrid switch behavior (DeltaT=5.0, tau=2.0, T_ref=298.15)

```
G1, T_K=299.15 (|delta|=1.0 <= tau) -> linear -> e_T= (1.0/5.0)=0.2
G2, T_K=305.15 (|delta|=7.0 > tau) -> log -> e_T= ln(305.15/298.15)
```

H) qlog behavior near zero (alpha=0.01, T_ref=298.15)

```
H1, T_K=1.0 -> e_T = ln( ((1.0/298.15)+0.01) / (1.0+0.01) ) # finite,
monotone
```

H.6 CI recipe (bash, copy-ready)

Run on every change to firmware/pipelines/manifests.

```
# 1) Lint & types (optional)
python -m pip install ruff mypy
ruff check ssmt
mypy ssmt

# 2) Unit tests (Tier S1)
python -m pip install pytest numpy
pytest -q

# 3) Golden vector diff (strict)
python ssmt_cli.py run-golden --in golden/vectors_A_to_E.csv --exp
golden/expected_A_to_E.csv --tol 1e-6

# 4) Optional hybrid/qlog checks
python ssmt_cli.py run-golden --in golden/vectors_optional_HQ.csv --exp
golden/expected_optional_HQ.csv --tol 1e-6

# 5) Manifest schema check (optional)
python -m pip install jsonschema pyyaml
python tools/validate_manifest.py manifests/*.yaml

# Reporting: emit a compact JUnit/Markdown summary: pass/fail, max abs
error per case, and seed/version info.
```

H.7 Minimal notebook (optional, one page)

Cells to:

1. Load a CSV `(date,temp_C)`, convert to `T_K = max(to_kelvin(temp_C,"C"), eps_TK)`, compute `e_T` (chosen lens).
2. If needed, compute `a_phase` near freezing (`T_m=273.15, DeltaT_m=2.0, c_m=1.2`).
3. (Optional) Compute `a_phase_fused` from multiple pivots; plot `e_T` and `a_phase` time series; render a small table (HotFraction, flicker) for the chosen thresholds.
4. Save a small CSV `(timestamp_utc,e_T,a_phase?,a_phase_fused?)` for audit. All formulas must remain ASCII; include alt-text for plots (see 8.6.3).

H.8 Determinism & precision rules (must)

• Use **float64**.
• **Clamp** before `atanh` (pooling) and after `tanh` (dials).
• **Default tolerances:** `1e-6` (spec math), `1e-12` (associativity check).
• **Round only** for human display; keep raw values in symbol space.
• **No per-sample lens switching** (resolve once at commissioning; see 1.11).
• If **eps_TK** is used, **publish** it with the manifest.

H.9 Sample CLI manifest (S1/S2/S3, skeleton)

```
SSMT_manifest_v1:
  version: "1.1"
  compliance_level: "S1"
  lens: "log"
  anchors:  { T_ref: 298.15, DeltaT: null, E_unit: null, tau: null, alpha:
null }
  guards:   { clamp_a_eps: 1e-6, eps_w: null, eps_TK: null,
T_valid_range_K: [180.0, 500.0] }
  phase_block: { T_m_list: [], rho: 0.90, k_side: 2.0 }
  outputs:  { emit_e_T: true, emit_a_phase: false, emit_Q_phase: false,
emit_a_T: false }
  notes: "Starter S1 manifest; see 1.12 for knobs & guards"
```

H.10 Release artifacts (what to publish)

• `ssmt_core.py`, `ssmt_cli.py` *(MIT/Apache-2.0 suggested).*
• `golden/vectors_A_to_E.csv` and `golden/expected_A_to_E.csv`.
• **OPTIONAL:** `vectors_optional_HQ.csv` and `expected_optional_HQ.csv` (hybrid/qlog/floor).
• `tests/test_tier_S1.py` with strict tolerances.
• `manifests/sample_*.yaml` and a short **Policy Pack** (`E_hot, E_cold, Phi_freeze, E_hyst, dwell times`).
• **README** with instructions to regenerate addenda and cite datasets in their respective addenda.

Addendum I — Visual Aids (ASCII-only, copy-ready)

I.1 Lens comparison (table; place after Section 1.3 “Compute the contrast (e_T)”)

Setup (fixed for this table):
• T_ref = 298.15 K
• DeltaT = 10.0 K

• tau = 5.0 K (hybrid threshold; uses linear inside |T_K - T_ref| <= tau, else log)
• alpha = 1.0 (qlog offset; e_T = ln((T_K/T_ref + alpha) / (1 + alpha)))
• kBT lens uses E_unit = R * T_ref (so e_T_kBT = T_K/T_ref - 1)
• width = 2.0 K (smooth_hybrid blend; s = 1 / (1 + exp(-(abs(T_K - T_ref) - tau) / width)); e_T_smooth_hybrid = s * ln(T_K / T_ref) + (1 - s) * (T_K - T_ref) / DeltaT)

Notes for editors
• Keep formulas and symbols in plain ASCII in captions and labels.
• If you render as a figure later, label each lens in text (do not rely on color alone), and place the legend outside the plot area (top-left or top-right) to avoid overlaps.

Table — e_T values from T_ref - 50 K to T_ref + 50 K (step = 5 K)

T_K	e_T_log	e_T_linear(DeltaT=10)	e_T_beta	e_T_kBT(E_unit=R*T_ref)	e_T_hybrid(tau=5,DeltaT=10)	e_T_smooth_hybrid(tau=5,width=2,DeltaT=10)
248.150000	-0.183563	-5.000000	0.201491	-0.167701	-0.183563	-0.183563
253.150000	-0.163615	-4.500000	0.177760	-0.150931	-0.163615	-0.163615
258.150000	-0.144056	-4.000000	0.154949	-0.134161	-0.144056	-0.144056
263.150000	-0.124873	-3.500000	0.133004	-0.117391	-0.124873	-0.124873
268.150000	-0.106050	-3.000000	0.111878	-0.100620	-0.106050	-0.106050
273.150000	-0.087576	-2.500000	0.091525	-0.083850	-0.087576	-0.087576
278.150000	-0.069436	-2.000000	0.071904	-0.067080	-0.069436	-0.069436
283.150000	-0.051620	-1.500000	0.052975	-0.050310	-0.051620	-0.051620
288.150000	-0.034116	-1.000000	0.034704	-0.033540	-0.034116	-0.034116
293.150000	-0.016912	-0.500000	0.017056	-0.016770	-0.500000	-0.258456
298.150000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
303.150000	0.016631	0.500000	-0.016493	0.016770	0.500000	0.258316
308.150000	0.032990	1.000000	-0.032452	0.033540	0.032990	0.032990
313.150000	0.049086	1.500000	-0.047900	0.050310	0.049086	0.049086
318.150000	0.064926	2.000000	-0.062863	0.067080	0.064926	0.064926
323.150000	0.080520	2.500000	-0.077363	0.083850	0.080520	0.080520
328.150000	0.095874	3.000000	-0.091422	0.100620	0.095874	0.095874
333.150000	0.110996	3.500000	-0.105058	0.117391	0.110996	0.110996
338.150000	0.125893	4.000000	-0.118291	0.134161	0.125893	0.125893
343.150000	0.140571	4.500000	-0.131138	0.150931	0.140571	0.140571
348.150000	0.155037	5.000000	-0.143616	0.167701	0.155037	0.155037

Notes for editors

- Keep formulas and symbols in plain ASCII in captions and labels.
- If you render as a figure later, label each lens in text (do not rely on color alone), and place the legend outside the plot area (top-left or top-right) to avoid overlaps.

I.2 — ASCII curve sketches

Use a monospaced font. Not to scale; labels indicate qualitative shape around T_ref.

I.2 A) Near-baseline shapes (T_K around T_ref; show tau band)

T_K	e_T_log	e_T_linear(DeltaT=10)	e_T_hybrid(tau=5)	e_T_smooth_hybrid(tau=5,width=2)
292.15	-0.020329	-0.600000	-0.020329	-0.239179
293.15	-0.016912	-0.500000	-0.500000	-0.258456
294.15	-0.013507	-0.400000	-0.400000	-0.254083
295.15	-0.010113	-0.300000	-0.300000	-0.222037
296.15	-0.006731	-0.200000	-0.200000	-0.164743
297.15	-0.003361	-0.100000	-0.100000	-0.090071
298.15	0.000000	0.000000	0.000000	0.000000
299.15	0.003352	0.100000	0.100000	0.090063
300.15	0.006686	0.200000	0.200000	0.164735
301.15	0.010012	0.300000	0.300000	0.222010
302.15	0.013327	0.400000	0.400000	0.254015
303.15	0.016631	0.500000	0.500000	0.258316
304.15	0.019924	0.600000	0.019924	0.238926

Captions (copy-ready):
• **linear:** e_T = (T_K - T_ref) / DeltaT
• **log:** e_T = ln(T_K / T_ref)
• **hybrid (piecewise):** if abs(T_K - T_ref) > tau → log; else → linear
• **smooth_hybrid:** e_T = s*ln(T_K / T_ref) + (1-s)*(T_K - T_ref)/DeltaT
with s = 1 / (1 + exp(-(abs(T_K - T_ref) - tau) / width))

I.2 B) Far-field behavior (extremes and asymmetry)

T_K	e_T_log	e_T_beta	e_T_qlog(alpha=1.0)	e_T_kBT(E_unit=R*T_ref)
1.00	-5.697597	297.150000	-0.689799	-0.996646
5.00	-4.088159	58.630000	-0.676516	-0.983230
10.00	-3.395012	28.815000	-0.660157	-0.966460
50.00	-1.785574	4.963000	-0.538110	-0.832299
100.00	-1.092427	1.981500	-0.403915	-0.664598
150.00	-0.686961	0.987667	-0.285616	-0.496898
200.00	-0.399279	0.490750	-0.179843	-0.329197
298.15	0.000000	0.000000	0.000000	0.000000
596.30	0.693147	-0.500000	0.405465	1.000000
1490.75	1.609438	-0.800000	1.098612	4.000000
2981.50	2.302585	-0.900000	1.704748	9.000000

Tables above are copy-ready and can be used directly for plotting or audits.

Addendum J — Privacy Modes & Inversion Risk (P0/P1/P2)

Purpose. Allow sharing of temperature symbols while managing the risk that `e_T` can be inverted back to `T_K` when anchors are public. Choose one mode per study and declare it in the manifest.

J.1 Modes (pick exactly one)

P0 (raw).
`e_T_raw = e_T`
• Highest fidelity; fully invertible if anchors are known. Use for internal labs or closed partners only.

P1 (quantized).
`e_T_q = step * round( e_T / step )` // `step > 0` declared
• Bucketizes `e_T` to reduce precision of inversion.
• Log lens error hint (relative, small-step): `T_est_rel_err ≤ exp( step / 2 ) − 1 ≈ 0.5 * step`.

P2 (DP-lite).
`e_T_dp = clip( −b , b , e_T + Laplace( 0 , b / eps ) )` // `b > 0`, `eps > 0` declared
• Adds calibrated noise inside a declared bound `[−b, b]`.
• Compose `eps` over time windows; publish `eps` per record and per 24h.
• Caution: “DP-lite” is not a formal privacy proof for all adversaries; do not over-claim.

J.2 Inversion risk (why this matters)

Given public anchors and lens, an observer can reconstruct `T_K`:
• log: `T_K = T_ref * exp( e_T )`
• linear: `T_K = T_ref + DeltaT * e_T`
• beta: `T_K = T_ref / ( 1 + e_T )`
• kBT: `T_K = T_ref + ( E_unit / R ) * e_T`
• qlog: `T_K = T_ref * ( ( 1 + alpha ) * exp( e_T ) − alpha )`

Hence: P0 is invertible; P1/P2 increase uncertainty (quantization or noise). If anchors are withheld, audits become harder—state the trade-off explicitly.

J.3 Anchor disclosure policy (declare once)

A (full). Publish lens and all anchors `{T_ref, DeltaT, E_unit, alpha, tau}`. Maximizes reproducibility; highest inversion risk.
B (bucketed). Publish coarse anchors (e.g., `T_ref` rounded to 1 K) and P1/P2 payloads. Balanced.
C (withheld). Do not publish certain anchors; provide a `manifest_hash` and replay service on request. Lowest risk; reduced auditability and portability.

J.4 Manifest keys (example)

```
privacy: { mode: "P1", step: 0.01 }
privacy: { mode: "P2", b: 1.0, eps: 1.0 }
anchors_policy: "bucketed" // or "full", "withheld"
```

J.5 Validator tips (privacy block)

• If `mode == "P1"`: require `step > 0`; verify quantization by golden vectors.
• If `mode == "P2"`: require `b > 0` and `eps > 0`; spot-check empirical variance `≈ 2 * (b/eps)^2` within bound; verify clipping.
• If `anchors_policy == "withheld"`: require `manifest_hash` present; require reproducibility note describing replay process.
• Reject payloads carrying both raw `e_T` and privacy-processed `e_T_*` simultaneously.

J.6 Operational guidance

• For public dashboards, prefer P1 with `step` in `[0.01, 0.05]`; for sensitive medical/industrial streams, consider P2 with `eps` in `[0.5, 2.0]` and `b` matching the declared `|e_T|` bound.
• When coupling to `g_t` (env-gate), compute `S_t` from the privacy-processed `e_T/a_phase/Q_phase` to avoid leaking raw signals.
• Always note the trade-off: privacy ↑ ⇒ audit precision ↓. Keep unit tests that replay decisions from privacy-processed streams.

J.7 Documentation snippet (copy-paste)

“This stream uses `privacy.mode = P1` with `step = 0.02`. Anchors are bucketed. Decisions and gates are computed on quantized symbols. Inversion to raw temperature is intentionally imprecise. For audit, we provide golden vectors and a replay script that reproduces alerts from the shared symbols only.”

J.8 Do/Don’t

• Do: declare privacy mode, parameters, and anchors policy in the manifest.
• Do: publish golden vectors for reproducibility under the chosen mode.
• Don’t: market SSMT as a privacy system; it is a symbolic representation with optional noise/quantization.
• Don’t: mix lenses within a study to “hide” raw values; use one lens and a declared privacy mode.

Addendum K — Starter Manifests (copy-paste, plain ASCII)

(Three ready-to-use examples; tweak ranges/knobs per deployment. Keys are minimal and self-explanatory.)

K.1 M1 — Earth Weather (freezing-aware, S2)

```
{
  "version": "1.1",
  "compliance_level": "S2",
  "lens": "log",
  "anchors": { "T_ref": 273.15, "DeltaT": null, "E_unit": null, "c_T":
null, "tau": null, "alpha": null },
  "guards": { "T_valid_range_K": [200.0, 330.0], "clamp_a_eps": 1e-6,
"eps_w": 1e-12, "eps_TK": 0.001 },
  "phase_block": {
    "T_m_list": [ { "T_m": 273.15, "DeltaT_m": 10.0, "c_m": 1.0, "T_m_tag":
"water_freeze" } ],
    "k_side": 0.5,
    "rho": 0.80
  },
  "privacy": { "mode": "P1", "step": 0.01 },
  "outputs": { "emit_e_T": true, "emit_a_phase": true, "emit_Q_phase":
true, "emit_a_T": false },
  "gate": {
    "enabled": true,
    "lane": "w1*abs(a_T)+w2*Q_phase+w3*abs(a_phase)",
    "w": { "w1": 0.5, "w2": 0.5, "w3": 0.0 },
    "L": 60,
    "kappa": 1.0,
    "mu": 3.0,
    "rho_g": 0.9,
    "theta_g": 0.2
  },
  "notes": "City/regional weather; S2 emits phase + Q for freeze-risk
logic."
}
```

K.2 M2 — Cryogenics (near-0K safe, S1)

```
{
  "version": "1.1",
  "compliance_level": "S1",
  "lens": "qlog",
  "anchors": { "T_ref": 4.0, "alpha": 0.02, "DeltaT": null, "E_unit": null,
"c_T": null, "tau": null },
  "guards": { "T_valid_range_K": [2.0, 50.0], "clamp_a_eps": 1e-6, "eps_w":
1e-12, "eps_TK": 0.0001 },
  "phase_block": { "T_m_list": [], "k_side": null, "rho": null },
  "privacy": { "mode": "P0" },
  "outputs": { "emit_e_T": true, "emit_a_phase": false, "emit_Q_phase":
false, "emit_a_T": false },
  "gate": { "enabled": false },
  "notes": "Lab-internal contrast only; quantum-safe log avoids undefined
behavior near 0 K."
}
```

K.3 M3 — Wearables/Healthcare (fever-aware, S3)

```
{
  "version": "1.1",
  "compliance_level": "S3",
  "lens": "linear",
  "anchors": { "T_ref": 310.15, "DeltaT": 1.0, "c_T": 1.0, "E_unit": null,
"tau": null, "alpha": null },
  "guards": { "T_valid_range_K": [308.0, 313.0], "clamp_a_eps": 1e-6,
"eps_w": 1e-12, "eps_TK": 0.001 },
  "phase_block": {
    "T_m_list": [ { "T_m": 310.15, "DeltaT_m": 0.5, "c_m": 1.0, "T_m_tag":
"body_baseline" } ],
    "k_side": 1.0,
    "rho": 0.90
  },
  "privacy": { "mode": "P2", "b": 0.5, "eps": 1.0 },
  "outputs": { "emit_e_T": true, "emit_a_phase": true, "emit_Q_phase":
true, "emit_a_T": true },
  "gate": {
    "enabled": true,
    "lane": "w1*abs(a_T)+w2*Q_phase+w3*abs(a_phase)",
    "w": { "w1": 0.0, "w2": 1.0, "w3": 0.0 },
    "L": 120,
    "kappa": 1.0,
    "mu": 3.0,
    "rho_g": 0.9,
    "theta_g": 0.3
  },
  "notes": "Personal devices; fever detection around 37 C with DP-lite
symbols."
}
```

Addendum L — Optional Extensions

Conformance note. Features in Addendum L are optional and do not affect SSMT conformance. S1/S2/S3 behavior is unchanged if these blocks are omitted.

L.1 Symbolic environment bundle (SSM-Env)

Purpose. Fuse multiple zero-centric contrasts into a single bounded environmental dial for control/ML, without altering core SSMT.

Inputs (each zero-centric).
`e_T` (temperature, SSMT); optionally `e_P` (pressure), `e_H` (humidity), `e_R` (radiation), `e_W` (wind), etc. Each `e_X` is computed with its own published lens and anchors.

Fusion (contrast-level).
```
e_env = w_T * e_T + w_P * e_P + w_H * e_H + ...
```
• **Knobs:** `w_*` real; start with non-negative weights and `sum <= 1` for interpretability.

Bounded dial (optional).
```
a_env = tanh( c_env * e_env )
```
• **Knob:** `c_env > 0`.

Gate coupling (optional).
Use `a_env` as an additional lane in Section 3.1:
```
S_t = clip( ... + w_env * abs( a_env ) , 0 , 1 )
```

Manifest block (example).

```
env:
  enabled: true
  channels:
    - { key: "T", lens: "log",    T_ref: 298.15, w: 0.5 }
    - { key: "P", lens: "linear", P_ref: 101325.0, DeltaP: 5000.0, w: 0.3 }
    - { key: "H", lens: "beta",   H_ref: 0.50, w: 0.2 }
  c_env: 1.0
  fuse_rule: "sum"        # sum | custom
```

Emitted payload (optional fields).
```
"e_env": <number, optional>,
"a_env": <number, optional>
```

Validator tips.
• Check all referenced channels publish their lens and anchors.
• Verify `w_*` within declared bounds; if `fuse_rule == "sum"`, recompute `e_env` and `a_env` on a sample to spot-check.

L.2 Uncertainty channel (delta-method)

Purpose. Emit lightweight uncertainty on bounded dials to support risk-aware decisions (for example, alert thresholds with confidence).

Assumptions.
Sensor provides an estimated standard deviation `sigma_T` (Kelvin), or estimate it from short-term variance.

Propagate to `e_T` (delta method).
`var(e_T) ≈ ( d e_T / d T_K )^2 * var(T_K )`
Let `sigma_e = sqrt( var(e_T) ).`

Derivatives by lens.
• **log:** `e_T = ln( T_K / T_ref )` → `d e_T / d T_K = 1 / T_K`
• **linear:** `e_T = ( T_K - T_ref ) / DeltaT` → `d e_T / d T_K = 1 / DeltaT`
• **beta:** `e_T = ( T_ref / T_K ) - 1` → `d e_T / d T_K = - T_ref / T_K^2`
• **kBT:** `e_T = ( R*T_K )/E_unit - ( R*T_ref )/E_unit` → `d e_T / d T_K = R / E_unit`
• **qlog:** `e_T = ln( ( T_K/T_ref + alpha ) / ( 1 + alpha ) )` → `d e_T / d T_K = 1 / ( T_K + alpha * T_ref )`
• **hybrid (piecewise):** use the active branch derivative.
• **smooth_hybrid:**

```
e_T = s * ln( T_K / T_ref ) + ( 1 - s ) * ( T_K - T_ref ) / DeltaT
s = 1 / ( 1 + exp( -( abs( T_K - T_ref ) - tau ) / width ) )
d e_T / d T_K = s*( 1 / T_K ) + ( 1 - s )*( 1 / DeltaT ) + ( d s / d T_K )*( ln( T_K / T_ref ) - ( T_K - T_ref ) / DeltaT )
d s / d T_K = s*( 1 - s )*( sign( T_K - T_ref ) / width )
```

Propagate to `a_T` (bounded).

```
a_T = tanh( c_T * e_T )
d a_T / d e_T = c_T * sech^2( c_T * e_T )
var( a_T ) ≈ ( c_T * sech^2( c_T * e_T ) )^2 * var( e_T )
a_T_std = sqrt( var( a_T ) )
```

Recommended outputs (optional).

```
"uncertainty": {
  "e_T_std": <number, optional>,
  "a_T_std": <number, optional>,
  "method": "delta"
}
```

Minimal recipe (pseudocode).

```
# given T_K, sigma_T, lens, anchors
sigma_e = abs( d_eT_d_TK( T_K, lens, anchors ) ) * sigma_T
a_T = tanh( c_T * e_T )
gain = c_T * ( 1 / cosh( c_T * e_T ) )^2     # sech^2(x) = 1 / cosh^2(x)
a_T_std = abs( gain ) * sigma_e
emit uncertainty block
```

Validator tips.
• Enforce non-negativity: `e_T_std >= 0`, `a_T_std >= 0`.

• If `sigma_T` is missing or implausible, omit the uncertainty block.
• Document `method = "delta"`; if another method is used (for example, bootstrap), name it explicitly.

Addendum M — Extreme Regimes, Space, and Multi-Pivot Survival Bands (informative)

Purpose.
This addendum explains how to apply SSMT safely in extreme environments such as cryogenic lines, orbital systems, deep cold storage, high-heat structural stress, and life-support envelopes (habitats, crew cabins, biomedical). It also describes how to express multiple critical temperature bands at once, and how to keep signals comparable across devices, fleets, cities, and planets.

This addendum is informative. It does not add new mandatory math. It shows how to reuse the existing SSMT blocks (`to_kelvin`, `e_T`, `a_phase`, hysteresis, pooling) in harsher regimes.

M.1 Cryogenic / Vacuum / Deep Cold Ranges

In very low ranges (cryogenic propellant lines, space-borne batteries, superconducting coils, biological payload freezers), absolute temperature `T_K` can approach values where naive Celsius logic becomes unstable. SSMT already supports alternate lenses that remain monotonic, numerically well-behaved, and audit-friendly.

Two common safe lenses are:

1. **Beta-style inverse contrast**
   Use a contrast defined as
   `e_T := (T_ref / T_K) - 1`
   where `T_ref` is a declared positive reference in Kelvin.
   • When `T_K` drops well below `T_ref`, `e_T` grows positive.
   • When `T_K` is close to `T_ref`, `e_T` hovers near 0.
   • This expresses "how far below nominal thermal safety" in a way that exaggerates dangerous cold instead of dangerous heat.
2. **Quantum-safe log lens (log(1+x) form)**
   Use
   `e_T := ln( 1 + (T_K / T_ref) )`
   instead of `e_T := ln( T_K / T_ref )`.
   • This guards against division by extremely small magnitudes or edge noise when `T_K` is near 0 K.

- It preserves monotonic behavior in `T_K`.
- It still yields a unitless `e_T` that can be compared across time and equipment.

In both cases, **you MUST declare the lens and `T_ref` in the manifest** so downstream consumers can interpret and audit it. The rule from the core spec still applies: **pick exactly one lens and keep it fixed for that run, fleet, or study.**

For extreme regimes, `T_ref` should not be arbitrary. It should be physically meaningful (for example: crew habitat nominal cabin temperature, liquid oxygen nominal storage band, or electronics survival band). This lets two different teams talk about "how close are we to unsafe" without needing to send raw °C/°F/K.

M.2 Multi-Pivot Survival Bands (fused phase dial)

Some systems have more than one temperature pivot that matters. Example:
- Human safety band (crew comfort / hypothermia / heat stress).
- Electronics survivability band (avionics throttle / shutdown).
- Fuel or coolant phase-change band (freeze, boil, off-nominal viscosity).
- Structural stress band (rail buckle, sun-kink, panel warp).

SSMT already defines the idea of a bounded phase proximity dial using a `tanh` around a pivot:

```
a_phase := tanh( c_m * ( T_K - T_m ) / DeltaT_m )
```

Where:
- `T_m` is the pivot (for example, water freeze point or fuel gel point).
- `DeltaT_m` is the softness width in Kelvin around that pivot (how quickly we transition from "safe" to "danger").
- `c_m` is a gain factor to tune how steep the response is.
- Output is always in (-1,+1), so it is safe to compare and to fuse.

When you have several pivots that all matter, you can fuse them into a single survival dial:

```
a_phase_fused := tanh( SUM_i( c_m_i * ( T_K - T_m_i ) / DeltaT_m_i ) )
```

Interpretation:
- Each pivot `i` contributes a signed "how close to trouble" term.
- The sum across all pivots is then passed through `tanh(...)`, so the final result is still bounded in (-1,+1).
- Large positive `a_phase_fused` means "we are stressing hot-side limits across one or more mission-critical bands."
- Large negative `a_phase_fused` means "we are stressing cold-side limits (freeze, gel, brittleness, survival risk)."
- Near zero means "inside nominal envelopes for all declared pivots."

Why this matters:
A single scalar that encodes multiple survival envelopes is easier for operations, dashboards, and alert routing.
Instead of lighting up three different red lights ("crew heat stress", "fuel is sludging", "electronics at risk"), you can carry one bounded dial that already encodes which side of safety you are drifting toward (hot or cold). It still preserves auditability because the manifest and pivots are declared.

This is especially valuable in:
• Rail buckling and sun-kink risk, where steel expansion, ballast condition, and sun exposure combine.
• Cold-chain logistics, where "frozen", "tempered", and "too warm for safety" are all different pivots.
• Spacecraft thermal loops, where battery survival, crew comfort, and propellant viscosity are tightly coupled.

Important:
All pivots and gains MUST be declared in the manifest.
Nothing here overrides the core rule: downstream readers must know how `a_phase_fused` was produced. Hidden pivots break auditability.

M.3 Cross-Fleet, Cross-City, Cross-Planet Comparability

The core promise of SSMT is that once you convert to Kelvin, choose a lens, and emit `e_T` (and optionally `a_phase`, `a_phase_fused`, `a_T`, etc.), you can compare signals across arbitrary contexts.

This section explains how to extend that same promise beyond "city to city" into "Earth to Mars", "warehouse to refrigerated truck", and "drone wing to lunar habitat wall."

1. **Declared reference, not silent magic.**
   SSMT does not invent physics or redefine temperature. It standardizes how you *express* thermal conditions as a stable, unitless symbolic layer.
   • `e_T` is a contrast like `e_T := ln( T_K / T_ref )` or one of the alternate safe lenses above.
   • `a_phase` and `a_phase_fused` are bounded, symmetric dials around critical pivots, always in (-1,+1).

Because these are declared, **two different systems can talk about comparable stress levels** even if:
• One system is running at -40 C in Antarctic storage.
• Another is at +38 C in a desert cell tower.
• A third is a Mars habitat wall at -60 C outside and +20 C inside.

Instead of arguing over units, they exchange:
• "This reading is approaching +0.8 in the hot direction"
or
• "We are at -0.7 in the cold direction relative to survival pivot T_m."

2. **Fleet governance and policy.**
   Once signals are comparable, you can define policy in plain symbolic terms.
   Examples:
   • "Escalate if any `a_phase_fused >= +0.75` for more than 10 minutes."
   • "Escalate if any `a_phase_fused <= -0.75` in a cryogenic loop feeding crew life support."
   • "Escalate if `e_T` exceeds a declared excursion band in any data center rack for more than 3 consecutive samples."

This is exactly the kind of cross-border, cross-vendor rule that lets a global operator (or mission control) run consistent safety governance across heterogeneous hardware and climates.

3. **Why this belongs in an addendum.**
   These policies are powerful but they are deployment-specific. Declaring pivots for survival in a space habitat is not the same as declaring pivots for lettuce cold-chain logistics.
   Placing this guidance in an informative addendum keeps the core spec stable and auditable, while giving operators and regulators a template for "planet-to-planet interoperability" without forcing them to reinterpret the math.

M.4 Practical Guidance

1. **You MUST still obey the core commissioning checklist.**
   Even in space or cryogenic use, you must still:

- Convert to Kelvin first.
- Pick and declare one lens per deployment (or per study).
- Publish your pivots `T_m`, widths `DeltaT_m`, and gains `c_m`.
- Declare `T_ref`.
- Declare whether you are emitting just `e_T` (Lite) or also `a_phase`, `a_phase_fused`, hysteresis metrics, and health flags.

2. **You MUST declare any obfuscation.**
   If you offset or otherwise transform `e_T` for privacy (for example, adding a constant bias before publishing externally), you must declare that fact so no one treats the obfuscated stream as raw telemetry.
3. **Do not silently mix different lenses in the same stream.**
   If two subsystems need two different lenses (for example, crew cabin comfort vs cryogenic fuel line), emit them as two separate channels with two separate declared manifests. Do not silently "switch lenses on the fly." That breaks comparability and audit.

Unified Temperature Language for a Shared World

Temperature is everywhere. It decides when a bridge warps, when a runway ices, when a vaccine spoils, when a battery fails, when a cold chain silently drifts out of compliance, when track metal kinks under sun load, when a server room crosses from safe to dangerous, when a habitat wall is livable, and when it is not. These are not isolated engineering events. They are the thermal backbone of public safety, infrastructure reliability, logistics integrity, energy continuity, and human survival.

Yet today, temperature still lives in fragments: local °C vs °F arguments, unaligned alert thresholds, vendor-proprietary dashboards, jurisdiction-specific tolerances, and un-audited "safe ranges" that can't be compared across fleets or regions. The result is confusion, delay, cost, and in many cases, harm.

The goal of this work is to end that fragmentation.

This specification defines a single symbolic temperature layer that can be carried everywhere — across devices, vendors, cities, borders, and even planets — without rewriting physics and without surrendering accountability.

The core signals are intentionally simple, and intentionally auditable:

- **`e_T`** is a declared contrast against a published reference `T_ref`.
  Example: `e_T := ln( T_K / T_ref ).`
  This is unitless and monotonic. After conversion to Kelvin, the meaning of "how far from nominal" is the same in Mumbai, Berlin, a refrigerated truck, a hospital ICU, a data center aisle, a launch pad, or a Mars habitat shell.
- **`a_phase`** is a bounded safety dial around a known pivot such as freeze, boil, gel, warp, or human danger:
  `a_phase := tanh( c_m * ( T_K - T_m ) / DeltaT_m ).`
  Output is always in (-1,+1).
  Negative means "drifting toward cold-side danger" (brittle rail, fuel gelling, hypothermia, deep-freeze risk).
  Positive means "drifting toward hot-side danger" (runway softening, structural buckle, biological overheating, battery runaway risk).
  Near zero means "inside the declared band of stability."
- **`a_phase_fused`** generalizes that to more than one survival band at once:
  `a_phase_fused := tanh( SUM_i( c_m_i * ( T_K - T_m_i ) / DeltaT_m_i ) ).`
  Instead of juggling five different red lights ("crew heat stress", "electronics shutdown envelope", "propellant viscosity risk", "rail expansion", "vaccine spoilage"), you can carry one bounded dial that expresses combined survivability pressure in real time.

These signals do not replace physics. They do not guess. They do not excuse negligence. They do one thing: they provide a **shared, auditable language of thermal stress and thermal safety.**

That changes who can act, and how fast.

For operational leaders and regulators.
You can now write policy in one symbolic lane and apply it everywhere.
"Escalate if any asset crosses `a_phase_fused >= +0.75` for more than 10 minutes."
That rule is valid for rail, cold-chain pharma, battery cabinets, ICU wards, runway surfaces, offshore wind gearboxes, and orbital coolant loops. It is not tied to °C vs °F. It is not tied to a single vendor. It is not tied to one jurisdiction. It is transparently enforceable.

For procurement and accountability.
You are no longer forced to trust a black box that says "within limits" without proof. You can demand:

- Emission of `e_T`, `a_phase`, and (if relevant) `a_phase_fused`.
- Publication of `T_ref`, `T_m`, `DeltaT_m`, and `c_m`.
- Declaration of hysteresis rules, smoothing constants, and alert policies.
- Manifest disclosure of lens choice and Kelvin conversion.

This turns "compliance" from a marketing slide into a measurable, timestamped, technically verifiable signal. It also destroys lock-in. If two vendors both emit SSMT-compliant signals, you can compare them directly.

For cities, infrastructure, and civil protection.
Road icing, bridge stress, track buckling, heatwave triage, vaccine spoilage, grid cooling failures, pipe freeze, data center excursions — these are not separate emergencies. They are manifestations of the same thermal instability expressed in different local languages.
With a shared symbolic layer, a city can rank these threats, assign resources, escalate where human life is at risk, justify those actions, and defend them.

For aerospace, deep cold, and habitation.
In cryogenic lines, in crew habitats, in high-radiation orbit, in polar storage, the question is simple: are we drifting out of survivable range, and on which side — cold collapse or thermal runaway?
With `e_T`, `a_phase`, and `a_phase_fused`, that question has a numeric answer that can be transmitted, logged, governed, and acted on without translation fights. The same safety logic you use in a ground warehouse can be projected to an off-world habitat, because the symbolic signal is consistent.

For finance, insurance, and risk.
Thermal instability is not just a maintenance problem. It is a financial exposure.
When out-of-band excursions become machine-readable and standardized across fleets, they become auditable.
When they become auditable, they become insurable, priceable, and contract-bound.
That is how safety stops being "best effort" and becomes part of regulated continuity.

For humanity.
Heat stroke in uncooled housing. Spoiled medicine in rural clinics. Runway overruns. Bridge failures. Cold-chain collapse in food routes. Lithium systems going unstable.
None of these failures are theoretical. All of them are real, recurring, and global.
A universal, open, verifiable temperature layer is not just an engineering upgrade. It is a public safety intervention.

And it is open.

There is no fee to emit `e_T`.
There is no gatekeeper to compute `a_phase`.
There is no proprietary lock on declaring `T_ref`, `T_m`, `DeltaT_m`, `c_m`, and the hysteresis rules you apply.
Any qualified team — public, private, national, municipal, industrial, research, orbital — can adopt this immediately.

This matters because the challenge we face is not local.
Extreme heat is not local.
Infrastructure fatigue is not local.
Cold-chain fragility is not local.
Orbital survivability is not local.

Leadership, today, means being able to see and govern all of that at once.

This specification exists to make that possible — without rewriting physics, without hiding responsibility, and without silencing the truth in the numbers.

OMP